

dtr-implementation.patch文件分析

GitHub地址: https://github.com/uwsampl/dtr-prototype/blob/master/dtr_code/dtr-implementation.patch

 - 无关紧要的改动

! - 比较重要的改动

!! - 非常重要的改动

`更新 .gitmodules

修改内容: 添加了一个名为 `json` 的子模块的配置信息。

```
1 diff --git a/.gitmodules b/.gitmodules
2 index 509ab94f1c..ceb096acc9 100644
3 --- a/.gitmodules
4 +++ b/.gitmodules
5 @@ -130,3 +130,6 @@
6     ignore = dirty
7     path = third_party/tensorpipe
8     url = https://github.com/pytorch/tensorpipe.git
9 +[submodule "third_party/json"]
10 +    path = third_party/json
11 +    url = git@github.com:nlohmann/json.git
```

`url = git@github.com:nlohmann/json.git` 这里后面改一下
<https://github.com/nlohmann/json>

`新增 third_party/json

```
1 diff --git a/third_party/json b/third_party/json
2 new file mode 160000
3 index 000000000000..19843b038c
4 --- /dev/null
5 +++ b/third_party/json
6 @@ -0,0 +1 @@
```

```
7 +Subproject commit 19843b038caa463164d6f89ea1b2765fae7552e9
```

新增 test.py

```
1 diff --git a/test.py b/test.py
2 new file mode 100644
3 index 000000000000..9a6ee104db
4 --- /dev/null
5 +++ b/test.py
6 @@ -0,0 +1,4 @@
7 +import torch
8 +x = torch.Tensor([1]).checkpoint()
9 +y = x
10 +z = y
```

修改 torch/nn/functional.py 解决 multi_head_attention_forward 维度不匹配错误

```
1 diff --git a/torch/nn/functional.py b/torch/nn/functional.py
2 index 916fab5e6e..f7e8385e86 100644
3 --- a/torch/nn/functional.py
4 +++ b/torch/nn/functional.py
5 @@ -4082,9 +4082,10 @@ def multi_head_attention_forward(query,
6         # type: Tensor
7
8         q = q.contiguous().view(tgt_len, bsz * num_heads, head_dim).transpose(0,
9         1)
10         if k is not None:
11             - k = k.contiguous().view(-1, bsz * num_heads, head_dim).transpose(0,
12             1)
13             + # extremely bizarre, but got errors of dimension mismatch here and
14             + below unless I changed view -> reshape 非常奇怪，但是这里和下面都会出现尺寸不匹配的错误，除非我更改视图 -> reshape
15             + k = k.contiguous().reshape(-1, bsz * num_heads,
16             head_dim).transpose(0, 1)
17         if v is not None:
18             - v = v.contiguous().view(-1, bsz * num_heads, head_dim).transpose(0,
19             1)
20             + v = v.contiguous().reshape(-1, bsz * num_heads,
21             head_dim).transpose(0, 1)
```

```

15
16     if static_k is not None:
17         assert static_k.size(0) == bsz * num_heads
18     @@ -4135,7 +4136,8 @@ def multi_head_attention_forward(query,
        # type: Tensor
19
20     attn_output = torch.bmm(attn_output_weights, v)
21     assert list(attn_output.size()) == [bsz * num_heads, tgt_len, head_dim]
22 -     attn_output = attn_output.transpose(0, 1).contiguous().view(tgt_len,
        bsz, embed_dim)
23 +     # also had to change view to reshape 也不得不改变视图来重塑
24 +     attn_output = attn_output.transpose(0, 1).contiguous().reshape(tgt_len,
        bsz, embed_dim)
25     attn_output = linear(attn_output, out_proj_weight, out_proj_bias)
26
27     if need_weights:

```

笔记

在PyTorch中，`torch.nn.functional` 模块提供了一系列的神经网络函数，用于构建神经网络的各种操作和层。关于你提到的这三个文件：

```

1  |— functional.py
2  |— functional.pyi
3  |— functional.pyi.in

```

1. **functional.py**: 这是实际的Python源代码文件，包含了 `torch.nn.functional` 模块的实现。它包含了各种函数的定义，例如激活函数、损失函数、卷积函数等等。
2. **functional.pyi**: 这个文件是类型注释文件（Type Annotations），在编译器静态类型检查和类型推断时使用。这个文件中包含了函数的签名和参数类型信息，帮助IDE和编译器更好地理解函数的使用和调用。
3. **functional.pyi.in**: 这个文件是用来生成 `functional.pyi` 的模板文件，其中包含了函数签名的大概结构和注释。它不是实际的代码文件，而是作为一个模板，用来自动生成 `functional.pyi` 文件。

!修改 `torch/csrc/utils/tensor_numpy.cpp` 将 PyTorch 的 Tensor 对象转换为 NumPy 数组对象

```

1  diff --git a/torch/csrc/utils/tensor_numpy.cpp

```

```

b/torch/csrc/utils/tensor_numpy.cpp
2  index 879618f652..7b96cd4104 100644
3  --- a/torch/csrc/utils/tensor_numpy.cpp
4  +++ b/torch/csrc/utils/tensor_numpy.cpp
5  @@ -73,7 +73,8 @@ static std::vector<int64_t> seq_to_aten_shape(PyObject
    *py_seq) {
6      return result;
7  }
8
9  -PyObject* tensor_to_numpy(const at::Tensor& tensor) {
10 +PyObject* tensor_to_numpy(const at::Tensor& tensor_) {
11 +  Tensor tensor = tensor_.decheckpoint();
12     if (tensor.device().type() != DeviceType::CPU) {
13         throw TypeError(
14             "can't convert %s device type tensor to numpy. Use Tensor.cpu() to "

```

!修改 `tools/codegen/gen.py` 根据用户提供的选项生成不同后端 (backends) 的类型和方法文件

```

1  diff --git a/tools/codegen/gen.py b/tools/codegen/gen.py
2  index e4acb369f0..bcc3da690f 100644
3  --- a/tools/codegen/gen.py
4  +++ b/tools/codegen/gen.py
5  @@ -967,7 +967,7 @@ def main() -> None:
6      #include <ATen/hip/HIPDevice.h>
7      #include <ATen/hip/HIPContext.h>'''
8
9  -     backends = ["CPU", "SparseCPU", "MkldnnCPU", "CUDA", "SparseCUDA",
10 +     backends = ["CPU", "SparseCPU", "MkldnnCPU", "CUDA", "SparseCUDA",
11 +     "QuantizedCPU", "QuantizedCUDA"]
12 +     backends = ["CPU", "SparseCPU", "MkldnnCPU", "CUDA", "SparseCUDA",
13 +     "QuantizedCPU", "QuantizedCUDA", "Checkpoint"]
14     if options.vulkan:
15         backends.append("Vulkan")
16     if options.backend_whitelist:

```

!更新 `c10/core/DispatchKey.h` 在枚举类中增加新的成员 `Checkpoint`

```

1  diff --git a/c10/core/DispatchKey.h b/c10/core/DispatchKey.h
2  index b0c75a261c..2cb80c3ddb 100644

```

```

3  --- a/c10/core/DispatchKey.h
4  +++ b/c10/core/DispatchKey.h
5  @@ -187,6 +187,9 @@ enum class DispatchKey : uint8_t {
6      // constituent parts.
7      Named,
8
9  + // Checkpoint must go after Autograd. This way, Autograd will hook ad
10 outside of CheckpointTensor.
11 + Checkpoint,
12 +
13 // Note [Alias Dispatch Key : Autograd]
14 // All backends are oblivious to autograd; autograd is handled as a
15 // layer which happens on top of all backends. It inspects the autograd

```

!更新 c10/core/DispatchKey.cpp 新增用于处理 DispatchKey::Checkpoint 的case语句

```

1  diff --git a/c10/core/DispatchKey.cpp b/c10/core/DispatchKey.cpp
2  index be8efbbe09..ab2a19f75c 100644
3  --- a/c10/core/DispatchKey.cpp
4  +++ b/c10/core/DispatchKey.cpp
5  @@ -79,6 +79,8 @@ const char* toString(DispatchKey t) {
6      return "AutogradPrivateUse3";
7      case DispatchKey::AutogradOther:
8          return "AutogradOther";
9  + case DispatchKey::Checkpoint:
10 +     return "Checkpoint";
11      case DispatchKey::BackendSelect:
12          return "BackendSelect";
13      case DispatchKey::Named:

```

!修改 c10/core/Backend.h 为 Backend 添加了新的后端类型 Checkpoint

```

1  diff --git a/c10/core/Backend.h b/c10/core/Backend.h
2  index ab0c45ad1f..6afb9ce6a8 100644
3  --- a/c10/core/Backend.h
4  +++ b/c10/core/Backend.h
5  @@ -39,11 +39,48 @@ enum class Backend {
6      Vulkan,

```

```

7     QuantizedCPU,
8     QuantizedCUDA,
9 +   Checkpoint,
10    Undefined,
11    MklDnnCPU,
12    NumOptions
13 };
14
15 +// TODO: This probably shouldn't actually be static inline
16 +static inline const char* toString(Backend b) {
17 +   switch (b) {
18 +   case Backend::CPU:
19 +       return "CPU";
20 +   case Backend::CUDA:
21 +       return "CUDA";
22 +   case Backend::HIP:
23 +       return "HIP";
24 +   case Backend::FPGA:
25 +       return "FPGA";
26 +   case Backend::MSNPU:
27 +       return "MSNPU";
28 +   case Backend::XLA:
29 +       return "XLA";
30 +   case Backend::SparseCPU:
31 +       return "SparseCPU";
32 +   case Backend::SparseCUDA:
33 +       return "SparseCUDA";
34 +   case Backend::SparseHIP:
35 +       return "SparseHIP";
36 +   case Backend::MklDnnCPU:
37 +       return "MklDnnCPU";
38 +   case Backend::Vulkan:
39 +       return "Vulkan";
40 +   case Backend::QuantizedCPU:
41 +       return "QuantizedCPU";
42 +   case Backend::QuantizedCUDA:
43 +       return "QuantizedCUDA";
44 +   case Backend::Checkpoint:
45 +       return "Checkpoint";
46 +   default:
47 +       return "UNKNOWN_BACKEND";
48 +   }
49 +}
50 +
51 static inline Backend toSparse(Backend b) {
52     switch (b) {
53         case Backend::CPU:

```

```

54 @@ -59,7 +96,7 @@ static inline Backend toSparse(Backend b) {
55     case Backend::SparseHIP:
56         return Backend::SparseHIP;
57     default:
58 -         throw std::runtime_error("Unknown backend");
59 +         throw std::runtime_error(std::string("Unknown backend: ") +
60 + to_string(b));
61     }
62 }
63 @@ -88,7 +125,7 @@ static inline Backend toDense(Backend b) {
64     case Backend::QuantizedCUDA:
65         return Backend::QuantizedCUDA;
66     default:
67 -         throw std::runtime_error("Unknown backend");
68 +         throw std::runtime_error(std::string("Unknown backend: ") +
69 + to_string(b));
70     }
71 }
72 @@ -121,6 +158,8 @@ static inline Backend dispatchKeyToBackend(DispatchKey t)
73 {
74     return Backend::QuantizedCUDA;
75 } else if (t == DispatchKey::Undefined) {
76     return Backend::Undefined;
77 + } else if (t == DispatchKey::Checkpoint) {
78 +     return Backend::Checkpoint;
79 } else {
80     AT_ERROR("Unrecognized tensor type ID: ", t);
81 }
82 @@ -154,10 +193,12 @@ static inline DispatchKey backendToDispatchKey(Backend
83 b) {
84     return DispatchKey::QuantizedCPU;
85     case Backend::QuantizedCUDA:
86         return DispatchKey::QuantizedCUDA;
87 + case Backend::Checkpoint:
88 +     return DispatchKey::Checkpoint;
89     case Backend::Undefined:
90         return DispatchKey::Undefined;
91     default:
92 -         throw std::runtime_error("Unknown backend");
93 +         throw std::runtime_error(std::string("Unknown backend: ") +
94 + to_string(b));
95     }
96 }
97 }
98 @@ -191,7 +232,7 @@ static inline DeviceType backendToDeviceType(Backend b) {

```

```

96     case Backend::Undefined:
97         AT_ERROR("Undefined backend is not a valid device type");
98     default:
99 -         AT_ERROR("Unknown backend");
100 +         AT_ERROR(std::string("Unknown backend: ") + toString(b));
101     }
102 }
103
104 @@ -223,7 +264,7 @@ static inline Backend backendToCPU(Backend b) {
105     case Backend::Undefined:
106         return Backend::Undefined;
107     default:
108 -         AT_ERROR("Unknown backend");
109 +         AT_ERROR(std::string("Unknown backend: ") + toString(b));
110     }
111 }
112
113 @@ -243,7 +284,7 @@ static inline Backend backendToCUDA(Backend b) {
114     case Backend::Undefined:
115         return Backend::Undefined;
116     default:
117 -         AT_ERROR("Unknown backend");
118 +         AT_ERROR(std::string("Unknown backend: ") + toString(b));
119     }
120 }
121
122 @@ -267,40 +308,6 @@ static inline Backend backendToHIP(Backend b) {
123     }
124 }
125
126 -// TODO: This probably shouldn't actually be static inline
127 -static inline const char* toString(Backend b) {
128 -    switch (b) {
129 -        case Backend::CPU:
130 -            return "CPU";
131 -        case Backend::CUDA:
132 -            return "CUDA";
133 -        case Backend::HIP:
134 -            return "HIP";
135 -        case Backend::FPGA:
136 -            return "FPGA";
137 -        case Backend::MSNPU:
138 -            return "MSNPU";
139 -        case Backend::XLA:
140 -            return "XLA";
141 -        case Backend::SparseCPU:
142 -            return "SparseCPU";

```



```

143 -     case Backend::SparseCUDA:
144 -         return "SparseCUDA";
145 -     case Backend::SparseHIP:
146 -         return "SparseHIP";
147 -     case Backend::MkldnnCPU:
148 -         return "MkldnnCPU";
149 -     case Backend::Vulkan:
150 -         return "Vulkan";
151 -     case Backend::QuantizedCPU:
152 -         return "QuantizedCPU";
153 -     case Backend::QuantizedCUDA:
154 -         return "QuantizedCUDA";
155 -     default:
156 -         return "UNKNOWN_BACKEND";
157 - }
158 -}
159 -
160 static inline bool isSparse(Backend b) {
161     switch (b) {
162         case Backend::SparseCPU:

```

!修改 `c10/core/TensorImpl.h` 确保存在 Checkpoint 分发键的情况下，不会执行浅拷贝操作

```

1  diff --git a/c10/core/TensorImpl.h b/c10/core/TensorImpl.h
2  index 80917e5745..c792dc8a19 100644
3  --- a/c10/core/TensorImpl.h
4  +++ b/c10/core/TensorImpl.h
5  @@ -493,6 +493,10 @@ struct C10_API TensorImpl : public
6  c10::intrusive_ptr_target {
7  +     return *device_opt_;
8  + }
9  +
10 + c10::optional<c10::Device> optional_device() const {
11 +     return device_opt_;
12 + }
13 +
14 Layout layout() const {
15     // NB: This method is not virtual and avoid dispatches for perf.
16     if (is_sparse()) {
17 @@ -910,6 +914,12 @@ struct C10_API TensorImpl : public
18 c10::intrusive_ptr_target {
19     * compatible with SparseCUDA.

```

```

18     */
19     inline bool has_compatible_shallow_copy_type(DispatchKeySet from) {
20 +     if (key_set_ == from) {
21 +         return true;
22 +     }
23 +     if (key_set_.has(DispatchKey::Checkpoint) ||
        from.has(DispatchKey::Checkpoint)) {
24 +         return false;
25 +     }
26     auto is_dense = [](DispatchKeySet ts) {
27         return ts.has(DispatchKey::CPU) ||
28             ts.has(DispatchKey::CUDA) ||
29 @@ -920,7 +930,7 @@ struct C10_API TensorImpl : public
        c10::intrusive_ptr_target {
30             ts.has(DispatchKey::SparseCUDA) ||
31             ts.has(DispatchKey::SparseHIP);
32     };
33 -     return (key_set_ == from) || (is_dense(key_set_) && is_dense(from)) ||
        (is_sparse(key_set_) && is_sparse(from));
34 +     return (is_dense(key_set_) && is_dense(from)) || (is_sparse(key_set_) &&
        is_sparse(from));
35     }
36
37     /**

```

!更新 `aten/src/ATen/templates/TensorBody.h` 和 `c10/core/TensorImpl.h` 中第一处修改有关系

```

1  diff --git a/aten/src/ATen/templates/TensorBody.h
    b/aten/src/ATen/templates/TensorBody.h
2  index 44f0c9561a..98320dd638 100644
3  --- a/aten/src/ATen/templates/TensorBody.h
4  +++ b/aten/src/ATen/templates/TensorBody.h
5  @@ -310,6 +310,9 @@ class CAFFE2_API Tensor {
6      /// Returns a `Tensor`'s device.
7      Device device() const;
8
9  +    /// Returns a `Tensor`'s device.
10 +    c10::optional<Device> optional_device() const;
11 +
12     /// Returns a `Tensor`'s device index.
13     int64_t get_device() const;

```

!更新

aten/src/ATen/templates/TensorMethods.cpp 和
aten/src/ATen/templates/TensorBody.h 中的修改有
关系

```
1 diff --git a/aten/src/ATen/templates/TensorMethods.cpp
  b/aten/src/ATen/templates/TensorMethods.cpp
2 index 064f5911cb..aed046b8b6 100644
3 --- a/aten/src/ATen/templates/TensorMethods.cpp
4 +++ b/aten/src/ATen/templates/TensorMethods.cpp
5 @@ -60,6 +60,10 @@ Device Tensor::device() const {
6     return impl_->device();
7 }
8
9 +c10::optional<Device> Tensor::optional_device() const {
10 + return impl_->optional_device();
11 +}
12 +
13 int64_t Tensor::get_device() const {
14     // NB: this is not a native function to avoid dispatching overhead.
15     return impl_->get_device();
```

`新增` aten/src/ATen/Logger.h 定义了一个用于记录日志
的结构体 DTRLogger 和一系列用于记录不同类型日志的函
数。

```
1 diff --git a/aten/src/ATen/Logger.h b/aten/src/ATen/Logger.h
2 new file mode 100644
3 index 0000000000..7e3c01a9af
4 --- /dev/null
5 +++ b/aten/src/ATen/Logger.h
6 @@ -0,0 +1,197 @@
7 +#pragma once
8 +
9 +#include <string>
10 +#include <iostream>
11 +#include <../third_party/json/single_include/nlohmann/json.hpp>
12 +
13 +namespace at {
```

```

14 +
15 +struct DTRLogger {
16 + std::string time_prefix;
17 + std::ofstream out;
18 + static std::string get_time_prefix() {
19 +     std::time_t t = std::time(nullptr);
20 +     std::tm* tm = std::localtime(&t);
21 +     return
22 +         std::to_string(1900+tm->tm_year) + "-" +
23 +         std::to_string(1+tm->tm_mon) + "-" +
24 +         std::to_string(tm->tm_mday) + "-" +
25 +         std::to_string(tm->tm_hour) + "-" +
26 +         std::to_string(tm->tm_min) + "-" +
27 +         std::to_string(tm->tm_sec);
28 + }
29 + std::string get_filename(const std::string& name) {
30 +     return time_prefix + "-" + name + ".log";
31 + }
32 + DTRLogger() : time_prefix(get_time_prefix()), out(get_filename("default"))
33 + { }
34 + void log(const std::string& str) {
35 +     out << str << std::endl;
36 + }
37 + static DTRLogger& logger() {
38 +     static DTRLogger ret;
39 +     return ret;
40 + }
41 +};
42 +
43 +using json = nlohmann::json;
44 +constexpr bool log_json = true;
45 +const std::string INSTRUCTION = "INSTRUCTION";
46 +const std::string ANNOTATION = "ANNOTATION";
47 +const std::string RELEASE = "RELEASE";
48 +const std::string PIN = "PIN";
49 +const std::string TIME = "TIME";
50 +const std::string ARGS = "ARGS";
51 +const std::string MEMORY = "MEMORY";
52 +const std::string ALIAS = "ALIAS";
53 +const std::string NAME = "NAME";
54 +const std::string CONSTANT = "CONSTANT";
55 +
56 +void DTRLogConstant(const std::string& name) {
57 +     if (log_json) {
58 +         json j;
59 +         j[INSTRUCTION] = CONSTANT;

```

```

60 +     j[NAME] = name;
61 +     DTRLogger::logger().log(j.dump());
62 + } else {
63 +     DTRLogger::logger().log(CONSTANT + " " + name);
64 + }
65 +}
66 +
67 +void DTRLogMemory(const std::string& name, size_t memory) {
68 +    if (log_json) {
69 +        json j;
70 +        j[INSTRUCTION] = MEMORY;
71 +        j[NAME] = name;
72 +        j[MEMORY] = std::to_string(memory);
73 +        DTRLogger::logger().log(j.dump());
74 +    } else {
75 +        DTRLogger::logger().log(name + " " + MEMORY + ": " +
std::to_string(memory));
76 +    }
77 +}
78 +
79 +void DTRLogAlias(const std::string& name, int index) {
80 +    if (log_json) {
81 +        json j;
82 +        j[INSTRUCTION] = ALIAS;
83 +        j[NAME] = name;
84 +        j[ALIAS] = std::to_string(index);
85 +        DTRLogger::logger().log(j.dump());
86 +    } else {
87 +        DTRLogger::logger().log(name + " " + ALIAS + ": " +
std::to_string(index));
88 +    }
89 +}
90 +
91 +void DTRLogCopyFrom(const std::string& to, const std::string& from) {
92 +    if (log_json) {
93 +        json j;
94 +        j[INSTRUCTION] = "COPY_FROM";
95 +        j["DST"] = to;
96 +        j["SRC"] = from;
97 +        DTRLogger::logger().log(j.dump());
98 +    } else {
99 +        DTRLogger::logger().log(to + " <- " + from);
100 +    }
101 +}
102 +
103 +void DTRLogCopy(const std::string& new_name, const std::string& old_name) {
104 +    if (log_json) {

```

```

105 +     json j;
106 +     j[INSTRUCTION] = "COPY";
107 +     j["DST"] = new_name;
108 +     j["SRC"] = old_name;
109 +     DTRLogger::logger().log(j.dump());
110 + } else {
111 +     DTRLogger::logger().log(new_name + " = " + old_name);
112 + }
113 +}
114 +
115 +void DTRLogMutate(const std::string& name,
116 +                  const std::vector<std::string>& args,
117 +                  const std::vector<size_t>& mutate,
118 +                  const std::string& time) {
119 +     if (log_json) {
120 +         json j;
121 +         j[INSTRUCTION] = "MUTATE";
122 +         j[NAME] = name;
123 +         j[ARGS] = args;
124 +         j["MUTATE"] = mutate;
125 +         j[TIME] = time;
126 +         DTRLogger::logger().log(j.dump());
127 +     } else {
128 +         std::string log = name;
129 +         log += "(";
130 +         for (const auto& s : args) {
131 +             log += s;
132 +             log += ", ";
133 +         }
134 +         log += ") ";
135 +         log += " MUTATING: ";
136 +         log += "(";
137 +         for (const size_t i : mutate) {
138 +             log += std::to_string(i);
139 +             log += ", ";
140 +         }
141 +         log += ") ";
142 +         log += TIME;
143 +         log += ": ";
144 +         log += time;
145 +         DTRLogger::logger().log(log);
146 +     }
147 +}
148 +
149 +void DTRLogRelease(const std::string& name) {
150 +     if (log_json) {
151 +         json j;

```

```

152 +     j[INSTRUCTION] = RELEASE;
153 +     j[NAME] = name;
154 +     DTRLogger::logger().log(j.dump());
155 + } else {
156 +     DTRLogger::logger().log(RELEASE + ": " + name);
157 + }
158 +}
159 +
160 +void DTRLogPin(const std::string& name) {
161 +    if (log_json) {
162 +        json j;
163 +        j[INSTRUCTION] = PIN;
164 +        j[NAME] = name;
165 +        DTRLogger::logger().log(j.dump());
166 +    } else {
167 +        DTRLogger::logger().log(RELEASE + ": " + name);
168 +    }
169 +}
170 +
171 +void DTRLogCall(const std::vector<std::string>& res,
172 +                const std::string& name,
173 +                const std::vector<std::string>& args,
174 +                const std::string& time) {
175 +    if (log_json) {
176 +        json j;
177 +        j[INSTRUCTION] = "CALL";
178 +        j[NAME] = name;
179 +        j["RESULT"] = res;
180 +        j[ARGS] = args;
181 +        j[TIME] = time;
182 +        DTRLogger::logger().log(j.dump());
183 +    } else {
184 +        std::string arg = name + "(";
185 +        for (const auto& s : args) {
186 +            arg += s;
187 +            arg += ", ";
188 +        }
189 +        arg += ")";
190 +        std::string log = "(";
191 +        for (const auto& s: res) {
192 +            log += s;
193 +            log += ", ";
194 +        }
195 +        log += ") = ";
196 +        log += arg;
197 +        log += " TIME: ";
198 +        log += time;

```

```
199 +     DTRLogger::logger().log(log);
200 + }
201 +}
202 +
203 +}
```

!!修改

aten/src/ATen/native/native_functions.yaml

DTR

```
1  diff --git a/aten/src/ATen/native/native_functions.yaml
    b/aten/src/ATen/native/native_functions.yaml
2  index dcf349324e..6d54e1cc9c 100644
3  --- a/aten/src/ATen/native/native_functions.yaml
4  +++ b/aten/src/ATen/native/native_functions.yaml
5  @@ -5,6 +5,90 @@
6  # representing ScalarType's. They are now superseded by usage of
7  # `aten::to()`. The ops remain here for backward compatibility purposes.
8
9  +- func: checkpoint(Tensor self) -> Tensor
10 + variants: method
11 +
12 +# convert a tensor to a checkpoint tensor.
13 +# if the input is already a checkpoint tensor,
14 +# checkpoint will fail while try_checkpoint will
15 +# simply return the input.
16 +- func: try_checkpoint(Tensor self) -> Tensor
17 + variants: method
18 +
19 +- func: is_checkpoint(Tensor self) -> bool
20 + variants: method
21 +
22 +# convert checkpointed tensor into normal tensor.
23 +# uncheckpoint assume the input is checkpointed and will fail otherwise.
24 +# decheckpoint return the input if it is not checkpointed.
25 +- func: uncheckpoint(Tensor self) -> Tensor
26 + variants: method
27 +
28 +- func: decheckpoint(Tensor self) -> Tensor
29 + variants: method
30 +
31 +- func: pin(Tensor(a!) self) -> ()
```



```
32 + variants: method
33 +
34 +- func: new_log(str logname) -> ()
35 + variants: function
36 +
37 +- func: annotate_log(str logname) -> ()
38 + variants: function
39 +
40 +- func: toggle_log(bool use_log) -> ()
41 + variants: function
42 +
43 +- func: clear_checkpointpool() -> ()
44 + variants: function
45 +
46 +- func: set_memory_budget(int budget) -> ()
47 + variants: function
48 +
49 +- func: unset_memory_budget() -> ()
50 + variants: function
51 +
52 +- func: toggle_sampling(bool sample) -> ()
53 + variants: function
54 +
55 +- func: toggle_ignore_small_tensors(bool ignore) -> ()
56 + variants: function
57 +
58 +- func: reset_profile() -> ()
59 + variants: function
60 +
61 +# the compute time of kernel, ignoring remat
62 +- func: base_compute_time() -> int
63 + variants: function
64 +
65 +# the compute time of remat kernel
66 +- func: remat_compute_time() -> int
67 + variants: function
68 +
69 +# base_compute_time() + remat_compute_time()
70 +- func: compute_time() -> int
71 + variants: function
72 +
73 +# total time spent in searching.
74 +- func: search_time() -> int
75 + variants: function
76 +
77 +# total time spent in evaluating cost
78 +- func: cost_time() -> int
```

```

79 + variants: function
80 +
81 + # time spent looping over tensors. search_time() - cost_time()
82 +- func: loop_time() -> int
83 + variants: function
84 +
85 +- func: toggle_profile(bool profile) -> ()
86 + variants: function
87 +
88 + # Temporary type cast operators. These are needed to trace type-casts now
    since
89 + # Type's are not supported in the IR. Instead, we call down to these
90 + # specialized operators for each datatype.
91 + # TODO: remove when we have Type support in the IR
92 +
93   # DEPRECATED. DO NOT USE
94   - func: _cast_Byte(Tensor self, bool non_blocking=False) -> Tensor
95     use_c10_dispatcher: full
96 @@ -131,11 +215,13 @@
97     use_c10_dispatcher: full
98     dispatch:
99       CUDA: _use_cudnn_ctc_loss
100 +   Checkpoint: checkpoint__use_cudnn_ctc_loss
101
102   - func: _cudnn_ctc_loss(Tensor log_probs, Tensor targets, int[]
    input_lengths, int[] target_lengths, int blank, bool deterministic, bool
    zero_infinity) -> (Tensor, Tensor)
103     use_c10_dispatcher: full
104     dispatch:
105       CUDA: _cudnn_ctc_loss
106 +   Checkpoint: checkpoint__cudnn_ctc_loss
107
108   - func: _use_cudnn_rnn_flatten_weight() -> bool
109     use_c10_dispatcher: full
110 @@ -168,12 +254,14 @@
111     variants: function
112     dispatch:
113       CUDA: fused_dropout_cuda
114 +   Checkpoint: checkpoint__fused_dropout
115
116   - func: _masked_scale(Tensor self, Tensor mask, float scale) -> Tensor
117     use_c10_dispatcher: full
118     variants: function
119     dispatch:
120       CUDA: masked_scale_cuda
121 +   Checkpoint: checkpoint__masked_scale
122

```

```

123     - func: _sobol_engine_draw(Tensor quasi, int n, Tensor sobolstate, int
dimension, int num_generated, ScalarType? dtype) -> (Tensor, Tensor)
124     use_c10_dispatcher: full
125 @@ -220,6 +308,9 @@
126     - func: abs(Tensor self) -> Tensor
127     use_c10_dispatcher: full
128     variants: function, method
129 + dispatch:
130 +   CUDA, CPU: abs
131 +   Checkpoint: checkpoint_abs
132
133     - func: abs_(Tensor(a!) self) -> Tensor(a!)
134     use_c10_dispatcher: full
135 @@ -326,6 +417,7 @@
136     CPU, CUDA: add
137     SparseCPU, SparseCUDA: add_sparse
138     MklDnnCPU: mklDnn_add
139 +   Checkpoint: checkpoint_add
140
141     - func: add_.Tensor(Tensor(a!) self, Tensor other, *, Scalar alpha=1) ->
Tensor(a!)
142     use_c10_dispatcher: full
143 @@ -334,6 +426,7 @@
144     CPU, CUDA: add_
145     SparseCPU, SparseCUDA: add_sparse_
146     MklDnnCPU: mklDnn_add_
147 +   Checkpoint: checkpoint_add_
148
149     - func: add.out(Tensor self, Tensor other, *, Scalar alpha=1, Tensor(a!)
out) -> Tensor(a!)
150     dispatch:
151 @@ -363,6 +456,9 @@
152     - func: add.Scalar(Tensor self, Scalar other, Scalar alpha=1) -> Tensor
153     use_c10_dispatcher: full
154     variants: function, method
155 + dispatch:
156 +   CPU, CUDA: add
157 +   Checkpoint: checkpoint_add
158
159     - func: add_.Scalar(Tensor(a!) self, Scalar other, Scalar alpha=1) ->
Tensor(a!)
160     use_c10_dispatcher: full
161 @@ -533,6 +629,7 @@
162     dispatch:
163     CPU, CUDA: as_strided_tensorimpl
164     QuantizedCPU, QuantizedCUDA: as_strided_qtensorimpl
165 +   Checkpoint: checkpoint_as_strided

```

```

166     device_guard: False
167
168     - func: as_strided_(Tensor(a!) self, int[] size, int[] stride, int?
storage_offset=None) -> Tensor(a!)
169 @@ -677,6 +774,7 @@
170     dispatch:
171         CPU: binary_cross_entropy_cpu
172         CUDA: binary_cross_entropy_cuda
173 +     Checkpoint: checkpoint_binary_cross_entropy
174
175     - func: binary_cross_entropy.out(Tensor self, Tensor target, Tensor?
weight=None, int reduction=Mean, *, Tensor(a!) out) -> Tensor(a!)
176     python_module: nn
177 @@ -684,6 +782,7 @@
178     dispatch:
179         CPU: binary_cross_entropy_out_cpu
180         CUDA: binary_cross_entropy_out_cuda
181 +     Checkpoint: checkpoint_binary_cross_entropy_out
182
183     - func: binary_cross_entropy_backward(Tensor grad_output, Tensor self,
Tensor target, Tensor? weight=None, int reduction=Mean) -> Tensor
184     use_c10_dispatcher: full
185 @@ -692,6 +791,7 @@
186     dispatch:
187         CPU: binary_cross_entropy_backward_cpu
188         CUDA: binary_cross_entropy_backward_cuda
189 +     Checkpoint: checkpoint_binary_cross_entropy_backward
190
191     - func: binary_cross_entropy_backward.grad_input(Tensor grad_output, Tensor
self, Tensor target, Tensor? weight=None, int reduction=Mean, *, Tensor(a!)
grad_input) -> Tensor(a!)
192     python_module: nn
193 @@ -699,14 +799,23 @@
194     dispatch:
195         CPU: binary_cross_entropy_backward_out_cpu
196         CUDA: binary_cross_entropy_backward_out_cuda
197 +     Checkpoint: checkpoint_binary_cross_entropy_backward_out
198
199     - func: binary_cross_entropy_with_logits(Tensor self, Tensor target, Tensor?
weight=None, Tensor? pos_weight=None, int reduction=Mean) -> Tensor
200     use_c10_dispatcher: full
201     variants: function
202 + dispatch:
203 +     CPU: binary_cross_entropy_with_logits
204 +     CUDA: binary_cross_entropy_with_logits
205 +     Checkpoint: checkpoint_binary_cross_entropy_with_logits
206

```

```

207 - func: binary_cross_entropy_with_logits_backward(Tensor grad_output, Tensor
self, Tensor target, Tensor? weight=None, Tensor? pos_weight=None, int
reduction=Mean) -> Tensor
208     use_c10_dispatcher: full
209     variants: function
210 + dispatch:
211 +   CPU: binary_cross_entropy_with_logits_backward
212 +   CUDA: binary_cross_entropy_with_logits_backward
213 +   Checkpoint: checkpoint_binary_cross_entropy_with_logits_backward
214
215 - func: bincount(Tensor self, Tensor? weights=None, int minlength=0) ->
Tensor
216     use_c10_dispatcher: full
217 @@ -789,12 +898,7 @@
218     CUDA: bmm_cuda
219     SparseCPU: bmm_sparse_cpu
220     SparseCUDA: bmm_sparse_cuda
221 -
222 -- func: _bmm(Tensor self, Tensor mat2, *, bool deterministic=False) -> Tensor
223 - use_c10_dispatcher: full
224 - variants: function
225 - dispatch:
226 -   SparseCUDA: _bmm_sparse_cuda
227 +   Checkpoint: checkpoint_bmm
228
229 - func: bmm.out(Tensor self, Tensor mat2, *, Tensor(a!) out) -> Tensor(a!)
230     variants: function
231 @@ -804,6 +908,12 @@
232     SparseCPU: bmm_out_sparse_cpu
233     SparseCUDA: bmm_out_sparse_cuda
234
235 +- func: _bmm(Tensor self, Tensor mat2, *, bool deterministic=False) -> Tensor
236 + use_c10_dispatcher: full
237 + variants: function
238 + dispatch:
239 +   SparseCUDA: _bmm_sparse_cuda
240 +
241 - func: _bmm.out(Tensor self, Tensor mat2, *, bool deterministic=False,
Tensor(a!) out) -> Tensor(a!)
242     variants: function
243     dispatch:
244 @@ -858,12 +968,16 @@
245     dispatch:
246     CPU, CUDA: clamp
247     QuantizedCPU: clamp_quantized_cpu
248 +   Checkpoint: checkpoint_clamp
249

```

```

250     - func: clamp_(Tensor(a!) self, Scalar? min=None, Scalar? max=None) ->
Tensor(a!)
251     use_c10_dispatcher: full
252     variants: function, method
253
254     - func: clamp.out(Tensor self, Scalar? min=None, Scalar? max=None, *,
Tensor(a!) out) -> Tensor(a!)
255     + dispatch:
256     +   CPU, CUDA: clamp_out
257     +   Checkpoint: checkpoint_clamp_out
258
259     - func: clamp_max(Tensor self, Scalar max) -> Tensor
260     use_c10_dispatcher: full
261 @@ -882,8 +996,14 @@
262     - func: clamp_min_(Tensor(a!) self, Scalar min) -> Tensor(a!)
263     use_c10_dispatcher: full
264     variants: function, method
265     + dispatch:
266     +   CPU, CUDA: clamp_min_
267     +   Checkpoint: checkpoint_clamp_min_
268
269     - func: clamp_min.out(Tensor self, Scalar min, *, Tensor(a!) out) ->
Tensor(a!)
270     + dispatch:
271     +   CPU, CUDA: clamp_min_out
272     +   Checkpoint: checkpoint_clamp_min_out
273
274     # clip is an alias for clamp
275     - func: clip(Tensor self, Scalar? min=None, Scalar? max=None) -> Tensor
276 @@ -914,6 +1034,10 @@
277     - func: constant_pad_nd(Tensor self, int[] pad, Scalar value=0) -> Tensor
278     use_c10_dispatcher: full
279     variants: function
280     + dispatch:
281     +   CPU: constant_pad_nd
282     +   CUDA: constant_pad_nd
283     +   Checkpoint: checkpoint_constant_pad_nd
284
285     - func: contiguous(Tensor(a) self, *, MemoryFormat
memory_format=contiguous_format) -> Tensor(a)
286     use_c10_dispatcher: full
287 @@ -1020,12 +1144,14 @@
288     use_c10_dispatcher: full
289     dispatch:
290     CUDA: cudnn_batch_norm
291     +   Checkpoint: checkpoint_cudnn_batch_norm
292

```

```

293 # NB: You can only use this if you used cudnn_batch_norm training=True
294 - func: cudnn_batch_norm_backward(Tensor input, Tensor grad_output, Tensor
weight, Tensor? running_mean, Tensor? running_var, Tensor? save_mean, Tensor?
save_var, float epsilon, Tensor reserveSpace) -> (Tensor, Tensor, Tensor)
295     use_c10_dispatcher: full
296     dispatch:
297         CUDA: cudnn_batch_norm_backward
298 +     Checkpoint: checkpoint_cudnn_batch_norm_backward
299
300 - func: cudnn_convolution.deprecated(Tensor self, Tensor weight, Tensor?
bias, int[] padding, int[] stride, int[] dilation, int groups, bool
benchmark, bool deterministic) -> Tensor
301     use_c10_dispatcher: full
302 @@ -1041,21 +1167,25 @@
303     use_c10_dispatcher: full
304     dispatch:
305         CUDA: cudnn_convolution
306 +     Checkpoint: checkpoint_cudnn_convolution
307
308 - func: cudnn_convolution_backward_input(int[] self_size, Tensor
grad_output, Tensor weight, int[] padding, int[] stride, int[] dilation, int
groups, bool benchmark, bool deterministic, bool allow_tf32) -> Tensor
309     use_c10_dispatcher: full
310     dispatch:
311         CUDA: cudnn_convolution_backward_input
312 +     Checkpoint: checkpoint_cudnn_convolution_backward_input
313
314 - func: cudnn_convolution_backward(Tensor self, Tensor grad_output, Tensor
weight, int[] padding, int[] stride, int[] dilation, int groups, bool
benchmark, bool deterministic, bool allow_tf32, bool[2] output_mask) ->
(Tensor, Tensor)
315     use_c10_dispatcher: full
316     dispatch:
317         CUDA: cudnn_convolution_backward
318 +     Checkpoint: checkpoint_cudnn_convolution_backward
319
320 - func: cudnn_convolution_backward_weight(int[] weight_size, Tensor
grad_output, Tensor self, int[] padding, int[] stride, int[] dilation, int
groups, bool benchmark, bool deterministic, bool allow_tf32) -> Tensor
321     use_c10_dispatcher: full
322     dispatch:
323         CUDA: cudnn_convolution_backward_weight
324 +     Checkpoint: checkpoint_cudnn_convolution_backward_weight
325
326 - func: cudnn_convolution_transpose.deprecated(Tensor self, Tensor weight,
Tensor? bias, int[] padding, int[] output_padding, int[] stride, int[]
dilation, int groups, bool benchmark, bool deterministic) -> Tensor

```

```

327     use_c10_dispatcher: full
328 @@ -1071,6 +1201,7 @@
329     use_c10_dispatcher: full
330     dispatch:
331         CUDA: cudnn_convolution_transpose
332 +     Checkpoint: checkpoint_cudnn_convolution_transpose
333
334     # NB: output_padding not strictly needed here, but it's helpful for the float
335     # backwards
336 @@ -1078,16 +1209,19 @@
337     use_c10_dispatcher: full
338     dispatch:
339         CUDA: cudnn_convolution_transpose_backward
340 +     Checkpoint: checkpoint_cudnn_convolution_transpose_backward
341
342     - func: cudnn_convolution_transpose_backward_input(Tensor grad_output,
Tensor weight, int[] padding, int[] stride, int[] dilation, int groups, bool
benchmark, bool deterministic, bool allow_tf32) -> Tensor
343     use_c10_dispatcher: full
344     dispatch:
345         CUDA: cudnn_convolution_transpose_backward_input
346 +     Checkpoint: checkpoint_cudnn_convolution_transpose_backward_input
347
348     - func: cudnn_convolution_transpose_backward_weight(int[] weight_size,
Tensor grad_output, Tensor self, int[] padding, int[] stride, int[] dilation,
int groups, bool benchmark, bool deterministic, bool allow_tf32) -> Tensor
349     use_c10_dispatcher: full
350     dispatch:
351         CUDA: cudnn_convolution_transpose_backward_weight
352 +     Checkpoint: checkpoint_cudnn_convolution_transpose_backward_weight
353
354     # NB: input is special cased in a way I don't quite understand
355     - func: cudnn_grid_sampler(Tensor self, Tensor grid) -> Tensor output
356 @@ -1168,12 +1302,14 @@
357     dispatch:
358         CPU: ctc_loss_cpu
359         CUDA: ctc_loss_gpu
360 +     Checkpoint: checkpoint__ctc_loss
361
362     - func: _ctc_loss_backward(Tensor grad, Tensor log_probs, Tensor targets,
int[] input_lengths, int[] target_lengths, Tensor neg_log_likelihood, Tensor
log_alpha, int blank, bool zero_infinity=False) -> Tensor
363     use_c10_dispatcher: full
364     dispatch:
365         CPU: ctc_loss_backward_cpu
366         CUDA: ctc_loss_backward_gpu
367 +     Checkpoint: checkpoint__ctc_loss_backward

```



```

368
369     - func: diag_embed(Tensor self, int offset=0, int dim1=-2, int dim2=-1) ->
Tensor
370     use_c10_dispatcher: full
371 @@ -1200,6 +1336,7 @@
372     dispatch:
373         CPU, CUDA: div
374         SparseCPU, SparseCUDA: div_sparse
375 +     Checkpoint: checkpoint_div
376
377     - func: div_.Tensor(Tensor(a!) self, Tensor other) -> Tensor(a!)
378     use_c10_dispatcher: full
379 @@ -1207,6 +1344,7 @@
380     dispatch:
381         CPU, CUDA: div_
382         SparseCPU, SparseCUDA: div_sparse_
383 +     Checkpoint: checkpoint_div_
384
385     - func: div.out(Tensor self, Tensor other, *, Tensor(a!) out) -> Tensor(a!)
386     dispatch:
387 @@ -1236,9 +1374,17 @@
388
389     - func: embedding(Tensor weight, Tensor indices, int padding_idx=-1, bool
scale_grad_by_freq=False, bool sparse=False) -> Tensor
390     use_c10_dispatcher: full
391 +     dispatch:
392 +         CPU: embedding
393 +         CUDA: embedding
394 +         Checkpoint: checkpoint_embedding
395
396     - func: embedding_backward(Tensor grad, Tensor indices, int num_weights, int
padding_idx, bool scale_grad_by_freq, bool sparse) -> Tensor
397     use_c10_dispatcher: full
398 +     dispatch:
399 +         CPU: embedding_backward
400 +         CUDA: embedding_backward
401 +         Checkpoint: checkpoint_embedding_backward
402
403     - func: embedding_dense_backward(Tensor grad_output, Tensor indices, int
num_weights, int padding_idx, bool scale_grad_by_freq) -> Tensor
404     use_c10_dispatcher: full
405 @@ -1417,6 +1563,10 @@
406     use_c10_dispatcher: full
407     variants: method # This is method-only to match the previous tensor API.
In the future we could make this a function too.
408     device_guard: False
409 +     dispatch:

```

```

410 + CPU: expand
411 + CUDA: expand
412 + Checkpoint: checkpoint_expand
413
414 - func: expand_as(Tensor(a) self, Tensor other) -> Tensor(a)
415     use_c10_dispatcher: full
416 @@ -1461,10 +1611,18 @@
417 - func: fill_.Scalar(Tensor(a!) self, Scalar value) -> Tensor(a!)
418     use_c10_dispatcher: full
419     variants: function, method
420 + dispatch:
421 + CPU: fill_
422 + CUDA: fill_
423 + Checkpoint: checkpoint_fill_
424
425 - func: fill_.Tensor(Tensor(a!) self, Tensor value) -> Tensor(a!)
426     use_c10_dispatcher: full
427     variants: function, method
428 + dispatch:
429 + CPU: fill_
430 + CUDA: fill_
431 + Checkpoint: checkpoint_fill_
432
433 - func: floor(Tensor self) -> Tensor
434     use_c10_dispatcher: full
435 @@ -1667,6 +1825,10 @@
436
437 - func: index.Tensor(Tensor self, Tensor?[] indices) -> Tensor
438     variants: function, method
439 + dispatch:
440 + CPU: index
441 + CUDA: index
442 + Checkpoint: checkpoint_index
443     # NB: This function is special-cased in tools/autograd/gen_variable_type.py
444     # NB: The following functions are declared in
445     #       aten/src/ATen/templates/TensorBody.h and defined in
446     #       aten/src/ATen/TensorIndexing.cpp:
447     # - Tensor Tensor::index(ArrayRef<TensorIndex> indices)
448 @@ -1688,6 +1850,10 @@
449
450 - func: index_put_(Tensor(a!) self, Tensor?[] indices, Tensor values, bool
451     accumulate=False) -> Tensor(a!)
452     variants: function, method
453 + dispatch:
454 + CPU: index_put_
455 + CUDA: index_put_
456 + Checkpoint: checkpoint_index_put_

```

```

454     # NB: The following functions are declared in
    aten/src/ATen/templates/TensorBody.h and defined in
    aten/src/ATen/TensorIndexing.cpp:
455     # - Tensor & Tensor::index_put_(ArrayRef<TensorIndex> indices, Tensor
    const & rhs)
456     # - Tensor & Tensor::index_put_(ArrayRef<TensorIndex> indices, Scalar v)
457 @@ -1765,12 +1931,16 @@
458
459     - func: kl_div(Tensor self, Tensor target, int reduction=Mean, *, bool
    log_target=False) -> Tensor
460         use_c10_dispatcher: full
461 +   dispatch:
462 +     CPU, CUDA: kl_div
463 +     Checkpoint: checkpoint_kl_div
464
465     - func: kl_div_backward(Tensor grad_output, Tensor self, Tensor target, int
    reduction=Mean, *, bool log_target=False) -> Tensor
466         use_c10_dispatcher: full
467         dispatch:
468             CPU: kl_div_backward_cpu
469             CUDA: kl_div_backward_cuda
470 +     Checkpoint: checkpoint_kl_div_backward
471
472     - func: kthvalue(Tensor self, int k, int dim=-1, bool keepdim=False) ->
    (Tensor values, Tensor indices)
473         use_c10_dispatcher: full
474 @@ -1794,12 +1964,14 @@
475         dispatch:
476             CPU: layer_norm_cpu
477             CUDA: layer_norm_cuda
478 +     Checkpoint: checkpoint_layer_norm
479
480     - func: native_layer_norm_backward(Tensor grad_out, Tensor input, Tensor
    mean, Tensor rstd, Tensor? weight, int M, int N, bool[3] output_mask) ->
    (Tensor, Tensor, Tensor)
481         use_c10_dispatcher: full
482         dispatch:
483             CPU: layer_norm_backward_cpu
484             CUDA: layer_norm_backward_cuda
485 +     Checkpoint: checkpoint_layer_norm_backward
486
487     - func: linear(Tensor input, Tensor weight, Tensor? bias=None) -> Tensor
488         use_c10_dispatcher: full
489 @@ -1846,6 +2018,10 @@
490     - func: log(Tensor self) -> Tensor
491         use_c10_dispatcher: full
492         variants: function, method

```

```

493 + dispatch:
494 +   CPU: log
495 +   CUDA: log
496 +   Checkpoint: checkpoint_log
497
498 - func: log_(Tensor(a!) self) -> Tensor(a!)
499   use_c10_dispatcher: full
500 @@ -1854,6 +2030,7 @@
501 - func: log.out(Tensor self, *, Tensor(a!) out) -> Tensor(a!)
502   dispatch:
503     CPU, CUDA: log_out
504 +   Checkpoint: checkpoint_log_out
505
506 - func: log10(Tensor self) -> Tensor
507   use_c10_dispatcher: full
508 @@ -1932,12 +2109,14 @@
509   dispatch:
510     CPU: log_softmax_cpu
511     CUDA: log_softmax_cuda
512 +   Checkpoint: checkpoint__log_softmax
513
514 - func: _log_softmax_backward_data(Tensor grad_output, Tensor output, int
dim, Tensor self) -> Tensor
515   use_c10_dispatcher: full
516   dispatch:
517     CPU: log_softmax_backward_cpu
518     CUDA: log_softmax_backward_cuda
519 +   Checkpoint: checkpoint__log_softmax_backward_data
520
521 - func: _logcumsumexp(Tensor self, int dim) -> Tensor
522   use_c10_dispatcher: full
523 @@ -2059,6 +2238,7 @@
524   dispatch:
525     CPU, CUDA: mean_cpu_gpu
526     QuantizedCPU: mean_quantized_cpu
527 +   Checkpoint: checkpoint_mean
528
529 - func: mean.dim(Tensor self, int[1] dim, bool keepdim=False, *, ScalarType?
dtype=None) -> Tensor
530   use_c10_dispatcher: full
531 @@ -2066,6 +2246,7 @@
532   dispatch:
533     CPU, CUDA: mean_cpu_gpu
534     QuantizedCPU: mean_quantized_cpu
535 +   Checkpoint: checkpoint_mean
536

```

```

537     - func: mean.out(Tensor self, int[1] dim, bool keepdim=False, *, ScalarType?
dtype=None, Tensor(a!) out) -> Tensor(a!)
538     dispatch:
539     @@ -2211,12 +2392,14 @@
540         CPU: mm_cpu
541         CUDA: mm_cuda
542         SparseCPU, SparseCUDA: _sparse_mm
543     +     Checkpoint: checkpoint_mm
544
545     - func: mm.out(Tensor self, Tensor mat2, *, Tensor(a!) out) -> Tensor(a!)
546     dispatch:
547         CPU: mm_cpu_out
548         CUDA: mm_out_cuda
549         SparseCPU, SparseCUDA: _sparse_mm_out
550     +     Checkpoint: checkpoint_mm_out
551
552     - func: _sparse_mm(Tensor sparse, Tensor dense) -> Tensor
553     use_c10_dispatcher: full
554     @@ -2239,6 +2422,7 @@
555         CPU, CUDA: mul
556         SparseCPU, SparseCUDA: mul_sparse
557         MklDnnCPU: mklDnn_mul
558     +     Checkpoint: checkpoint_mul
559
560     - func: mul_.Tensor(Tensor(a!) self, Tensor other) -> Tensor(a!)
561     use_c10_dispatcher: full
562     @@ -2247,6 +2431,7 @@
563         CPU, CUDA: mul_
564         SparseCPU, SparseCUDA: mul_sparse_
565         MklDnnCPU: mklDnn_mul_
566     +     Checkpoint: checkpoint_mul_
567
568     - func: mul.out(Tensor self, Tensor other, *, Tensor(a!) out) -> Tensor(a!)
569     dispatch:
570     @@ -2259,10 +2444,21 @@
571     - func: mul.Scalar(Tensor self, Scalar other) -> Tensor
572     use_c10_dispatcher: full
573     variants: function, method
574     + dispatch:
575     +     CPU: mul
576     +     CUDA: mul
577     +     SparseCPU: mul
578     +     SparseCUDA: mul
579     +     MklDnnCPU: mul
580     +     Checkpoint: checkpoint_mul
581
582     - func: mul_.Scalar(Tensor(a!) self, Scalar other) -> Tensor(a!)

```

```

583     use_c10_dispatcher: full
584     variants: method
585 +   dispatch:
586 +     CPU: mul_
587 +     CUDA: mul_
588 +     Checkpoint: checkpoint_mul_
589
590 - func: mv(Tensor self, Tensor vec) -> Tensor
591     use_c10_dispatcher: full
592 @@ -2270,8 +2466,12 @@
593     dispatch:
594         CPU, CUDA: mv
595         SparseCPU, SparseCUDA: mv_sparse
596 +     Checkpoint: checkpoint_mv
597
598 - func: mv.out(Tensor self, Tensor vec, *, Tensor(a!) out) -> Tensor(a!)
599 +   dispatch:
600 +     CPU, CUDA: mv_out
601 +     Checkpoint: checkpoint_mv_out
602
603 - func: mvlgamma(Tensor self, int p) -> Tensor
604     use_c10_dispatcher: full
605 @@ -2304,10 +2504,12 @@
606         CPU: batch_norm_cpu
607         CUDA: batch_norm_cuda
608         MklDnnCPU: mklDnn_batch_norm
609 +     Checkpoint: checkpoint_native_batch_norm
610
611 - func: native_batch_norm.out(Tensor input, Tensor? weight, Tensor? bias,
Tensor? running_mean, Tensor? running_var, bool training, float momentum,
float eps, *, Tensor(a!) out, Tensor(b!) save_mean, Tensor(c!) save_invstd) -
> (Tensor(a!), Tensor(b!), Tensor(c!))
612     dispatch:
613         CUDA: batch_norm_cuda_out
614 +     Checkpoint: checkpoint_native_batch_norm_out
615
616 - func: batch_norm_stats(Tensor input, float eps) -> (Tensor, Tensor)
617     use_c10_dispatcher: full
618 @@ -2339,6 +2541,7 @@
619     dispatch:
620         CPU: batch_norm_backward_cpu
621         CUDA: batch_norm_backward_cuda
622 +     Checkpoint: checkpoint_native_batch_norm_backward
623
624 - func: batch_norm_backward_reduce(Tensor grad_out, Tensor input, Tensor
mean, Tensor invstd, Tensor? weight, bool input_g, bool weight_g, bool
bias_g) -> (Tensor, Tensor, Tensor, Tensor)

```

```

625     use_c10_dispatcher: full
626 @@ -2388,6 +2591,9 @@
627
628     - func: ones_like(Tensor self, *, ScalarType? dtype=None, Layout?
        layout=None, Device? device=None, bool? pin_memory=None, MemoryFormat?
        memory_format=None) -> Tensor
629     use_c10_dispatcher: full
630 +   dispatch:
631 +     CPU, CUDA: ones_like
632 +     Checkpoint: checkpoint_ones_like
633
634     - func: pairwise_distance(Tensor x1, Tensor x2, float p=2, float eps=1e-06,
        bool keepdim=False) -> Tensor
635     use_c10_dispatcher: full
636 @@ -2584,6 +2790,10 @@
637     - func: neg(Tensor self) -> Tensor
638     use_c10_dispatcher: full
639     variants: function, method
640 +   dispatch:
641 +     CPU: neg
642 +     CUDA: neg
643 +     Checkpoint: checkpoint_neg
644
645     - func: neg_(Tensor(a!) self) -> Tensor(a!)
646     use_c10_dispatcher: full
647 @@ -2605,6 +2815,10 @@
648     - func: repeat(Tensor self, int[] repeats) -> Tensor
649     use_c10_dispatcher: full
650     variants: method # This is method-only to match the previous tensor API.
        In the future we could make this a function too.
651 +   dispatch:
652 +     CPU: repeat
653 +     CUDA: repeat
654 +     Checkpoint: checkpoint_repeat
655
656     - func: repeat_interleave.Tensor(Tensor repeats) -> Tensor
657     use_c10_dispatcher: full
658 @@ -2661,6 +2875,7 @@
659     CPU, CUDA: relu
660     MklDnnCPU: mklDnn_relu
661     QuantizedCPU: relu_quantized_cpu
662 +   Checkpoint: checkpoint_relu
663
664     - func: relu_(Tensor(a!) self) -> Tensor(a!)
665     use_c10_dispatcher: full
666 @@ -2669,6 +2884,7 @@
667     CPU, CUDA: relu_

```

```

668         MklDnnCPU: mklDnn_relu_
669         QuantizedCPU: relu_quantized_cpu_
670 +     Checkpoint: checkpoint_relu_
671
672 - func: prelu(Tensor self, Tensor weight) -> Tensor
673     use_c10_dispatcher: full
674 @@ -2726,6 +2942,16 @@
675     use_c10_dispatcher: full
676     variants: function, method
677     device_guard: False
678 + dispatch:
679 +     CPU, CUDA: select
680 +     Checkpoint: checkpoint_select
681 +
682 +- func: select_backward(Tensor grad, int[] sizes, int dim, int index) ->
    Tensor
683 + variants: function
684 + device_guard: False
685 + dispatch:
686 +     CPU, CUDA: select_backward
687 +     Checkpoint: checkpoint_select_backward
688
689 - func: selu(Tensor self) -> Tensor
690     use_c10_dispatcher: full
691 @@ -2761,6 +2987,7 @@
692     CPU, CUDA: sigmoid
693     QuantizedCPU: sigmoid_quantized_cpu
694     MklDnnCPU: mklDnn_sigmoid
695 +     Checkpoint: checkpoint_sigmoid
696
697 - func: sigmoid_(Tensor(a!) self) -> Tensor(a!)
698     use_c10_dispatcher: full
699 @@ -2768,6 +2995,7 @@
700     dispatch:
701         CPU, CUDA: sigmoid_
702         MklDnnCPU: mklDnn_sigmoid_
703 +     Checkpoint: checkpoint_sigmoid_
704
705 - func: sigmoid.out(Tensor self, *, Tensor(a!) out) -> Tensor(a!)
706
707 @@ -2842,6 +3070,18 @@
708     use_c10_dispatcher: full
709     variants: function, method
710     device_guard: False
711 + dispatch:
712 +     CPU: slice
713 +     CUDA: slice

```



```

714 +     Checkpoint: checkpoint_slice
715 +
716 +- func: slice_backward(Tensor grad, int[] input_sizes, int dim, int start,
    int end, int step) -> Tensor
717 + variants: function
718 + device_guard: False
719 + dispatch:
720 +     CPU: slice_backward
721 +     CUDA: slice_backward
722 +     Checkpoint: checkpoint_slice_backward
723
724 - func: slogdet(Tensor self) -> (Tensor sign, Tensor logabsdet)
725     use_c10_dispatcher: full
726 @@ -2865,12 +3105,14 @@
727     CPU: softmax_cpu
728     CUDA: softmax_cuda
729     MklDnnCPU: mklDnn_softmax
730 +     Checkpoint: checkpoint__softmax
731
732 - func: _softmax_backward_data(Tensor grad_output, Tensor output, int dim,
    Tensor self) -> Tensor
733     use_c10_dispatcher: full
734     dispatch:
735         CPU: softmax_backward_cpu
736         CUDA: softmax_backward_cuda
737 +     Checkpoint: checkpoint__softmax_backward_data
738
739 - func: unsafe_split.Tensor(Tensor self, int split_size, int dim=0) ->
    Tensor[]
740     use_c10_dispatcher: full
741 @@ -2881,6 +3123,17 @@
742     use_c10_dispatcher: full
743     variants: function, method
744     device_guard: False
745 + dispatch:
746 +     CPU: split
747 +     CUDA: split
748 +     Checkpoint: checkpoint_split
749 +
750 +- func: split_backward(Tensor[] grads, int split_size, int dim, int[] sizes,
    *, ScalarType? dtype=None, Layout? layout=None, Device? device=None, bool?
    pin_memory=None) -> Tensor
751 + variants: function
752 + dispatch:
753 +     CPU: split_backward
754 +     CUDA: split_backward
755 +     Checkpoint: checkpoint_split_backward

```

```

756
757   - func: unsafe_split_with_sizes(Tensor self, int[] split_sizes, int dim=0) -
    > Tensor[]
758     use_c10_dispatcher: full
759 @@ -2891,6 +3144,17 @@
760     use_c10_dispatcher: full
761     variants: function, method
762     device_guard: False
763 +   dispatch:
764 +     CPU: split_with_sizes
765 +     CUDA: split_with_sizes
766 +     Checkpoint: checkpoint_split_with_sizes
767 +
768 +- func: split_with_sizes_backward(Tensor[] grads, int[] split_sizes, int
    dim, int[] sizes, *, ScalarType? dtype=None, Layout? layout=None, Device?
    device=None, bool? pin_memory=None) -> Tensor
769 +   variants: function
770 +   dispatch:
771 +     CPU: split_with_sizes_backward
772 +     CUDA: split_with_sizes_backward
773 +     Checkpoint: checkpoint_split_with_sizes_backward
774
775   - func: squeeze(Tensor(a) self) -> Tensor(a)
776     use_c10_dispatcher: full
777 @@ -2910,15 +3174,27 @@
778     use_c10_dispatcher: full
779     variants: method
780     device_guard: False
781 +   dispatch:
782 +     CPU: squeeze_
783 +     CUDA: squeeze_
784 +     Checkpoint: checkpoint_squeeze_
785
786   - func: squeeze_.dim(Tensor(a!) self, int dim) -> Tensor(a!)
787     use_c10_dispatcher: full
788     variants: method
789     device_guard: False
790 +   dispatch:
791 +     CPU: squeeze_
792 +     CUDA: squeeze_
793 +     Checkpoint: checkpoint_squeeze_
794
795   - func: squeeze_.dimname(Tensor(a!) self, Dimname dim) -> Tensor(a!)
796     variants: method
797     device_guard: False
798 +   dispatch:
799 +     CPU: squeeze_

```

```

800 +     CUDA: squeeze_
801 +     Checkpoint: checkpoint_squeeze_
802
803 - func: sspaddmm(Tensor self, Tensor mat1, Tensor mat2, *, Scalar beta=1,
Scalar alpha=1) -> Tensor
804     use_c10_dispatcher: full
805 @@ -2975,10 +3251,18 @@
806 - func: sum(Tensor self, *, ScalarType? dtype=None) -> Tensor
807     use_c10_dispatcher: full
808     variants: function, method
809 + dispatch:
810 +     CPU: sum
811 +     CUDA: sum
812 +     Checkpoint: checkpoint_sum
813
814 - func: sum.dim_IntList(Tensor self, int[1] dim, bool keepdim=False, *,
ScalarType? dtype=None) -> Tensor
815     use_c10_dispatcher: full
816     variants: function, method
817 + dispatch:
818 +     CPU: sum
819 +     CUDA: sum
820 +     Checkpoint: checkpoint_sum_dim_IntList
821
822 - func: sum.dim_DimnameList(Tensor self, Dimname[1] dim, bool keepdim=False,
*, ScalarType? dtype=None) -> Tensor
823     variants: function, method
824 @@ -3005,6 +3289,10 @@
825 - func: sqrt(Tensor self) -> Tensor
826     use_c10_dispatcher: full
827     variants: function, method
828 + dispatch:
829 +     CPU: sqrt
830 +     CUDA: sqrt
831 +     Checkpoint: checkpoint_sqrt
832
833 - func: sqrt_(Tensor(a!) self) -> Tensor(a!)
834     use_c10_dispatcher: full
835 @@ -3065,6 +3353,9 @@
836     use_c10_dispatcher: full
837     device_guard: False
838     variants: function, method
839 + dispatch:
840 +     CPU, CUDA: t
841 +     Checkpoint: checkpoint_t
842
843 - func: t_(Tensor(a!) self) -> Tensor(a!)

```

```

844     use_c10_dispatcher: full
845 @@ -3087,6 +3378,7 @@
846     dispatch:
847         CPU, CUDA: tanh
848         QuantizedCPU: tanh_quantized_cpu
849 +     Checkpoint: checkpoint_tanh
850
851 - func: tanh_(Tensor(a!) self) -> Tensor(a!)
852     use_c10_dispatcher: full
853 @@ -3106,6 +3398,7 @@
854         CPU: threshold
855         CUDA: threshold_cuda
856         QuantizedCPU: threshold_quantized_cpu
857 +     Checkpoint: checkpoint_threshold
858
859 - func: threshold_(Tensor(a!) self, Scalar threshold, Scalar value) ->
Tensor(a!)
860     use_c10_dispatcher: full
861 @@ -3113,11 +3406,13 @@
862     dispatch:
863         CPU: threshold_
864         CUDA: threshold__cuda
865 +     Checkpoint: checkpoint_threshold_
866
867 - func: threshold.out(Tensor self, Scalar threshold, Scalar value, *,
Tensor(a!) out) -> Tensor(a!)
868     dispatch:
869         CPU: threshold_out
870         CUDA: threshold_out_cuda
871 +     Checkpoint: checkpoint_threshold_out
872
873 - func: threshold_backward(Tensor grad_output, Tensor self, Scalar
threshold) -> Tensor
874     use_c10_dispatcher: full
875 @@ -3125,6 +3420,7 @@
876     dispatch:
877         CPU: threshold_backward
878         CUDA: threshold_backward_cuda
879 +     Checkpoint: checkpoint_threshold_backward
880
881 - func: transpose.int(Tensor(a) self, int dim0, int dim1) -> Tensor(a)
882     use_c10_dispatcher: full
883 @@ -3345,6 +3641,10 @@
884 - func: where.self(Tensor condition, Tensor self, Tensor other) -> Tensor
885     use_c10_dispatcher: full
886     variants: function, method
887 +     dispatch:

```

```

888 + CPU: where
889 + CUDA: where
890 + Checkpoint: checkpoint_where
891
892 - func: where.ScalarSelf(Tensor condition, Scalar self, Tensor other) ->
Tensor
893 use_c10_dispatcher: full
894 @@ -3402,6 +3702,9 @@
895
896 - func: zeros_like(Tensor self, *, ScalarType? dtype=None, Layout?
layout=None, Device? device=None, bool? pin_memory=None, MemoryFormat?
memory_format=None) -> Tensor
897 use_c10_dispatcher: full
898 + dispatch:
899 + CPU, CUDA: zeros_like
900 + Checkpoint: checkpoint_zeros_like
901
902 - func: _standard_gamma_grad(Tensor self, Tensor output) -> Tensor
903 use_c10_dispatcher: full
904 @@ -3567,6 +3870,7 @@
905 SparseCPU, SparseCUDA: clone_sparse
906 MklDnnCPU: mklDnn_clone
907 QuantizedCPU, QuantizedCUDA: quantized_clone
908 + Checkpoint: checkpoint_clone
909
910 - func: resize_as_(Tensor(a!) self, Tensor the_template, *, MemoryFormat?
memory_format=None) -> Tensor(a!)
911 use_c10_dispatcher: full
912 @@ -3591,6 +3895,7 @@
913 CPU, CUDA: zero_
914 SparseCPU, SparseCUDA: zero_sparse_
915 MklDnnCPU: mklDnn_zero_
916 + Checkpoint: checkpoint_zero_
917
918 - func: sub.out(Tensor self, Tensor other, *, Scalar alpha=1, Tensor(a!)
out) -> Tensor(a!)
919 dispatch:
920 @@ -3603,6 +3908,7 @@
921 dispatch:
922 CPU, CUDA: sub
923 SparseCPU, SparseCUDA: sub_sparse
924 + Checkpoint: checkpoint_sub
925
926 - func: sub_.Tensor(Tensor(a!) self, Tensor other, *, Scalar alpha=1) ->
Tensor(a!)
927 use_c10_dispatcher: full
928 @@ -3610,6 +3916,7 @@

```

```

929     dispatch:
930         CPU, CUDA: sub_
931         SparseCPU, SparseCUDA: sub_sparse_
932 +     Checkpoint: checkpoint_sub_
933
934     # For C++ only, until we have conversion from C++ numbers to Tensor
935     - func: sub.Scalar(Tensor self, Scalar other, Scalar alpha=1) -> Tensor
936 @@ -3643,6 +3950,9 @@
937     - func: rsub.Tensor(Tensor self, Tensor other, *, Scalar alpha=1) -> Tensor
938     use_c10_dispatcher: full
939     variants: function
940 + dispatch:
941 +     CPU, CUDA, SparseCPU, SparseCUDA: rsub
942 +     Checkpoint: checkpoint_rsub
943
944     - func: heaviside.out(Tensor self, Tensor values, *, Tensor(a!) out) ->
Tensor(a!)
945     dispatch:
946 @@ -3659,6 +3969,9 @@
947     - func: rsub.Scalar(Tensor self, Scalar other, Scalar alpha=1) -> Tensor
948     use_c10_dispatcher: full
949     variants: function
950 + dispatch:
951 +     CPU, CUDA, SparseCPU, SparseCUDA: rsub
952 +     Checkpoint: checkpoint_rsub
953
954     # Functionally the same as addmm, but we give it a different derivative
formula
955     # that doesn't propagate gradients to non-present entries on sparse.
956 @@ -3671,6 +3984,7 @@
957         CUDA: addmm_out_cuda
958         SparseCPU: addmm_out_sparse_dense_cpu
959         SparseCUDA: addmm_out_sparse_dense_cuda
960 +     Checkpoint: checkpoint_addmm_out
961
962     - func: addmm(Tensor self, Tensor mat1, Tensor mat2, *, Scalar beta=1,
Scalar alpha=1) -> Tensor
963     use_c10_dispatcher: full
964 @@ -3680,6 +3994,7 @@
965         CUDA: addmm_cuda
966         SparseCPU: addmm_sparse_dense_cpu
967         SparseCUDA: addmm_sparse_dense_cuda
968 +     Checkpoint: checkpoint_addmm
969
970     - func: addmm_(Tensor(a!) self, Tensor mat1, Tensor mat2, *, Scalar beta=1,
Scalar alpha=1) -> Tensor(a!)
971     use_c10_dispatcher: full

```

```

972 @@ -3691,6 +4006,8 @@
973     # broadcasting
974     SparseCPU: s_addmm_sparse_dense_cpu_
975     SparseCUDA: s_addmm_sparse_dense_cuda_
976 +     Checkpoint: checkpoint_addmm_
977 +
978
979     # NOTE [ Sparse: autograd and API ]
980     #
981 @@ -4136,6 +4453,9 @@
982     use_c10_dispatcher: full
983     variants: method
984     device_guard: False
985 +     dispatch:
986 +         CPU, CUDA: to
987 +         Checkpoint: checkpoint_to
988
989     - func: to.device(Tensor self, Device device, ScalarType dtype, bool
non_blocking=False, bool copy=False, MemoryFormat? memory_format=None) ->
Tensor
990     use_c10_dispatcher: full
991 @@ -4196,6 +4516,7 @@
992     dispatch:
993         CPU: _local_scalar_dense_cpu
994         CUDA: _local_scalar_dense_cuda
995 +     Checkpoint: checkpoint__local_scalar_dense
996     variants: function
997
998     # Fused RNN kernels
999 @@ -4203,11 +4524,13 @@
1000     use_c10_dispatcher: full
1001     dispatch:
1002         CUDA: _thnn_fused_lstm_cell_cuda
1003 +     Checkpoint: checkpoint__thnn_fused_lstm_cell
1004
1005     - func: _thnn_fused_lstm_cell_backward(Tensor? grad_hy, Tensor? grad_cy,
Tensor cx, Tensor cy, Tensor workspace, bool has_bias) -> (Tensor, Tensor,
Tensor, Tensor, Tensor)
1006     use_c10_dispatcher: full
1007     dispatch:
1008         CUDA: _thnn_fused_lstm_cell_backward_cuda
1009 +     Checkpoint: checkpoint__thnn_fused_lstm_cell_backward
1010
1011     - func: _thnn_differentiable_lstm_cell_backward(Tensor? grad_hy, Tensor?
grad_cy, Tensor input_gates, Tensor hidden_gates, Tensor? input_bias, Tensor?
hidden_bias, Tensor cx, Tensor cy) -> (Tensor, Tensor, Tensor, Tensor, Tensor)
1012     use_c10_dispatcher: full

```

```

1013 @@ -4216,11 +4539,13 @@
1014     use_c10_dispatcher: full
1015     dispatch:
1016         CUDA: _thnn_fused_gru_cell_cuda
1017 +     Checkpoint: checkpoint__thnn_fused_gru_cell
1018
1019     - func: _thnn_fused_gru_cell_backward(Tensor grad_hy, Tensor workspace, bool
has_bias) -> (Tensor, Tensor, Tensor, Tensor, Tensor)
1020     use_c10_dispatcher: full
1021     dispatch:
1022         CUDA: _thnn_fused_gru_cell_backward_cuda
1023 +     Checkpoint: checkpoint__thnn_fused_gru_cell_backward
1024
1025     - func: _thnn_differentiable_gru_cell_backward(Tensor grad_hy, Tensor
input_gates, Tensor hidden_gates, Tensor hx, Tensor? input_bias, Tensor?
hidden_bias) -> (Tensor, Tensor, Tensor, Tensor, Tensor)
1026     use_c10_dispatcher: full
1027 @@ -4350,6 +4675,7 @@
1028     dispatch:
1029         CPU: masked_fill__cpu
1030         CUDA: masked_fill__cuda
1031 +     Checkpoint: checkpoint_masked_fill_
1032
1033     - func: masked_fill.Scalar(Tensor self, Tensor mask, Scalar value) -> Tensor
1034     use_c10_dispatcher: full
1035 @@ -4361,6 +4687,7 @@
1036     dispatch:
1037         CPU: masked_fill__cpu
1038         CUDA: masked_fill__cuda
1039 +     Checkpoint: checkpoint_masked_fill_
1040
1041     - func: masked_fill.Tensor(Tensor self, Tensor mask, Tensor value) -> Tensor
1042     use_c10_dispatcher: full
1043 @@ -4384,6 +4711,7 @@
1044     dispatch:
1045         CPU, CUDA, QuantizedCPU, QuantizedCUDA: view
1046         MklDnnCPU: mklDnn_view
1047 +     Checkpoint: checkpoint_view
1048
1049     - func: put_(Tensor(a!) self, Tensor index, Tensor source, bool
accumulate=False) -> Tensor(a!)
1050     use_c10_dispatcher: full
1051 @@ -4542,11 +4870,13 @@
1052     variants: function
1053     dispatch:
1054         CPU, CUDA: bitwise_and_out
1055 +     Checkpoint: checkpoint_bitwise_and_out

```



```

1056
1057     - func: bitwise_and.Scalar_out(Tensor self, Scalar other, *, Tensor(a!) out)
-> Tensor(a!)
1058     variants: function
1059     dispatch:
1060         CPU, CUDA: bitwise_and_out
1061 +     Checkpoint: checkpoint_bitwise_and_out
1062
1063     - func: bitwise_and.Scalar(Tensor self, Scalar other) -> Tensor
1064     use_c10_dispatcher: full
1065 @@ -4826,6 +5156,9 @@
1066     - func: addcddiv_(Tensor(a!) self, Tensor tensor1, Tensor tensor2, *, Scalar
value=1) -> Tensor(a!)
1067     use_c10_dispatcher: full
1068     variants: method
1069 +     dispatch:
1070 +     CPU, CUDA: addcddiv_
1071 +     Checkpoint: checkpoint_addcddiv_
1072
1073     - func: random_.from(Tensor(a!) self, int from, int? to, *, Generator?
generator=None) -> Tensor(a!)
1074     variants: method
1075 @@ -4857,10 +5190,14 @@
1076     dispatch:
1077         CPU: diag_cpu_out
1078         CUDA: diag_cuda_out
1079 +     Checkpoint: checkpoint_diag_out
1080
1081     - func: diag(Tensor self, int diagonal=0) -> Tensor
1082     use_c10_dispatcher: full
1083     variants: method, function
1084 +     dispatch:
1085 +     CPU, CUDA: diag
1086 +     Checkpoint: checkpoint_diag
1087
1088     - func: cross.out(Tensor self, Tensor other, int? dim=None, *, Tensor(a!)
out) -> Tensor(a!)
1089
1090 @@ -4909,6 +5246,7 @@
1091     dispatch:
1092         CPU, CUDA: ne_out
1093         QuantizedCPU: ne_out_quantized_cpu
1094 +     Checkpoint: checkpoint_ne_Scalar_out
1095
1096     - func: ne.Scalar(Tensor self, Scalar other) -> Tensor
1097     use_c10_dispatcher: full
1098 @@ -4916,11 +5254,13 @@

```

```

1099     dispatch:
1100         CPU, CUDA: ne
1101         QuantizedCPU: ne_quantized_cpu
1102 +     Checkpoint: checkpoint_ne_Scalar
1103
1104 - func: ne.Tensor_out(Tensor self, Tensor other, *, Tensor(a!) out) ->
Tensor(a!)
1105     dispatch:
1106         CPU, CUDA: ne_out
1107         QuantizedCPU: ne_out_quantized_cpu
1108 +     Checkpoint: checkpoint_ne_Tensor_out
1109
1110 - func: ne.Tensor(Tensor self, Tensor other) -> Tensor
1111     use_c10_dispatcher: full
1112 @@ -4928,11 +5268,13 @@
1113     dispatch:
1114         CPU, CUDA: ne
1115         QuantizedCPU: ne_quantized_cpu
1116 +     Checkpoint: checkpoint_ne_Tensor
1117
1118 - func: eq.Scalar_out(Tensor self, Scalar other, *, Tensor(a!) out) ->
Tensor(a!)
1119     dispatch:
1120         CPU, CUDA: eq_out
1121         QuantizedCPU: eq_out_quantized_cpu
1122 +     Checkpoint: checkpoint_eq_Scalar_out
1123
1124 - func: eq.Scalar(Tensor self, Scalar other) -> Tensor
1125     use_c10_dispatcher: full
1126 @@ -4940,11 +5282,13 @@
1127     dispatch:
1128         CPU, CUDA: eq
1129         QuantizedCPU: eq_quantized_cpu
1130 +     Checkpoint: checkpoint_eq_Scalar
1131
1132 - func: eq.Tensor_out(Tensor self, Tensor other, *, Tensor(a!) out) ->
Tensor(a!)
1133     dispatch:
1134         CPU, CUDA: eq_out
1135         QuantizedCPU: eq_out_quantized_cpu
1136 +     Checkpoint: checkpoint_eq_Tensor_out
1137
1138 - func: eq.Tensor(Tensor self, Tensor other) -> Tensor
1139     use_c10_dispatcher: full
1140 @@ -4952,6 +5296,7 @@
1141     dispatch:
1142         CPU, CUDA: eq

```

```

1143     QuantizedCPU: eq_quantized_cpu
1144 +     Checkpoint: checkpoint_eq_Tensor
1145
1146 - func: ge.Scalar_out(Tensor self, Scalar other, *, Tensor(a!) out) ->
Tensor(a!)
1147     dispatch:
1148 @@ -5029,6 +5374,7 @@
1149     dispatch:
1150         CPU, CUDA: lt_out
1151         QuantizedCPU: lt_out_quantized_cpu
1152 +     Checkpoint: checkpoint_lt_out
1153
1154 - func: lt.Scalar(Tensor self, Scalar other) -> Tensor
1155     use_c10_dispatcher: full
1156 @@ -5036,11 +5382,13 @@
1157     dispatch:
1158         CPU, CUDA: lt
1159         QuantizedCPU: lt_quantized_cpu
1160 +     Checkpoint: checkpoint_lt
1161
1162 - func: lt.Tensor_out(Tensor self, Tensor other, *, Tensor(a!) out) ->
Tensor(a!)
1163     dispatch:
1164         CPU, CUDA: lt_out
1165         QuantizedCPU: lt_out_quantized_cpu
1166 +     Checkpoint: checkpoint_lt_out
1167
1168 - func: lt.Tensor(Tensor self, Tensor other) -> Tensor
1169     use_c10_dispatcher: full
1170 @@ -5048,6 +5396,7 @@
1171     dispatch:
1172         CPU, CUDA: lt
1173         QuantizedCPU: lt_quantized_cpu
1174 +     Checkpoint: checkpoint_lt
1175
1176 - func: take.out(Tensor self, Tensor index, *, Tensor(a!) out) -> Tensor(a!)
1177     dispatch:
1178 @@ -5084,6 +5433,7 @@
1179     dispatch:
1180         CPU: masked_select_out_cpu
1181         CUDA: masked_select_out_cuda
1182 +     Checkpoint: checkpoint_masked_select_out
1183
1184 - func: masked_select(Tensor self, Tensor mask) -> Tensor
1185     use_c10_dispatcher: full
1186 @@ -5091,11 +5441,13 @@
1187     dispatch:

```

```

1188     CPU: masked_select_cpu
1189     CUDA: masked_select_cuda
1190 +   Checkpoint: checkpoint_masked_select
1191
1192 - func: nonzero.out(Tensor self, *, Tensor(a!) out) -> Tensor(a!)
1193 dispatch:
1194     CPU: legacy::cpu::_th_nonzero_out
1195     CUDA: legacy::cuda::_th_nonzero_out
1196 +   Checkpoint: checkpoint_nonzero_out
1197
1198 - func: nonzero(Tensor self) -> Tensor
1199 use_c10_dispatcher: full
1200 @@ -5103,6 +5455,7 @@
1201 dispatch:
1202     CPU: legacy::cpu::_th_nonzero
1203     CUDA: legacy::cuda::_th_nonzero
1204 +   Checkpoint: checkpoint_nonzero
1205
1206 - func: nonzero_numpy(Tensor self) -> Tensor[]
1207 use_c10_dispatcher: full
1208 @@ -5136,12 +5489,18 @@
1209 - func: addcmul_(Tensor(a!) self, Tensor tensor1, Tensor tensor2, *, Scalar
value=1) -> Tensor(a!)
1210 use_c10_dispatcher: full
1211 variants: method
1212 + dispatch:
1213 +   CPU, CUDA: addcmul_
1214 +   Checkpoint: checkpoint_addcmul_
1215
1216 - func: addcddiv.out(Tensor self, Tensor tensor1, Tensor tensor2, *, Scalar
value=1, Tensor(a!) out) -> Tensor(a!)
1217
1218 - func: addcddiv(Tensor self, Tensor tensor1, Tensor tensor2, *, Scalar
value=1) -> Tensor
1219 use_c10_dispatcher: full
1220 variants: method, function
1221 + dispatch:
1222 +   CPU, CUDA: addcddiv
1223 +   Checkpoint: checkpoint_addcddiv
1224
1225 - func: lstsq.X(Tensor self, Tensor A, *, Tensor(a!) X, Tensor(b!) qr) ->
(Tensor(a!) solution, Tensor(b!) QR)
1226 dispatch:
1227 @@ -5389,6 +5748,9 @@
1228 - func: sign(Tensor self) -> Tensor
1229 use_c10_dispatcher: full
1230 variants: function, method

```

```

1231 + dispatch:
1232 +   CPU, CUDA: sign
1233 +   Checkpoint: checkpoint_sign
1234
1235 - func: sign_(Tensor(a!) self) -> Tensor(a!)
1236   use_c10_dispatcher: full
1237 @@ -5397,6 +5759,7 @@
1238 - func: sign.out(Tensor self, *, Tensor(a!) out) -> Tensor(a!)
1239   dispatch:
1240     CPU, CUDA: sign_out
1241 +   Checkpoint: checkpoint_sign_out
1242
1243 - func: signbit(Tensor self) -> Tensor
1244   use_c10_dispatcher: full
1245 @@ -5612,6 +5975,7 @@
1246   dispatch:
1247     CPU: topk_out_cpu
1248     CUDA: legacy::cuda::_th_topk_out
1249 +   Checkpoint: checkpoint_topk_values
1250
1251 - func: topk(Tensor self, int k, int dim=-1, bool largest=True, bool
sorted=True) -> (Tensor values, Tensor indices)
1252   use_c10_dispatcher: full
1253 @@ -5619,6 +5983,7 @@
1254   dispatch:
1255     CPU, CUDA: topk
1256     QuantizedCPU: topk_quantized_cpu
1257 +   Checkpoint: checkpoint_topk
1258
1259 - func: all(Tensor self) -> Tensor
1260   use_c10_dispatcher: full
1261 @@ -5630,6 +5995,7 @@
1262   dispatch:
1263     CPU, CUDA: any
1264     SparseCPU, SparseCUDA: any_sparse
1265 +   Checkpoint: checkpoint_any
1266
1267 - func: renorm.out(Tensor self, Scalar p, int dim, Scalar maxnorm, *,
Tensor(a!) out) -> Tensor(a!)
1268   dispatch:
1269 @@ -5664,6 +6030,7 @@
1270     CPU: cpu_equal
1271     CUDA: cuda_equal
1272     QuantizedCPU: equal_quantized_cpu
1273 +   Checkpoint: checkpoint_equal
1274

```

```

1275     - func: pow.Tensor_Tensor_out(Tensor self, Tensor exponent, *, Tensor(a!)
out) -> Tensor(a!)
1276     dispatch:
1277     @@ -5782,12 +6149,13 @@
1278         CPU: _cat_cpu
1279         CUDA: cat_cuda
1280         QuantizedCPU: cat_quantized_cpu
1281     + Checkpoint: checkpoint__cat
1282
1283     - func: _cat.out(Tensor[] tensors, int dim=0, *, Tensor(a!) out) ->
Tensor(a!)
1284     dispatch:
1285         CPU: _cat_out_cpu
1286         CUDA: cat_out_cuda
1287     - QuantizedCPU: cat_out_quantized_cpu
1288     + Checkpoint: checkpoint__cat_out
1289
1290     - func: _foreach_add.Scalar(Tensor[] tensors, Scalar scalar) -> Tensor[]
1291     use_c10_dispatcher: full
1292     @@ -5956,6 +6324,7 @@
1293     dispatch:
1294         CPU: nll_loss_forward_out_cpu
1295         CUDA: legacy::cuda::_thnn_nll_loss_forward_out
1296     + Checkpoint: checkpoint_nll_loss_forward_out
1297
1298     - func: nll_loss_forward(Tensor self, Tensor target, Tensor? weight, int
reduction, int ignore_index) -> (Tensor output, Tensor total_weight)
1299     use_c10_dispatcher: full
1300     @@ -5963,12 +6332,14 @@
1301     dispatch:
1302         CPU: nll_loss_forward_cpu
1303         CUDA: legacy::cuda::_thnn_nll_loss_forward
1304     + Checkpoint: checkpoint_nll_loss_forward
1305
1306     - func: nll_loss_backward.grad_input(Tensor grad_output, Tensor self, Tensor
target, Tensor? weight, int reduction, int ignore_index, Tensor total_weight,
*, Tensor(a!) grad_input) -> Tensor(a!)
1307     python_module: nn
1308     dispatch:
1309         CPU: nll_loss_backward_out_cpu
1310         CUDA: legacy::cuda::_thnn_nll_loss_backward_out
1311     + Checkpoint: checkpoint_nll_loss_backward_grad_input
1312
1313     - func: nll_loss_backward(Tensor grad_output, Tensor self, Tensor target,
Tensor? weight, int reduction, int ignore_index, Tensor total_weight) ->
Tensor
1314     use_c10_dispatcher: full

```

```

1315 @@ -5976,6 +6347,7 @@
1316     dispatch:
1317         CPU: nll_loss_backward_cpu
1318         CUDA: legacy::cuda::_thnn_nll_loss_backward
1319 +     Checkpoint: checkpoint_nll_loss_backward
1320
1321     - func: nll_loss2d.out(Tensor self, Tensor target, Tensor? weight=None, int
reduction=Mean, int ignore_index=-100, *, Tensor(a!) out) -> Tensor(a!)
1322     python_module: nn
1323 @@ -5989,6 +6361,7 @@
1324     dispatch:
1325         CPU: nll_loss2d_forward_out_cpu
1326         CUDA: legacy::cuda::_thnn_nll_loss2d_forward_out
1327 +     Checkpoint: checkpoint_nll_loss2d_forward_out
1328
1329     - func: nll_loss2d_forward(Tensor self, Tensor target, Tensor? weight, int
reduction, int ignore_index) -> (Tensor output, Tensor total_weight)
1330     use_c10_dispatcher: full
1331 @@ -5996,12 +6369,14 @@
1332     dispatch:
1333         CPU: nll_loss2d_forward_cpu
1334         CUDA: legacy::cuda::_thnn_nll_loss2d_forward
1335 +     Checkpoint: checkpoint_nll_loss2d_forward
1336
1337     - func: nll_loss2d_backward.grad_input(Tensor grad_output, Tensor self,
Tensor target, Tensor? weight, int reduction, int ignore_index, Tensor
total_weight, *, Tensor(a!) grad_input) -> Tensor(a!)
1338     python_module: nn
1339     dispatch:
1340         CPU: nll_loss2d_backward_out_cpu
1341         CUDA: legacy::cuda::_thnn_nll_loss2d_backward_out
1342 +     Checkpoint: checkpoint_nll_loss2d_backward_out
1343
1344     - func: nll_loss2d_backward(Tensor grad_output, Tensor self, Tensor target,
Tensor? weight, int reduction, int ignore_index, Tensor total_weight) ->
Tensor
1345     use_c10_dispatcher: full
1346 @@ -6009,6 +6384,7 @@
1347     dispatch:
1348         CPU: nll_loss2d_backward_cpu
1349         CUDA: legacy::cuda::_thnn_nll_loss2d_backward
1350 +     Checkpoint: checkpoint_nll_loss2d_backward
1351
1352     - func: smooth_l1_loss.out(Tensor self, Tensor target, int reduction=Mean,
*, Tensor(a!) out) -> Tensor(a!)
1353     python_module: nn
1354 @@ -6127,10 +6503,15 @@

```

```

1355     python_module: nn
1356     dispatch:
1357         CPU, CUDA: hardtanh_backward_out
1358 +     Checkpoint: checkpoint_hardtanh_backward_out
1359
1360     - func: hardtanh_backward(Tensor grad_output, Tensor self, Scalar min_val,
Scalar max_val) -> Tensor
1361         use_c10_dispatcher: full
1362         python_module: nn
1363 +     dispatch:
1364 +     CPU: hardtanh_backward
1365 +     CUDA: hardtanh_backward
1366 +     Checkpoint: checkpoint_hardtanh_backward
1367
1368     - func: hardtanh_(Tensor(a!) self, Scalar min_val=-1, Scalar max_val=1) ->
Tensor(a!)
1369         use_c10_dispatcher: full
1370 @@ -6386,6 +6767,7 @@
1371         CPU: avg_pool2d_out_cpu
1372         CUDA: avg_pool2d_out_cuda
1373         MklDnnCPU: mklDnn_avg_pool2d_out
1374 +     Checkpoint: checkpoint_avg_pool2d_out
1375
1376     - func: avg_pool2d(Tensor self, int[2] kernel_size, int[2] stride=[], int[2]
padding=0, bool ceil_mode=False, bool count_include_pad=True, int?
divisor_override=None) -> Tensor
1377         use_c10_dispatcher: full
1378 @@ -6395,12 +6777,14 @@
1379         CUDA: avg_pool2d_cuda
1380         MklDnnCPU: mklDnn_avg_pool2d
1381         QuantizedCPU: avg_pool2d_quantized_cpu
1382 +     Checkpoint: checkpoint_avg_pool2d
1383
1384     - func: avg_pool2d_backward.grad_input(Tensor grad_output, Tensor self,
int[2] kernel_size, int[2] stride, int[2] padding, bool ceil_mode, bool
count_include_pad, int? divisor_override, *, Tensor(a!) grad_input) ->
Tensor(a!)
1385     python_module: nn
1386     dispatch:
1387         CPU: avg_pool2d_backward_out_cpu
1388         CUDA: avg_pool2d_backward_out_cuda
1389 +     Checkpoint: checkpoint_avg_pool2d_backward_grad_input
1390
1391     - func: avg_pool2d_backward(Tensor grad_output, Tensor self, int[2]
kernel_size, int[2] stride, int[2] padding, bool ceil_mode, bool
count_include_pad, int? divisor_override) -> Tensor
1392         use_c10_dispatcher: full

```



```

1393 @@ -6408,6 +6792,7 @@
1394     dispatch:
1395         CPU: avg_pool2d_backward_cpu
1396         CUDA: avg_pool2d_backward_cuda
1397 +     Checkpoint: checkpoint_avg_pool2d_backward
1398
1399     - func: avg_pool3d.out(Tensor self, int[3] kernel_size, int[3] stride=[],
1400     int[3] padding=0, bool ceil_mode=False, bool count_include_pad=True, int?
1401     divisor_override=None, *, Tensor(a!) out) -> Tensor(a!)
1402     python_module: nn
1403 @@ -6500,6 +6885,7 @@
1404     dispatch:
1405         CPU: max_pool2d_with_indices_out_cpu
1406         CUDA: max_pool2d_with_indices_out_cuda
1407 +     Checkpoint: checkpoint_max_pool2d_with_indices_out
1408
1409     # Return: (Tensor output, Tensor indices)
1410     - func: max_pool2d_with_indices(Tensor self, int[2] kernel_size, int[2]
1411     stride=[], int[2] padding=0, int[2] dilation=1, bool ceil_mode=False) ->
1412     (Tensor, Tensor)
1413 @@ -6508,12 +6894,14 @@
1414     dispatch:
1415         CPU: max_pool2d_with_indices_cpu
1416         CUDA: max_pool2d_with_indices_cuda
1417 +     Checkpoint: checkpoint_max_pool2d_with_indices
1418
1419     - func: max_pool2d_with_indices_backward.grad_input(Tensor grad_output,
1420     Tensor self, int[2] kernel_size, int[2] stride, int[2] padding, int[2]
1421     dilation, bool ceil_mode, Tensor indices, *, Tensor(a!) grad_input) ->
1422     Tensor(a!)
1423     python_module: nn
1424     dispatch:
1425         CPU: max_pool2d_with_indices_backward_out_cpu
1426         CUDA: max_pool2d_with_indices_backward_out_cuda
1427 +     Checkpoint: checkpoint_max_pool2d_with_indices_backward_grad_input
1428
1429     - func: max_pool2d_with_indices_backward(Tensor grad_output, Tensor self,
1430     int[2] kernel_size, int[2] stride, int[2] padding, int[2] dilation, bool
1431     ceil_mode, Tensor indices) -> Tensor
1432     use_c10_dispatcher: full
1433 @@ -6521,6 +6909,7 @@
1434     dispatch:
1435         CPU: max_pool2d_with_indices_backward_cpu
1436         CUDA: max_pool2d_with_indices_backward_cuda
1437 +     Checkpoint: checkpoint_max_pool2d_with_indices_backward
1438
1439     # Return: (Tensor output, Tensor indices)

```

```

1431 - func: max_pool3d_with_indices.out(Tensor self, int[3] kernel_size, int[3]
stride=[], int[3] padding=0, int[3] dilation=1, bool ceil_mode=False, *,
Tensor(a!) out, Tensor(b!) indices) -> (Tensor(a!), Tensor(b!))
1432 @@ -6764,6 +7153,7 @@
1433     dispatch:
1434         CPU: upsample_bilinear2d_out_cpu
1435         CUDA: upsample_bilinear2d_out_cuda
1436 +     Checkpoint: checkpoint_upsample_bilinear2d_out
1437
1438 - func: upsample_bilinear2d(Tensor self, int[2] output_size, bool
align_corners, float? scales_h=None, float? scales_w=None) -> Tensor
1439     use_c10_dispatcher: full
1440 @@ -6772,12 +7162,14 @@
1441     CPU: upsample_bilinear2d_cpu
1442     CUDA: upsample_bilinear2d_cuda
1443     QuantizedCPU: upsample_bilinear2d_quantized_cpu
1444 +     Checkpoint: checkpoint_upsample_bilinear2d
1445
1446 - func: upsample_bilinear2d_backward.grad_input(Tensor grad_output, int[2]
output_size, int[4] input_size, bool align_corners, float? scales_h=None,
float? scales_w=None, *, Tensor(a!) grad_input) -> Tensor(a!)
1447     python_module: nn
1448     dispatch:
1449         CPU: upsample_bilinear2d_backward_out_cpu
1450         CUDA: upsample_bilinear2d_backward_out_cuda
1451 +     Checkpoint: checkpoint_upsample_bilinear2d_backward_out
1452
1453 - func: upsample_bilinear2d_backward(Tensor grad_output, int[2] output_size,
int[4] input_size, bool align_corners, float? scales_h=None, float?
scales_w=None) -> Tensor
1454     use_c10_dispatcher: full
1455 @@ -6785,6 +7177,7 @@
1456     dispatch:
1457         CPU: upsample_bilinear2d_backward_cpu
1458         CUDA: upsample_bilinear2d_backward_cuda
1459 +     Checkpoint: checkpoint_upsample_bilinear2d_backward
1460
1461 - func: upsample_bicubic2d.out(Tensor self, int[2] output_size, bool
align_corners, float? scales_h=None, float? scales_w=None, *, Tensor(a!) out)
-> Tensor(a!)
1462     python_module: nn
1463 @@ -7023,10 +7416,15 @@
1464     python_module: nn
1465     dispatch:
1466         CPU, CUDA: sigmoid_backward_out
1467 +     Checkpoint: checkpoint_sigmoid_backward_out
1468

```

```

1469 - func: sigmoid_backward(Tensor grad_output, Tensor output) -> Tensor
1470     use_c10_dispatcher: full
1471     python_module: nn
1472 + dispatch:
1473 +   CPU: sigmoid_backward
1474 +   CUDA: sigmoid_backward
1475 +   Checkpoint: checkpoint_sigmoid_backward
1476
1477 - func: logit_backward.grad_input(Tensor grad_output, Tensor self, float?
eps=None, *, Tensor(a!) grad_input) -> Tensor(a!)
1478     python_module: nn
1479 @@ -7041,10 +7439,15 @@
1480     python_module: nn
1481     dispatch:
1482         CPU, CUDA: tanh_backward_out
1483 +   Checkpoint: checkpoint_tanh_backward_out
1484
1485 - func: tanh_backward(Tensor grad_output, Tensor output) -> Tensor
1486     use_c10_dispatcher: full
1487     python_module: nn
1488 + dispatch:
1489 +   CPU: tanh_backward
1490 +   CUDA: tanh_backward
1491 +   Checkpoint: checkpoint_tanh_backward
1492
1493 # What's a thnn_conv_ versus a slow_conv_?
1494 #
1495 @@ -7128,6 +7531,7 @@
1496     dispatch:
1497         CPU: slow_conv2d_forward_out_cpu
1498         CUDA: legacy::cuda::_thnn_conv2d_forward_out
1499 +   Checkpoint: checkpoint_thnn_conv2d_forward_out
1500
1501 - func: thnn_conv2d_forward(Tensor self, Tensor weight, int[2] kernel_size,
Tensor? bias, int[2] stride, int[2] padding) -> (Tensor output, Tensor
finput, Tensor fgrad_input)
1502     use_c10_dispatcher: full
1503 @@ -7135,12 +7539,14 @@
1504     dispatch:
1505         CPU: slow_conv2d_forward_cpu
1506         CUDA: legacy::cuda::_thnn_conv2d_forward
1507 +   Checkpoint: checkpoint_thnn_conv2d_forward
1508
1509 - func: thnn_conv2d_backward.grad_input(Tensor grad_output, Tensor self,
Tensor weight, int[2] kernel_size, int[2] stride, int[2] padding, Tensor
finput, Tensor fgrad_input, *, Tensor(a!)? grad_input, Tensor(b!)?
grad_weight, Tensor(c!)? grad_bias) -> (Tensor(a!), Tensor(b!), Tensor(c!))

```

```

1510     python_module: nn
1511     dispatch:
1512         CPU: slow_conv2d_backward_out_cpu
1513         CUDA: slow_conv2d_backward_out_cuda
1514 +     Checkpoint: checkpoint_thnn_conv2d_backward_out
1515
1516 - func: thnn_conv2d_backward.output_mask(Tensor grad_output, Tensor self,
Tensor weight, int[2] kernel_size, int[2] stride, int[2] padding, Tensor
finput, Tensor fgrad_input, bool[3] output_mask) -> (Tensor grad_input,
Tensor grad_weight, Tensor grad_bias)
1517     use_c10_dispatcher: full
1518 @@ -7148,6 +7554,7 @@
1519     dispatch:
1520         CPU: slow_conv2d_backward_cpu
1521         CUDA: slow_conv2d_backward_cuda
1522 +     Checkpoint: checkpoint_thnn_conv2d_backward
1523
1524 - func: thnn_conv_depthwise2d.out(Tensor self, Tensor weight, int[2]
kernel_size, Tensor? bias=None, int[2] stride=1, int[2] padding=0, int[2]
dilation=1, *, Tensor(a!) out) -> Tensor(a!)
1525     python_module: nn

```

DELTA

```

1  diff -ru dtr_pytorch/aten/src/ATen/native/native_functions.yaml
delta/aten/src/ATen/native/native_functions.yaml
2  --- dtr_pytorch/aten/src/ATen/native/native_functions.yaml      2024-04-28
16:09:10.546175266 +0800
3  +++ delta/aten/src/ATen/native/native_functions.yaml      2024-04-09
14:45:52.889659390 +0800
4  @@ -907,17 +907,21 @@
5      CUDA: bmm_out_cuda
6      SparseCPU: bmm_out_sparse_cpu
7      SparseCUDA: bmm_out_sparse_cuda
8  +     Checkpoint: checkpoint_bmm_out
9
10 - func: _bmm(Tensor self, Tensor mat2, *, bool deterministic=False) -> Tensor
11 use_c10_dispatcher: full
12 variants: function
13 dispatch:
14     SparseCUDA: _bmm_sparse_cuda
15 +     # Checkpoint: checkpoint_bmm_
16 +
17

```

```

18     - func: _bmm.out(Tensor self, Tensor mat2, *, bool deterministic=False,
Tensor(a!) out) -> Tensor(a!)
19     variants: function
20     dispatch:
21         SparseCUDA: _bmm_out_sparse_cuda
22 +     # Checkpoint: checkpoint_bmm_out_
23
24     - func: broadcast_tensors(Tensor[] tensors) -> Tensor[]
25     use_c10_dispatcher: full
26 @@ -1520,30 +1524,54 @@
27     - func: erf(Tensor self) -> Tensor
28     use_c10_dispatcher: full
29     variants: function, method
30 + dispatch:
31 +     CPU, CUDA: erf
32 +     Checkpoint: checkpoint_erf
33
34     - func: erf_(Tensor(a!) self) -> Tensor(a!)
35     use_c10_dispatcher: full
36     variants: function, method
37 + dispatch:
38 +     CPU, CUDA: erf_
39 +     Checkpoint: checkpoint_erf_
40
41     - func: erf.out(Tensor self, *, Tensor(a!) out) -> Tensor(a!)
42 + dispatch:
43 +     CPU, CUDA: erf_out
44 +     Checkpoint: checkpoint_erf_out
45
46     - func: erfc(Tensor self) -> Tensor
47     use_c10_dispatcher: full
48     variants: function, method
49 + dispatch:
50 +     CPU, CUDA: erfc
51 +     Checkpoint: checkpoint_erfc
52
53     - func: erfc_(Tensor(a!) self) -> Tensor(a!)
54     use_c10_dispatcher: full
55     variants: function, method
56 + dispatch:
57 +     CPU, CUDA: erfc_
58 +     Checkpoint: checkpoint_erfc_
59
60     - func: erfc.out(Tensor self, *, Tensor(a!) out) -> Tensor(a!)
61 + dispatch:
62 +     CPU, CUDA: erfc_out
63 +     Checkpoint: checkpoint_erfc_out

```

```

64
65     - func: exp(Tensor self) -> Tensor
66         use_c10_dispatcher: full
67         variants: function, method
68     + dispatch:
69     +     CPU, CUDA: exp
70     +     Checkpoint: checkpoint_exp
71
72     - func: exp_(Tensor(a!) self) -> Tensor(a!)
73         use_c10_dispatcher: full
74         variants: function, method
75     + dispatch:
76     +     CPU, CUDA: exp_
77     +     Checkpoint: checkpoint_exp_
78
79     - func: exp.out(Tensor self, *, Tensor(a!) out) -> Tensor(a!)
80
81 @@ -2157,8 +2185,17 @@
82     - func: matmul(Tensor self, Tensor other) -> Tensor
83         use_c10_dispatcher: full
84         variants: function, method
85     + # dispatch:
86     + #     CPU, CUDA: matmul
87     + #     Checkpoint: checkpoint_matmul
88
89     - func: matmul.out(Tensor self, Tensor other, *, Tensor(a!) out) ->
Tensor(a!)
90     + #use_c10_dispatcher: full
91     + #variants: function, method
92     + #dispatch:
93     + #     CPU, CUDA: matmul_out
94     + #     Checkpoint: checkpoint_matmul_out
95     + # there is some pt bug which disallow the above code.
96
97     - func: matrix_rank.tol(Tensor self, float tol, bool symmetric=False) ->
Tensor
98         use_c10_dispatcher: full
99 @@ -2906,6 +2943,7 @@
100     dispatch:
101         CPU: gelu_cpu
102         CUDA: gelu_cuda
103     +     Checkpoint: checkpoint_gelu
104
105     - func: gelu_backward(Tensor grad, Tensor self) -> Tensor
106         use_c10_dispatcher: full
107 @@ -3426,6 +3464,9 @@
108         use_c10_dispatcher: full

```

```

109     variants: function, method
110     device_guard: False
111 +   dispatch:
112 +     CPU, CUDA: transpose
113 +     Checkpoint: checkpoint_transpose
114
115   - func: transpose.Dimname(Tensor(a) self, Dimname dim0, Dimname dim1) ->
Tensor(a)
116     variants: function, method
117 @@ -3441,6 +3482,9 @@
118     use_c10_dispatcher: full
119     variants: method
120     device_guard: False
121 +   dispatch:
122 +     CPU, CUDA: transpose_
123 +     Checkpoint: checkpoint_transpose_
124
125   - func: _mkldnn_transpose_(Tensor(a!) self, int dim0, int dim1) -> Tensor(a!)
126     use_c10_dispatcher: full
127 @@ -3880,6 +3924,7 @@
128     dispatch:
129       CPU, CUDA: pow_out
130       SparseCPU, SparseCUDA: pow_out_sparse_scalar
131 +     Checkpoint: checkpoint_pow_out
132
133   - func: pow.Tensor_Scalar(Tensor self, Scalar exponent) -> Tensor
134     use_c10_dispatcher: full
135 @@ -3887,6 +3932,7 @@
136     dispatch:
137       CPU, CUDA: pow
138       SparseCPU, SparseCUDA: pow_sparse_scalar
139 +     Checkpoint: checkpoint_pow
140
141   - func: zero_(Tensor(a!) self) -> Tensor(a!)
142     use_c10_dispatcher: full
143 @@ -4449,6 +4495,14 @@
144   # to(Device) must not exist because all constructors of Device also works for
145   # TensorOptions. Otherwise, an ambiguity error is thrown.
146   # See NOTE [ TensorOptions Constructors ].
147 +# - func: to(Tensor self) -> Tensor
148 +#   use_c10_dispatcher: full
149 +#   variants: method
150 +#   device_guard: False
151 +
152 +- func: to_backward(Tensor grad, Tensor input) -> Tensor
153 + use_c10_dispatcher: full
154 +

```

```

155     - func: to.dtype_layout(Tensor self, *, ScalarType? dtype=None, Layout?
      layout=None, Device? device=None, bool? pin_memory=None, bool
      non_blocking=False, bool copy=False, MemoryFormat? memory_format=None) ->
      Tensor
156     use_c10_dispatcher: full
157     variants: method
158 @@ -4461,16 +4515,25 @@
159     use_c10_dispatcher: full
160     variants: method
161     device_guard: False
162 +   dispatch:
163 +     CPU, CUDA: to
164 +     Checkpoint: checkpoint_to
165
166     - func: to.dtype(Tensor self, ScalarType dtype, bool non_blocking=False,
      bool copy=False, MemoryFormat? memory_format=None) -> Tensor
167     use_c10_dispatcher: full
168     variants: method
169     device_guard: False
170 +   dispatch:
171 +     CPU, CUDA: to
172 +     Checkpoint: checkpoint_to
173
174     - func: to.other(Tensor self, Tensor other, bool non_blocking=False, bool
      copy=False, MemoryFormat? memory_format=None) -> Tensor
175     use_c10_dispatcher: full
176     variants: method
177     device_guard: False
178 +   dispatch:
179 +     CPU, CUDA: to
180 +     Checkpoint: checkpoint_to
181
182     - func: meshgrid(Tensor[] tensors) -> Tensor[]
183     use_c10_dispatcher: full

```

!!DELTA更新 tools/autograd/derivatives.yaml

```

1  diff -ru dtr_pytorch/tools/autograd/derivatives.yaml
    delta/tools/autograd/derivatives.yaml
2  --- dtr_pytorch/tools/autograd/derivatives.yaml      2024-04-28
    16:07:35.413944557 +0800
3  +++ delta/tools/autograd/derivatives.yaml      2024-04-09 14:45:53.121704278
    +0800
4  @@ -1066,6 +1066,18 @@

```



```

5   - name: trunc(Tensor self) -> Tensor
6     self: zeros_like(grad, at::MemoryFormat::Preserve)
7
8   +- name: to.device(Tensor self, Device device, ScalarType dtype, bool
  non_blocking=False, bool copy=False, MemoryFormat? memory_format=None) ->
  Tensor
9   + self: to_backward(grad, self)
10  +
11  +- name: to.dtype_layout(Tensor self, *, ScalarType? dtype=None, Layout?
  layout=None, Device? device=None, bool? pin_memory=None, bool
  non_blocking=False, bool copy=False, MemoryFormat? memory_format=None) ->
  Tensor
12  + self: to_backward(grad, self)
13  +
14  +- name: to.dtype(Tensor self, ScalarType dtype, bool non_blocking=False, bool
  copy=False, MemoryFormat? memory_format=None) -> Tensor
15  + self: to_backward(grad, self)
16  +
17  +- name: to.other(Tensor self, Tensor other, bool non_blocking=False, bool
  copy=False, MemoryFormat? memory_format=None) -> Tensor
18  + self: to_backward(grad, self)
19  +
20  - name: to_dense(Tensor self) -> Tensor
21    self: to_dense_backward(grad, self)
22
23  @@ -1110,6 +1122,15 @@
24
25  - name: view_as_complex(Tensor(a) self) -> Tensor(a)
26    self: at::view_as_real(grad.contiguous()).conj()) # [gx, -gy]
27  +
28  +- name: where(Tensor condition) -> Tensor[]
29  + condition: non_differentiable
30  + self: where(condition, grad, zeros_like(grad, at::MemoryFormat::Preserve))
31  +
32  +- name: where.self(Tensor condition, Tensor self, Tensor other) -> Tensor
33  + condition: non_differentiable
34  + self: where(condition, grad, zeros_like(grad, at::MemoryFormat::Preserve))
35  + other: where(condition, zeros_like(grad, at::MemoryFormat::Preserve), grad)
36
37  - name: _s_where(Tensor condition, Tensor self, Tensor other) -> Tensor
38    condition: non_differentiable

```

!修改 tools/autograd/templates/Functions.cpp 将 slice_backward、select_backward、split_with_sizes_backward 和 split_backward 移动到 aten/src/ATen/native/Activation.cpp

```
1 diff --git a/tools/autograd/templates/Functions.cpp
  b/tools/autograd/templates/Functions.cpp
2 index 4109d1e799..1cda83ce2f 100644
3 --- a/tools/autograd/templates/Functions.cpp
4 +++ b/tools/autograd/templates/Functions.cpp
5 @@ -769,18 +769,6 @@ Tensor index_select_backward(Tensor grad, int64_t dim,
  Tensor indices, IntArrayR
6     return at::zeros(sizes, grad.options()).scatter_(dim, indices, grad);
7 }
8
9 -Tensor slice_backward(Tensor grad, IntArrayRef input_sizes, int64_t dim,
  int64_t start, int64_t end, int64_t step) {
10 - auto grad_input = at::zeros(input_sizes, grad.options());
11 - grad_input.slice(dim, start, end, step).copy_(grad);
12 - return grad_input;
13 -}
14 -
15 -Tensor select_backward(Tensor grad, IntArrayRef input_sizes, int64_t dim,
  int64_t index) {
16 - auto grad_input = at::zeros(input_sizes, grad.options());
17 - grad_input.select(dim, index).copy_(grad);
18 - return grad_input;
19 -}
20 -
21 Tensor trace_backward(const Tensor & grad, IntArrayRef sizes) {
22     if (sizes.size() != 2) {
23         throw std::runtime_error("expected matrix input");
24 @@ -901,38 +889,6 @@ Tensor cholesky_inverse_backward(Tensor grad, Tensor L,
  bool upper, Tensor inver
25     return grad_L;
26 }
27
28 -Tensor split_with_sizes_backward(const
  std::vector<torch::autograd::Variable> &grads,
29 -                               IntArrayRef split_sizes, int64_t dim,
  IntArrayRef sizes, const at::TensorOptions &options) {
30 - dim = at::maybe_wrap_dim(dim, sizes.size());
31 -
```

```

32 - // it's possible some of the grads are not defined (represents tensors of
    all 0s).
33 - // Since at::cat can't handle those, let's define them
34 - std::vector<Tensor> grads_all_defined(grads.size());
35 - for (size_t j = 0; j < grads.size(); ++j) {
36 -     if (grads[j].defined()) {
37 -         grads_all_defined[j] = grads[j];
38 -     } else {
39 -         auto length = split_sizes[j];
40 -         auto grad_size = sizes.vec();
41 -         grad_size[dim] = length;
42 -         grads_all_defined[j] = at::zeros(grad_size, options);
43 -     }
44 - }
45 -
46 - auto ret = at::cat(grads_all_defined, dim);
47 - return ret;
48 -}
49 -
50 -Tensor split_backward(const std::vector<torch::autograd::Variable> &grads,
51 -                      int64_t split_size, int64_t dim, IntArrayRef sizes,
52 -                      const at::TensorOptions &options) {
53 -     dim = at::maybe_wrap_dim(dim, sizes.size());
54 -     int64_t dim_size = sizes[dim];
55 -     int64_t num_splits = grads.size();
56 -     std::vector<int64_t> split_sizes(num_splits, split_size);
57 -     split_sizes[num_splits - 1] = split_size - (split_size * num_splits -
58 -         dim_size);
59 -     return split_with_sizes_backward(grads, split_sizes, dim, sizes, options);
60 -}
61 -
62 Tensor max_pool_double_backward(const Tensor & grad, const Tensor & indices,
63                                 int dim) {
64     AT_ASSERT(indices.dim() >= dim);
65     auto size = indices.sizes().slice(0, indices.dim() - dim).vec();

```

!修改 `aten/src/ATen/native/TensorShape.cpp` 将 `split` 和 `split_with_sizes` 移动到 `aten/src/ATen/native/Activation.cpp`

```

1 diff --git a/aten/src/ATen/native/TensorShape.cpp
  b/aten/src/ATen/native/TensorShape.cpp
2 index 1854938a30..4ee19b6964 100644

```

```

3  --- a/aten/src/ATen/native/TensorShape.cpp
4  +++ b/aten/src/ATen/native/TensorShape.cpp
5  @@ -1047,30 +1047,6 @@ Tensor slice(const Tensor& self, int64_t dim, int64_t
    start, int64_t end, int64_t
6      return result;
7  }
8
9  -std::vector<Tensor> split(const Tensor& self, int64_t split_size, int64_t
    dim) {
10 - TORCH_CHECK(self.dim() != 0, "split expects at least a 1-dimensional
    tensor");
11 - TORCH_CHECK(split_size >= 0, "split expects split_size be non-negative,
    but got split_size=", split_size);
12 - int64_t dim_size = self.size(dim);
13 - TORCH_CHECK(split_size > 0 || self.size(dim) == 0,
14 -             "split_size can only be 0 if dimension size is 0, "
15 -             "but got dimension size of ", dim_size);
16 - // if split_size is 0 and dimension size is 0, there is 1 split.
17 - int64_t num_splits = 1;
18 - if (split_size != 0) {
19 -     // ensuring num_splits is at least 1 makes consistent the case where
    split_size > dim_size
20 -     // (returns a single split). We might want to error here, but keep it
    for BC.
21 -     num_splits = std::max<int64_t>((dim_size + split_size - 1) / split_size,
    1);
22 - }
23 - std::vector<Tensor> splits(num_splits);
24 - int64_t last_split_size = split_size - (split_size * num_splits -
    dim_size);
25 -
26 - for (int64_t i = 0; i < num_splits; ++i) {
27 -     auto length = i < num_splits - 1 ? split_size : last_split_size;
28 -     splits[i] = self.narrow(dim, i * split_size, length);
29 - }
30 - return splits;
31 -}
32 -
33 std::vector<Tensor> unsafe_split(const Tensor& self, int64_t split_size,
    int64_t dim) {
34     auto result = at::native::split(self, split_size, dim);
35     for (auto& t : result) {
36 @@ -1079,28 +1055,6 @@ std::vector<Tensor> unsafe_split(const Tensor& self,
    int64_t split_size, int64_t
37     return result;
38 }
39

```

```

40 -std::vector<Tensor> split_with_sizes(const Tensor& self, IntArrayRef
    split_sizes, int64_t dim) {
41 - TORCH_CHECK(self.dim() != 0, "split expects at least a 1-dimensional
    tensor");
42 - int64_t dim_size = self.size(dim);
43 - int64_t num_splits = split_sizes.size();
44 - std::vector<Tensor> splits(num_splits);
45 - int64_t start_idx = 0;
46 - int64_t i;
47 -
48 - for (i = 0; i < num_splits; ++i) {
49 -     auto length = split_sizes[i];
50 -     TORCH_CHECK(length >= 0,
51 -         "split_with_sizes expects split_sizes have only non-negative ",
52 -         "entries, but got split_sizes=", split_sizes);
53 -     splits[i] = self.narrow(dim, start_idx, length);
54 -     start_idx += length;
55 - }
56 - TORCH_CHECK(start_idx == dim_size,
57 -     "split_with_sizes expects split_sizes to sum exactly to ",
    dim_size,
58 -     " (input tensor's size at dimension ", dim, "), ", "but got
    split_sizes=", split_sizes);
59 - return splits;
60 -}
61 -
62 std::vector<Tensor> unsafe_split_with_sizes(const Tensor& self, IntArrayRef
    split_sizes, int64_t dim) {
63     auto result = at::native::split_with_sizes(self, split_sizes, dim);
64     for (auto& t : result) {

```

!更新 `aten/src/ATen/native/Activation.cpp` 新增上面两个文件转移的函数

```

1 diff --git a/aten/src/ATen/native/Activation.cpp
  b/aten/src/ATen/native/Activation.cpp
2 index a43f16465d..71cef81aad 100644
3 --- a/aten/src/ATen/native/Activation.cpp
4 +++ b/aten/src/ATen/native/Activation.cpp
5 @@ -828,4 +828,94 @@ Tensor& log_sigmoid_backward_out_cpu(
6     DEFINE_DISPATCH(GeluKernel);
7     DEFINE_DISPATCH(GeluBackwardKernel);
8

```

```

9  +Tensor slice_backward(const Tensor& grad, IntArrayRef input_sizes, int64_t
    dim, int64_t start, int64_t end, int64_t step) {
10 +  auto grad_input = at::zeros(input_sizes, grad.options());
11 +  grad_input.slice(dim, start, end, step).copy_(grad);
12 +  return grad_input;
13 +}
14 +
15 +Tensor select_backward(const Tensor& grad, IntArrayRef input_sizes, int64_t
    dim, int64_t index) {
16 +  auto grad_input = at::zeros(input_sizes, grad.options());
17 +  grad_input.select(dim, index).copy_(grad);
18 +  return grad_input;
19 +}
20 +
21 +std::vector<Tensor> split(const Tensor& self, int64_t split_size, int64_t
    dim) {
22 +  TORCH_CHECK(self.dim() != 0, "split expects at least a 1-dimensional
    tensor");
23 +  TORCH_CHECK(split_size >= 0, "split expects split_size be non-negative,
    but got split_size=", split_size);
24 +  int64_t dim_size = self.size(dim);
25 +  TORCH_CHECK(split_size > 0 || self.size(dim) == 0,
26 +              "split_size can only be 0 if dimension size is 0, "
27 +              "but got dimension size of ", dim_size);
28 +  // if split_size is 0 and dimension size is 0, there is 1 split.
29 +  int64_t num_splits = 1;
30 +  if (split_size != 0) {
31 +    // ensuring num_splits is at least 1 makes consistent the case where
    split_size > dim_size
32 +    // (returns a single split). We might want to error here, but keep it
    for BC.
33 +    num_splits = std::max<int64_t>((dim_size + split_size - 1) / split_size,
    1);
34 +  }
35 +  std::vector<Tensor> splits(num_splits);
36 +  int64_t last_split_size = split_size - (split_size * num_splits -
    dim_size);
37 +
38 +  for (int64_t i = 0; i < num_splits; ++i) {
39 +    auto length = i < num_splits - 1 ? split_size : last_split_size;
40 +    splits[i] = self.narrow(dim, i * split_size, length);
41 +  }
42 +  return splits;
43 +}
44 +
45 +std::vector<Tensor> split_with_sizes(const Tensor& self, IntArrayRef
    split_sizes, int64_t dim) {

```

```

46 + TORCH_CHECK(self.dim() != 0, "split expects at least a 1-dimensional
    tensor");
47 + int64_t dim_size = self.size(dim);
48 + int64_t num_splits = split_sizes.size();
49 + std::vector<Tensor> splits(num_splits);
50 + int64_t start_idx = 0;
51 + int64_t i;
52 +
53 + for (i = 0; i < num_splits; ++i) {
54 +     auto length = split_sizes[i];
55 +     TORCH_CHECK(length >= 0,
56 +                 "split_with_sizes expects split_sizes have only non-negative ",
57 +                 "entries, but got split_sizes=", split_sizes);
58 +     splits[i] = self.narrow(dim, start_idx, length);
59 +     start_idx += length;
60 + }
61 + TORCH_CHECK(start_idx == dim_size,
62 +             "split_with_sizes expects split_sizes to sum exactly to ",
        dim_size,
63 +             " (input tensor's size at dimension ", dim, "), ", "but got
        split_sizes=", split_sizes);
64 + return splits;
65 +}
66 +
67 +Tensor split_backward(c10::ArrayRef<Tensor> grads,
68 +                     int64_t split_size, int64_t dim, IntArrayRef sizes,
        const at::TensorOptions &options) {
69 +     dim = at::maybe_wrap_dim(dim, sizes.size());
70 +     int64_t dim_size = sizes[dim];
71 +     int64_t num_splits = grads.size();
72 +     std::vector<int64_t> split_sizes(num_splits, split_size);
73 +     split_sizes[num_splits - 1] = split_size - (split_size * num_splits -
        dim_size);
74 +     return at::native::split_with_sizes_backward(grads, split_sizes, dim,
        sizes, options);
75 +}
76 +
77 +Tensor split_with_sizes_backward(c10::ArrayRef<Tensor> grads,
78 +                                IntArrayRef split_sizes, int64_t dim,
        IntArrayRef sizes, const at::TensorOptions &options) {
79 +     dim = at::maybe_wrap_dim(dim, sizes.size());
80 +
81 +     // it's possible some of the grads are not defined (represents tensors of
        all 0s).
82 +     // Since at::cat can't handle those, let's define them
83 +     std::vector<Tensor> grads_all_defined(grads.size());
84 +     for (size_t j = 0; j < grads.size(); ++j) {

```

```

85 +     if (grads[j].defined()) {
86 +         grads_all_defined[j] = grads[j];
87 +     } else {
88 +         auto length = split_sizes[j];
89 +         auto grad_size = sizes.vec();
90 +         grad_size[dim] = length;
91 +         grads_all_defined[j] = at::zeros(grad_size, options);
92 +     }
93 + }
94 +
95 + auto ret = at::cat(grads_all_defined, dim);
96 + return ret;
97 +}
98 +
99 } } // namespace at::native

```

!!新增 aten/src/ATen/native/Checkpoint.cpp 添加算子表层实现

DTR

```

1  diff --git a/aten/src/ATen/native/Checkpoint.cpp
    b/aten/src/ATen/native/Checkpoint.cpp
2  new file mode 100644
3  index 000000000000..d6e01b9ffc
4  --- /dev/null
5  +++ b/aten/src/ATen/native/Checkpoint.cpp
6  @@ -0,0 +1,1654 @@
7  +#include <ATen/ATen.h>
8  +#include <ATen/CheckpointTensorImpl.h>
9  +
10 +namespace at { namespace native {
11 +
12 +Tensor checkpoint_add(const Tensor& a, const Tensor& b, c10::Scalar c) {
13 +    rematerialize_function_t rt =
14 +        [=](const Tensors& vec) -> Tensors {
15 +            return {at::add(vec.at(0), vec.at(1), c)};
16 +        };
17 +    return CheckpointTensorImpl::make("add", rt, {a, b})[0];
18 +}
19 +
20 +Tensor checkpoint_t(at::Tensor const& a) {
21 +    rematerialize_function_t rt =

```



```

22 +     [=](const Tensors& vec) -> Tensors {
23 +         return {at::t(vec.at(0))};
24 +     };
25 +     return CheckpointTensorImpl::make("t", rt, {a})[0];
26 +}
27 +
28 +Tensor checkpoint_add(at::Tensor const& a, c10::Scalar b, c10::Scalar c) {
29 +     rematerialize_function_t rt =
30 +         [=](const Tensors& vec) -> Tensors {
31 +             return {at::add(vec.at(0), b, c)};
32 +         };
33 +     return CheckpointTensorImpl::make("add", rt, {a})[0];
34 +}
35 +
36 +Tensor& checkpoint_add_(Tensor& a, const Tensor& b, Scalar c) {
37 +     mutate_function_t mt =
38 +         [=](const Tensors& vec) {
39 +             vec.at(0).add_(vec.at(1), c);
40 +         };
41 +     CheckpointTensorImpl::mutate("add_", mt, {a, b}, {0});
42 +     return a;
43 +}
44 +
45 +Tensor checkpoint_mul(at::Tensor const& a, at::Tensor const& b) {
46 +     rematerialize_function_t rt =
47 +         [=](const Tensors& vec) -> Tensors {
48 +             return {at::mul(vec.at(0), vec.at(1))};
49 +         };
50 +     return CheckpointTensorImpl::make("mul", rt, {a, b})[0];
51 +}
52 +
53 +Tensor& checkpoint_mul_(at::Tensor& a, at::Tensor const& b) {
54 +     mutate_function_t mt =
55 +         [=](const Tensors& vec) {
56 +             vec.at(0).mul_(vec.at(1));
57 +         };
58 +     CheckpointTensorImpl::mutate("mul_", mt, {a, b}, {0});
59 +     return a;
60 +}
61 +
62 +Tensor& checkpoint_mul_(at::Tensor& a, c10::Scalar b) {
63 +     mutate_function_t mt =
64 +         [=](const Tensors& vec) {
65 +             vec.at(0).mul_(b);
66 +         };
67 +     CheckpointTensorImpl::mutate("mul_", mt, {a}, {0});
68 +     return a;

```

```

69  +}
70  +
71  +Tensor checkpoint_zeros_like(at::Tensor const& a, c10::TensorOptions const&
    b, c10::optional<c10::MemoryFormat> c) {
72  +  rematerialize_function_t rt =
73  +    [=](const Tensors& vec) -> Tensors {
74  +      return {at::zeros_like(vec.at(0), b, c)};
75  +    };
76  +  return CheckpointTensorImpl::make("zeros_like", rt, {a})[0];
77  +}
78  +
79  +Tensor checkpoint_ones_like(at::Tensor const& a, c10::TensorOptions const&
    b, c10::optional<c10::MemoryFormat> c) {
80  +  rematerialize_function_t rt =
81  +    [=](const Tensors& vec) -> Tensors {
82  +      return {at::ones_like(vec.at(0), b, c)};
83  +    };
84  +  return CheckpointTensorImpl::make("ones_like", rt, {a})[0];
85  +}
86  +
87  +Tensor checkpoint_addcmul(at::Tensor const& a, at::Tensor const& b,
    at::Tensor const& c, c10::Scalar d) {
88  +  rematerialize_function_t rt =
89  +    [=](const Tensors& vec) -> Tensors {
90  +      return {at::addcmul(vec.at(0), vec.at(1), vec.at(2), d)};
91  +    };
92  +  return CheckpointTensorImpl::make("addcmul", rt, {a, b, c})[0];
93  +}
94  +
95  +Tensor& checkpoint_addcmul_(at::Tensor& a, at::Tensor const& b, at::Tensor
    const& c, c10::Scalar d) {
96  +  mutate_function_t mt =
97  +    [=](const Tensors& vec) {
98  +      vec.at(0).addcmul_(vec.at(1), vec.at(2), d);
99  +    };
100  +  CheckpointTensorImpl::mutate("addcmul_", mt, {a, b, c}, {0});
101  +  return a;
102  +}
103  +
104  +Tensor checkpoint_abs(const Tensor& a) {
105  +  rematerialize_function_t rt =
106  +    [=](const Tensors& vec) -> Tensors {
107  +      return {at::abs(vec.at(0))};
108  +    };
109  +  return CheckpointTensorImpl::make("abs", rt, {a})[0];
110  +}
111  +

```

```

112 +Tensor checkpoint_sqrt(const Tensor& a) {
113 +   rematerialize_function_t rt =
114 +       [=](const Tensors& vec) -> Tensors {
115 +           return {at::sqrt(vec.at(0))};
116 +       };
117 +   return CheckpointTensorImpl::make("sqrt", rt, {a})[0];
118 +}
119 +
120 +Tensor& checkpoint_addcddiv_(at::Tensor& a, at::Tensor const& b, at::Tensor
    const& c, c10::Scalar d) {
121 +   mutate_function_t mt =
122 +       [=](const Tensors& vec) {
123 +           vec.at(0).addcddiv_(vec.at(1), vec.at(2), d);
124 +       };
125 +   CheckpointTensorImpl::mutate("addcddiv_", mt, {a, b, c}, {0});
126 +   return a;
127 +}
128 +
129 +Tensor checkpoint_addcddiv(at::Tensor const& a, at::Tensor const& b,
    at::Tensor const& c, c10::Scalar d) {
130 +   rematerialize_function_t rt =
131 +       [=](const Tensors& vec) -> Tensors {
132 +           return {at::addcddiv(vec.at(0), vec.at(1), vec.at(2), d)};
133 +       };
134 +   return CheckpointTensorImpl::make("addcddiv", rt, {a, b, c})[0];
135 +}
136 +
137 +Tensor checkpoint_to(at::Tensor const& a, c10::TensorOptions const& b, bool
    c, bool d, c10::optional<c10::MemoryFormat> e) {
138 +   c10::TensorOptions b_ = b;
139 +   rematerialize_function_t rt =
140 +       [=](const Tensors& vec) -> Tensors {
141 +           return {vec.at(0).to(b_, c, d, e)};
142 +       };
143 +   return CheckpointTensorImpl::make("to", rt, {a})[0];
144 +}
145 +
146 +Tensor checkpoint_div(const Tensor& a, const Tensor& b) {
147 +   rematerialize_function_t rt =
148 +       [=](const Tensors& vec) -> Tensors {
149 +           return {at::div(vec.at(0), vec.at(1))};
150 +       };
151 +   return CheckpointTensorImpl::make("div", rt, {a, b})[0];
152 +}
153 +
154 +Tensor& checkpoint_div_(Tensor& a, const Tensor& b) {
155 +   mutate_function_t mt =

```

```

156 +     [=](const Tensors& vec) {
157 +         vec.at(0).div_(vec.at(1));
158 +     };
159 +     CheckpointTensorImpl::mutate("div_", mt, {a, b}, {0});
160 +     return a;
161 + }
162 +
163 +Tensor checkpoint_clone(at::Tensor const& a,
164 +    c10::optional<c10::MemoryFormat> b) {
165 +     rematerialize_function_t rt =
166 +     [=](const Tensors& vec) -> Tensors {
167 +         return {at::clone(vec.at(0), b)};
168 +     };
169 +     return CheckpointTensorImpl::make("clone", rt, {a})[0];
170 + }
171 +Tensor checkpoint_where(at::Tensor const& a, at::Tensor const& b, at::Tensor
172 +    const& c) {
173 +     rematerialize_function_t rt =
174 +     [=](const Tensors& vec) -> Tensors {
175 +         return {at::where(vec.at(0), vec.at(1), vec.at(2))};
176 +     };
177 +     return CheckpointTensorImpl::make("where", rt, {a, b, c})[0];
178 + }
179 +Tensor checkpoint_constant_pad_nd(Tensor const& a, c10::ArrayRef<long> b,
180 +    c10::Scalar c) {
181 +     std::vector<long> b_ = b.vec();
182 +     rematerialize_function_t rt =
183 +     [=](const Tensors& vec) -> Tensors {
184 +         return {at::constant_pad_nd(vec.at(0), b_, c)};
185 +     };
186 +     return CheckpointTensorImpl::make("constant_pad_nd", rt, {a})[0];
187 + }
188 +Tensor checkpoint_binary_cross_entropy(const Tensor& a, const Tensor& b,
189 +    const Tensor& c, long d) {
190 +     rematerialize_function_t rt =
191 +     [=](const Tensors& vec) -> Tensors {
192 +         return {at::binary_cross_entropy(vec.at(0), vec.at(1), vec.at(2), d)};
193 +     };
194 +     return CheckpointTensorImpl::make("binary_cross_entropy", rt, {a, b, c})
195 +     [0];
196 + }
197 +
198 +Tensor& checkpoint_binary_cross_entropy_out(Tensor& a, const Tensor& b,
199 +    const Tensor& c, const Tensor& d, long e) {

```

```

197 + mutate_function_t mt =
198 +     [=](const Tensors& vec) {
199 +         Tensor self = vec.at(0);
200 +         at::binary_cross_entropy_out(self, vec.at(1), vec.at(2), vec.at(3), e);
201 +     };
202 + CheckpointTensorImpl::mutate("binary_cross_entropy_out", mt, {a, b, c, d},
    {0});
203 + return a;
204 +}
205 +
206 +Tensor checkpoint_binary_cross_entropy_backward(const Tensor& a, const
    Tensor& b, const Tensor& c, const Tensor& d, long e) {
207 + rematerialize_function_t rt =
208 +     [=](const Tensors& vec) -> Tensors {
209 +         return {at::binary_cross_entropy_backward(vec.at(0), vec.at(1),
    vec.at(2), vec.at(3), e)};
210 +     };
211 + return CheckpointTensorImpl::make("binary_cross_entropy_backward", rt, {a,
    b, c, d})[0];
212 +}
213 +
214 +Tensor& checkpoint_binary_cross_entropy_backward_out(Tensor& a, const
    Tensor& b, const Tensor& c, const Tensor& d, const Tensor& e, long f) {
215 + mutate_function_t mt =
216 +     [=](const Tensors& vec) {
217 +         Tensor self = vec.at(0);
218 +         at::binary_cross_entropy_backward_out(self, vec.at(1), vec.at(2),
    vec.at(3), vec.at(4), f);
219 +     };
220 + CheckpointTensorImpl::mutate("binary_cross_entropy_backward_out", mt, {a,
    b, c, d, e}, {0});
221 + return a;
222 +}
223 +
224 +Tensor checkpoint_embedding(const Tensor& a, const Tensor& b, long c, bool
    d, bool e) {
225 + rematerialize_function_t rt =
226 +     [=](const Tensors& vec) -> Tensors {
227 +         return {at::embedding(vec.at(0), vec.at(1), c, d, e)};
228 +     };
229 + return CheckpointTensorImpl::make("embedding", rt, {a, b})[0];
230 +}
231 +
232 +Tensor checkpoint_embedding_backward(const Tensor& a, const Tensor& b, long
    c, long d, bool e, bool f) {
233 + rematerialize_function_t rt =
234 +     [=](const Tensors& vec) -> Tensors {

```

```

235 +     return {at::embedding_backward(vec.at(0), vec.at(1), c, d, e, f)};
236 + };
237 + return CheckpointTensorImpl::make("embedding", rt, {a, b})[0];
238 +}
239 +
240 +std::tuple<Tensor, Tensor, Tensor, Tensor>
241 +checkpoint_cudnn_batch_norm(const Tensor& a, const Tensor& b, const Tensor&
242 +    c, const Tensor& d, const Tensor& e, bool f, double g, double h) {
243 +    rematerialize_function_t rt =
244 +    [=](const Tensors& vec) -> Tensors {
245 +        auto ret = at::cudnn_batch_norm(vec.at(0), vec.at(1), vec.at(2),
246 +            vec.at(3), vec.at(4), f, g, h);
247 +        return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret),
248 +            std::get<3>(ret)};
249 +    };
250 +    auto ret = CheckpointTensorImpl::make("cudnn_batch_norm", rt, {a, b, c, d,
251 +        e});
252 +    return {ret[0], ret[1], ret[2], ret[3]};
253 +}
254 +
255 +std::tuple<Tensor, Tensor, Tensor>
256 +checkpoint_cudnn_batch_norm_backward(at::Tensor const& a, at::Tensor const&
257 +    b, at::Tensor const& c, at::Tensor const& d, at::Tensor const& e, at::Tensor
258 +    const& f, at::Tensor const& g, double h, at::Tensor const& i) {
259 +    rematerialize_function_t rt =
260 +    [=](const Tensors& vec) -> Tensors {
261 +        auto ret = at::cudnn_batch_norm_backward(vec.at(0), vec.at(1),
262 +            vec.at(2), vec.at(3), vec.at(4), vec.at(5), vec.at(6), h, vec.at(7));
263 +        return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret)};
264 +    };
265 +    auto ret = CheckpointTensorImpl::make("cudnn_batch_norm_backward", rt, {a,
266 +        b, c, d, e, f, g, i});
267 +    return {ret[0], ret[1], ret[2]};
268 +}
269 +
270 +Tensor checkpoint_as_strided(const Tensor& a, c10::ArrayRef<long> b,
271 +    c10::ArrayRef<long> c, c10::optional<long> d) {
272 +    std::vector<long> b_ = b.vec(), c_ = c.vec();
273 +    rematerialize_function_t rt =
274 +    [=](const Tensors& vec) -> Tensors {
275 +        return {at::as_strided(vec.at(0), b_, c_, d)};
276 +    };
277 +    return CheckpointTensorImpl::make("as_strided", rt, {a})[0];
278 +}
279 +
280 +Tensor checkpoint__masked_scale(const Tensor& a, const Tensor& b, double c) {
281 +    rematerialize_function_t rt =

```

```

272 +     [=](const Tensors& vec) -> Tensors {
273 +         return {at::_masked_scale(vec.at(0), vec.at(1), c)};
274 +     };
275 +     return CheckpointTensorImpl::make("_masked_scale", rt, {a, b})[0];
276 +}
277 +
278 +Tensor checkpoint_cudnn_convolution(const Tensor& a, const Tensor& b,
c10::ArrayRef<long> c, c10::ArrayRef<long> d, c10::ArrayRef<long> e, long f,
bool g, bool h, bool i) {
279 +     std::vector<long> c_ = c.vec(), d_ = d.vec(), e_ = e.vec();
280 +     rematerialize_function_t rt =
281 +         [=](const Tensors& vec) -> Tensors {
282 +             return {at::cudnn_convolution(vec.at(0), vec.at(1), c_, d_, e_, f, g,
h, i)};
283 +         };
284 +     return CheckpointTensorImpl::make("cudnn_convolution", rt, {a, b})[0];
285 +}
286 +
287 +Tensor checkpoint_cudnn_convolution_transpose(const Tensor& a, const Tensor&
b, c10::ArrayRef<long> c, c10::ArrayRef<long> d, c10::ArrayRef<long> e,
c10::ArrayRef<long> f, long g, bool h, bool i, bool j) {
288 +     std::vector<long> c_ = c.vec(), d_ = d.vec(), e_ = e.vec(), f_ = f.vec();
289 +     rematerialize_function_t rt =
290 +         [=](const Tensors& vec) -> Tensors {
291 +             return {at::cudnn_convolution_transpose(vec.at(0), vec.at(1), c_, d_,
e_, f_, g, h, i, j)};
292 +         };
293 +     return CheckpointTensorImpl::make("cudnn_convolution_transpose", rt, {a,
b})[0];
294 +}
295 +
296 +std::tuple<Tensor, Tensor> checkpoint_cudnn_convolution_backward(const
Tensor& a, const Tensor& b, const Tensor& c, c10::ArrayRef<long> d,
c10::ArrayRef<long> e, c10::ArrayRef<long> f, long g, bool h, bool i, bool j,
std::array<bool, 2ul> k) {
297 +     std::vector<long> d_ = d.vec(), e_ = e.vec(), f_ = f.vec();
298 +     rematerialize_function_t rt =
299 +         [=](const Tensors& vec) -> Tensors {
300 +             auto ret = at::cudnn_convolution_backward(vec.at(0), vec.at(1),
vec.at(2), d_, e_, f_, g, h, i, j, k);
301 +             return {std::get<0>(ret), std::get<1>(ret)};
302 +         };
303 +     auto ret = CheckpointTensorImpl::make("cudnn_convolution_backward", rt,
{a, b, c});
304 +     return {ret[0], ret[1]};
305 +}
306 +

```

```

307 +std::tuple<Tensor, Tensor>
    checkpoint_cudnn_convolution_transpose_backward(const Tensor& a, const
    Tensor& b, const Tensor& c, c10::ArrayRef<long> d, c10::ArrayRef<long> e,
    c10::ArrayRef<long> f, c10::ArrayRef<long> g, long h, bool i, bool j, bool k,
    std::array<bool, 2ul> l) {
308 +   std::vector<long> d_ = d.vec(), e_ = e.vec(), f_ = f.vec(), g_ = g.vec();
309 +   rematerialize_function_t rt =
310 +       [=](const Tensors& vec) -> Tensors {
311 +           auto ret = at::cudnn_convolution_transpose_backward(vec.at(0),
    vec.at(1), vec.at(2), d_, e_, f_, g_, h, i, j, k, l);
312 +           return {std::get<0>(ret), std::get<1>(ret)};
313 +       };
314 +   auto ret =
    CheckpointTensorImpl::make("cudnn_convolution_transpose_backward", rt, {a, b,
    c});
315 +   return {ret[0], ret[1]};
316 +}
317 +
318 +Tensor checkpoint_cudnn_convolution_backward_input(c10::ArrayRef<long> a,
    const Tensor& b, const Tensor& c, c10::ArrayRef<long> d, c10::ArrayRef<long>
    e, c10::ArrayRef<long> f, long g, bool h, bool i, bool j) {
319 +   std::vector<long> a_ = a.vec(), d_ = d.vec(), e_ = e.vec(), f_ = f.vec();
320 +   rematerialize_function_t rt =
321 +       [=](const Tensors& vec) -> Tensors {
322 +           return {at::cudnn_convolution_backward_input(a_, vec.at(0), vec.at(1),
    d_, e_, f_, g, h, i, j)};
323 +       };
324 +   return CheckpointTensorImpl::make("cudnn_convolution_backward_input", rt,
    {b, c})[0];
325 +}
326 +
327 +Tensor checkpoint_cudnn_convolution_transpose_backward_input(const Tensor&
    a, const Tensor& b, c10::ArrayRef<long> c, c10::ArrayRef<long> d,
    c10::ArrayRef<long> e, long f, bool g, bool h, bool i) {
328 +   std::vector<long> c_ = c.vec(), d_ = d.vec(), e_ = e.vec();
329 +   rematerialize_function_t rt =
330 +       [=](const Tensors& vec) -> Tensors {
331 +           return {at::cudnn_convolution_transpose_backward_input(vec.at(0),
    vec.at(1), c_, d_, e_, f, g, h, i)};
332 +       };
333 +   return
    CheckpointTensorImpl::make("cudnn_convolution_transpose_backward_input", rt,
    {a, b})[0];
334 +}
335 +
336 +Tensor checkpoint_cudnn_convolution_backward_weight(IntArrayRef a, const
    Tensor & b, const Tensor & c, IntArrayRef d, IntArrayRef e, IntArrayRef f,

```



```

    int64_t g, bool h, bool i, bool j) {
337 +   std::vector<long> a_ = a.vec(), d_ = d.vec(), e_ = e.vec(), f_ = f.vec();
338 +   rematerialize_function_t rt =
339 +       [=](const Tensors& vec) -> Tensors {
340 +           return {at::cudnn_convolution_backward_weight(a_, vec.at(0),
vec.at(1), d_, e_, f_, g, h, i, j)};
341 +       };
342 +   return CheckpointTensorImpl::make("cudnn_convolution_backward_weight", rt,
{b, c})[0];
343 +}
344 +
345 +Tensor
checkpoint_cudnn_convolution_transpose_backward_weight(c10::ArrayRef<long> a,
const Tensor& b, const Tensor& c, c10::ArrayRef<long> d, c10::ArrayRef<long>
e, c10::ArrayRef<long> f, long g, bool h, bool i, bool j) {
346 +   std::vector<long> a_ = a.vec(), d_ = d.vec(), e_ = e.vec(), f_ = f.vec();
347 +   rematerialize_function_t rt =
348 +       [=](const Tensors& vec) -> Tensors {
349 +           return {at::cudnn_convolution_transpose_backward_weight(a_, vec.at(0),
vec.at(1), d_, e_, f_, g, h, i, j)};
350 +       };
351 +   return
CheckpointTensorImpl::make("cudnn_convolution_transpose_backward_weight", rt,
{b, c})[0];
352 +}
353 +
354 +Tensor checkpoint_relu(const Tensor& a) {
355 +   rematerialize_function_t rt =
356 +       [=](const Tensors& vec) -> Tensors {
357 +           return {at::relu(vec.at(0))};
358 +       };
359 +   return CheckpointTensorImpl::make("relu", rt, {a})[0];
360 +}
361 +
362 +Tensor& checkpoint_relu_(Tensor& a) {
363 +   mutate_function_t mt =
364 +       [=](const Tensors& vec) {
365 +           vec.at(0).relu_();
366 +       };
367 +   CheckpointTensorImpl::mutate("relu_", mt, {a}, {0});
368 +   return a;
369 +}
370 +
371 +Tensor checkpoint_log(at::Tensor const& a) {
372 +   rematerialize_function_t rt =
373 +       [=](const Tensors& vec) -> Tensors {
374 +           return {at::log(vec.at(0))};

```

```

375 +     };
376 +     return CheckpointTensorImpl::make("log", rt, {a})[0];
377 +}
378 +
379 +Tensor& checkpoint_log_out(at::Tensor& a, at::Tensor const& b) {
380 +     mutate_function_t mt =
381 +         [=](const Tensors& vec) {
382 +             Tensor a_ = vec.at(0);
383 +             at::log_out(a_, vec.at(1));
384 +         };
385 +     CheckpointTensorImpl::mutate("log_out", mt, {a, b}, {0});
386 +     return a;
387 +}
388 +
389 +Tensor checkpoint_rsub(at::Tensor const& a, at::Tensor const& b, c10::Scalar
    c) {
390 +     rematerialize_function_t rt =
391 +         [=](const Tensors& vec) -> Tensors {
392 +             return {at::rsub(vec.at(0), vec.at(1), c)};
393 +         };
394 +     return CheckpointTensorImpl::make("rsub", rt, {a, b})[0];
395 +}
396 +
397 +Tensor checkpoint_rsub(at::Tensor const& a, c10::Scalar b, c10::Scalar c) {
398 +     rematerialize_function_t rt =
399 +         [=](const Tensors& vec) -> Tensors {
400 +             return {at::rsub(vec.at(0), b, c)};
401 +         };
402 +     return CheckpointTensorImpl::make("rsub", rt, {a})[0];
403 +}
404 +
405 +Tensor checkpoint_mul(at::Tensor const& a, c10::Scalar b) {
406 +     rematerialize_function_t rt =
407 +         [=](const Tensors& vec) -> Tensors {
408 +             return {at::mul(vec.at(0), b)};
409 +         };
410 +     return CheckpointTensorImpl::make("mul", rt, {a})[0];
411 +}
412 +
413 +std::tuple<Tensor&, Tensor&> checkpoint_max_pool2d_with_indices_out(Tensor&
    a, Tensor& b, const Tensor& c, c10::ArrayRef<long> d, c10::ArrayRef<long> e,
    c10::ArrayRef<long> f, c10::ArrayRef<long> g, bool h) {
414 +     std::vector<long> d_ = d.vec(), e_ = e.vec(), f_ = f.vec(), g_ = g.vec();
415 +     mutate_function_t mt =
416 +         [=](const Tensors& vec) {
417 +             Tensor a_ = vec.at(0), b_ = vec.at(1);
418 +             at::max_pool2d_with_indices_out(a_, b_, vec.at(2), d_, e_, f_, g_, h);

```

```

419 +     };
420 +     CheckpointTensorImpl::mutate("max_pool2d_with_indices_out", mt, {a, b, c},
    {0, 1});
421 +     return {a, b};
422 + }
423 +
424 + Tensor checkpoint_avg_pool2d(const Tensor& a, c10::ArrayRef<long> b,
    c10::ArrayRef<long> c, c10::ArrayRef<long> d, bool e, bool f,
    c10::optional<long> g) {
425 +     std::vector<long> b_ = b.vec(), c_ = c.vec(), d_ = d.vec();
426 +     rematerialize_function_t rt =
427 +         [=](const Tensors& vec) -> Tensors {
428 +             return {at::avg_pool2d(vec.at(0), b_, c_, d_, e, f, g)};
429 +         };
430 +     return CheckpointTensorImpl::make("avg_pool2d", rt, {a})[0];
431 + }
432 +
433 + Tensor checkpoint_avg_pool2d_backward(const Tensor& a, const Tensor& b,
    c10::ArrayRef<long> c, c10::ArrayRef<long> d, c10::ArrayRef<long> e, bool f,
    bool g, c10::optional<long> h) {
434 +     std::vector<long> c_ = c.vec(), d_ = d.vec(), e_ = e.vec();
435 +     rematerialize_function_t rt =
436 +         [=](const Tensors& vec) -> Tensors {
437 +             return {at::avg_pool2d_backward(vec.at(0), vec.at(1), c_, d_, e_, f,
    g, h)};
438 +         };
439 +     return CheckpointTensorImpl::make("avg_pool2d_backward", rt, {a, b})[0];
440 + }
441 +
442 + Tensor& checkpoint_avg_pool2d_out(Tensor& a, const Tensor& b,
    c10::ArrayRef<long> c, c10::ArrayRef<long> d, c10::ArrayRef<long> e, bool f,
    bool g, c10::optional<long> h) {
443 +     std::vector<long> c_ = c.vec(), d_ = d.vec(), e_ = e.vec();
444 +     mutate_function_t mt =
445 +         [=](const Tensors& vec) {
446 +             Tensor a_ = vec.at(0);
447 +             at::avg_pool2d_out(a_, vec.at(1), c_, d_, e_, f, g, h);
448 +         };
449 +     CheckpointTensorImpl::mutate("avg_pool2d_out", mt, {a, b}, {0});
450 +     return a;
451 + }
452 +
453 + Tensor& checkpoint_avg_pool2d_backward_grad_input(Tensor& a, const Tensor&
    b, const Tensor& c, c10::ArrayRef<long> d, c10::ArrayRef<long> e,
    c10::ArrayRef<long> f, bool g, bool h, c10::optional<long> i) {
454 +     std::vector<long> d_ = d.vec(), e_ = e.vec(), f_ = f.vec();
455 +     mutate_function_t mt =

```

```

456 +     [=](const Tensors& vec) {
457 +         Tensor a_ = vec.at(0);
458 +         at::avg_pool2d_backward_out(a_, vec.at(1), vec.at(2), d_, e_, f_, g,
459 +             h, i);
460 +     };
461 +     CheckpointTensorImpl::mutate("avg_pool2d_backward_grad_input", mt, {a, b,
462 +         c}, {0});
463 +     return a;
464 + }
465 +
466 + std::tuple<Tensor, Tensor> checkpoint_max_pool2d_with_indices(const Tensor&
467 +     a, c10::ArrayRef<long> b, c10::ArrayRef<long> c, c10::ArrayRef<long> d,
468 +     c10::ArrayRef<long> e, bool f) {
469 +     std::vector<long> b_ = b.vec(), c_ = c.vec(), d_ = d.vec(), e_ = e.vec();
470 +     rematerialize_function_t rt =
471 +     [=](const Tensors& vec) -> Tensors {
472 +         auto ret = at::max_pool2d_with_indices(vec.at(0), b_, c_, d_, e_, f);
473 +         return {std::get<0>(ret), std::get<1>(ret)};
474 +     };
475 +     auto ret = CheckpointTensorImpl::make("max_pool2d_backward", rt, {a});
476 +     return {ret[0], ret[1]};
477 + }
478 +
479 + Tensor& checkpoint_max_pool2d_with_indices_backward_grad_input(Tensor& a,
480 +     const Tensor& b, const Tensor& c, c10::ArrayRef<long> d, c10::ArrayRef<long>
481 +     e, c10::ArrayRef<long> f, c10::ArrayRef<long> g, bool h, const Tensor& i) {
482 +     std::vector<long> d_ = d.vec(), e_ = e.vec(), f_ = f.vec(), g_ = g.vec();
483 +     mutate_function_t mt =
484 +     [=](const Tensors& vec) {
485 +         Tensor a_ = vec.at(0);
486 +         at::max_pool2d_with_indices_backward_out(a_, vec.at(1), vec.at(2), d_,
487 +             e_, f_, g, h, vec.at(3));
488 +     };
489 +     CheckpointTensorImpl::mutate("max_pool2d_with_indices_backward_grad_input",
490 +         mt, {a, b, c, i}, {0});
491 +     return a;
492 + }
493 +
494 + Tensor checkpoint_max_pool2d_with_indices_backward(const Tensor& a, const
495 +     Tensor& b, c10::ArrayRef<long> c, c10::ArrayRef<long> d, c10::ArrayRef<long>
496 +     e, c10::ArrayRef<long> f, bool g, const Tensor& h) {
497 +     std::vector<long> c_ = c.vec(), d_ = d.vec(), e_ = e.vec(), f_ = f.vec();
498 +     rematerialize_function_t rt =
499 +     [=](const Tensors& vec) -> Tensors {
500 +         return {at::max_pool2d_with_indices_backward(vec.at(0), vec.at(1), c_,
501 +             d_, e_, f_, g, vec.at(2))};

```

```

491 +     };
492 +     return CheckpointTensorImpl::make("max_pool2d_with_indices_backward", rt,
    {a, b, h})[0];
493 +}
494 +
495 +Tensor checkpoint_view(const Tensor& a, c10::ArrayRef<long> b) {
496 +     std::vector<long> b_ = b.vec();
497 +     rematerialize_function_t rt =
498 +         [=](const Tensors& vec) -> Tensors {
499 +             return {vec.at(0).view(b_)};
500 +         };
501 +     return CheckpointTensorImpl::make("view", rt, {a})[0];
502 +}
503 +
504 +Tensor checkpoint_ne_Scalar(const Tensor& a, c10::Scalar b) {
505 +     rematerialize_function_t rt =
506 +         [=](const Tensors& vec) -> Tensors {
507 +             return {at::ne(vec.at(0), b)};
508 +         };
509 +     return CheckpointTensorImpl::make("ne_Scalar", rt, {a})[0];
510 +}
511 +
512 +Tensor& checkpoint_ne_Scalar_out(Tensor& a, const Tensor& b, c10::Scalar c) {
513 +     mutate_function_t mt =
514 +         [=](const Tensors& vec) {
515 +             Tensor a_ = vec.at(0);
516 +             at::ne_out(a_, vec.at(1), c);
517 +         };
518 +     CheckpointTensorImpl::mutate("ne_Scalar_out", mt, {a, b}, {0});
519 +     return a;
520 +}
521 +
522 +Tensor checkpoint_ne_Tensor(const Tensor& a, const Tensor& b) {
523 +     rematerialize_function_t rt =
524 +         [=](const Tensors& vec) -> Tensors {
525 +             return {at::ne(vec.at(0), vec.at(1))};
526 +         };
527 +     return CheckpointTensorImpl::make("ne_Tensor", rt, {a, b})[0];
528 +}
529 +
530 +Tensor& checkpoint_ne_Tensor_out(Tensor& a, const Tensor& b, const Tensor&
    c) {
531 +     mutate_function_t mt =
532 +         [=](const Tensors& vec) {
533 +             Tensor a_ = vec.at(0);
534 +             at::ne_out(a_, vec.at(1), vec.at(2));
535 +         };

```

```

536 + CheckpointTensorImpl::mutate("ne_Tensor_out", mt, {a, b, c}, {0});
537 + return a;
538 +}
539 +
540 +Tensor checkpoint_eq_Scalar(const Tensor& a, c10::Scalar b) {
541 +   rematerialize_function_t rt =
542 +       [=](const Tensors& vec) -> Tensors {
543 +           return {at::eq(vec.at(0), b)};
544 +       };
545 +   return CheckpointTensorImpl::make("eq_Scalar", rt, {a})[0];
546 +}
547 +
548 +Tensor& checkpoint_eq_Scalar_out(Tensor& a, const Tensor& b, c10::Scalar c) {
549 +   mutate_function_t mt =
550 +       [=](const Tensors& vec) {
551 +           Tensor a_ = vec.at(0);
552 +           at::eq_out(a_, vec.at(1), c);
553 +       };
554 +   CheckpointTensorImpl::mutate("eq_Scalar_out", mt, {a, b}, {0});
555 +   return a;
556 +}
557 +
558 +Tensor checkpoint_eq_Tensor(const Tensor& a, const Tensor& b) {
559 +   rematerialize_function_t rt =
560 +       [=](const Tensors& vec) -> Tensors {
561 +           return {at::eq(vec.at(0), vec.at(1))};
562 +       };
563 +   return CheckpointTensorImpl::make("eq_Tensor", rt, {a, b})[0];
564 +}
565 +
566 +Tensor& checkpoint_eq_Tensor_out(Tensor& a, const Tensor& b, const Tensor&
    c) {
567 +   mutate_function_t mt =
568 +       [=](const Tensors& vec) {
569 +           Tensor a_ = vec.at(0);
570 +           at::eq_out(a_, vec.at(1), vec.at(2));
571 +       };
572 +   CheckpointTensorImpl::mutate("eq_Tensor_out", mt, {a, b, c}, {0});
573 +   return a;
574 +}
575 +
576 +Tensor checkpoint_addmm(const Tensor& a, const Tensor& b, const Tensor& c,
    c10::Scalar d, c10::Scalar e) {
577 +   rematerialize_function_t rt =
578 +       [=](const Tensors& vec) -> Tensors {
579 +           return {at::addmm(vec.at(0), vec.at(1), vec.at(2), d, e)};
580 +       };

```

```

581 + return CheckpointTensorImpl::make("addmm", rt, {a, b, c})[0];
582 +}
583 +
584 +Tensor& checkpoint_addmm_out(Tensor& a, const Tensor& b, const Tensor& c,
    const Tensor& d, c10::Scalar e, c10::Scalar f) {
585 + mutate_function_t mt =
586 +     [=](const Tensors& vec) {
587 +         Tensor a_ = vec.at(0);
588 +         at::addmm_out(a_, vec.at(1), vec.at(2), d, e, f);
589 +     };
590 + CheckpointTensorImpl::mutate("addmm_out", mt, {a, b, c}, {0});
591 + return a;
592 +}
593 +
594 +Tensor& checkpoint_addmm_(Tensor& a, const Tensor& b, const Tensor& c,
    c10::Scalar d, c10::Scalar e) {
595 + mutate_function_t mt =
596 +     [=](const Tensors& vec) {
597 +         Tensor a_ = vec.at(0);
598 +         a.addmm_(vec.at(1), vec.at(2), d, e);
599 +     };
600 + CheckpointTensorImpl::mutate("addmm_", mt, {a, b, c}, {0});
601 + return a;
602 +}
603 +
604 +Tensor checkpoint_sigmoid(const Tensor& a) {
605 + rematerialize_function_t rt =
606 +     [=](const Tensors& vec) -> Tensors {
607 +         return {at::sigmoid(vec.at(0))};
608 +     };
609 + return CheckpointTensorImpl::make("sigmoid", rt, {a})[0];
610 +}
611 +
612 +Tensor& checkpoint_sigmoid_(Tensor& a) {
613 + mutate_function_t mt =
614 +     [=](const Tensors& vec) {
615 +         Tensor a_ = vec.at(0);
616 +         a.sigmoid_();
617 +     };
618 + CheckpointTensorImpl::mutate("sigmoid_", mt, {a}, {0});
619 + return a;
620 +}
621 +
622 +Tensor checkpoint__log_softmax(const Tensor& a, long b, bool c) {
623 + rematerialize_function_t rt =
624 +     [=](const Tensors& vec) -> Tensors {
625 +         return {at::_log_softmax(vec.at(0), b, c)};

```

```

626 +     };
627 +     return CheckpointTensorImpl::make("_log_softmax", rt, {a})[0];
628 +}
629 +
630 +Tensor checkpoint__log_softmax_backward_data(const Tensor& a, const Tensor&
        b, long c, const Tensor& d) {
631 +     rematerialize_function_t rt =
632 +         [=](const Tensors& vec) -> Tensors {
633 +             return {at::_log_softmax_backward_data(vec.at(0), vec.at(1), c,
                vec.at(2))};
634 +         };
635 +     return CheckpointTensorImpl::make("_log_softmax_backward_data", rt, {a, b,
        d})[0];
636 +}
637 +
638 +std::tuple<Tensor, Tensor> checkpoint_nll_loss_forward(const Tensor& a,
        const Tensor& b, const Tensor& c, long d, long e) {
639 +     rematerialize_function_t rt =
640 +         [=](const Tensors& vec) -> Tensors {
641 +             auto ret = at::nll_loss_forward(vec.at(0), vec.at(1), vec.at(2), d, e);
642 +             return {std::get<0>(ret), std::get<1>(ret)};
643 +         };
644 +     auto ret = CheckpointTensorImpl::make("nll_loss_forward", rt, {a, b, c});
645 +     return {ret[0], ret[1]};
646 +}
647 +
648 +std::tuple<Tensor&, Tensor&> checkpoint_nll_loss_forward_out(Tensor& a,
        Tensor& b, const Tensor& c, const Tensor& d, const Tensor& e, long f, long g)
        {
649 +     mutate_function_t mt =
650 +         [=](const Tensors& vec) {
651 +             Tensor a_ = vec.at(0);
652 +             Tensor b_ = vec.at(1);
653 +             at::nll_loss_forward_out(a_, b_, vec.at(2), vec.at(3), vec.at(4), f,
                g);
654 +         };
655 +     CheckpointTensorImpl::mutate("nll_loss_forward_out", mt, {a, b, c, d, e},
        {0, 1});
656 +     return {a, b};
657 +}
658 +
659 +Tensor checkpoint_nll_loss_backward(const Tensor& a, const Tensor& b, const
        Tensor& c, const Tensor& d, long e, long f, const Tensor& g) {
660 +     rematerialize_function_t rt =
661 +         [=](const Tensors& vec) -> Tensors {
662 +             return {at::nll_loss_backward(vec.at(0), vec.at(1), vec.at(2),
                vec.at(3), e, f, vec.at(4))};

```



```

663 +     };
664 +     return CheckpointTensorImpl::make("nll_loss_backward", rt, {a, b, c, d, g})
        [0];
665 +}
666 +
667 +Tensor& checkpoint_nll_loss_backward_grad_input(Tensor& a, const Tensor& b,
        const Tensor& c, const Tensor& d, const Tensor& e, long f, long g, const
        Tensor& h) {
668 +     mutate_function_t mt =
669 +         [=](const Tensors& vec) {
670 +             Tensor a_ = vec.at(0);
671 +             at::nll_loss_backward_out(a_, vec.at(1), vec.at(2), vec.at(3),
                vec.at(4), f, g, vec.at(5));
672 +         };
673 +     CheckpointTensorImpl::mutate("nll_loss_backward_grad_input", mt, {a, b, c,
        d, e, h}, {0});
674 +     return a;
675 +}
676 +
677 +Tensor checkpoint_mm(const Tensor& a, const Tensor& b) {
678 +     rematerialize_function_t rt =
679 +         [=](const Tensors& vec) -> Tensors {
680 +             return {at::mm(vec.at(0), vec.at(1))};
681 +         };
682 +     return CheckpointTensorImpl::make("mm", rt, {a, b})[0];
683 +}
684 +
685 +Tensor& checkpoint_mm_out(Tensor& a, const Tensor& b, const Tensor& c) {
686 +     mutate_function_t mt =
687 +         [=](const Tensors& vec) {
688 +             Tensor a_ = vec.at(0);
689 +             at::mm_out(a_, vec.at(1), vec.at(2));
690 +         };
691 +     CheckpointTensorImpl::mutate("mm_out", mt, {a, b, c}, {0});
692 +     return a;
693 +}
694 +
695 +Tensor checkpoint_sum(const Tensor& a, c10::optional<c10::ScalarType> b) {
696 +     rematerialize_function_t rt =
697 +         [=](const Tensors& vec) -> Tensors {
698 +             return {at::sum(vec.at(0), b)};
699 +         };
700 +     return CheckpointTensorImpl::make("sum", rt, {a})[0];
701 +}
702 +
703 +Tensor checkpoint_sum_dim_IntList(const Tensor& a, c10::ArrayRef<long> b,
        bool c, c10::optional<c10::ScalarType> d) {

```

```

704 + std::vector<long> b_ = b.vec();
705 + rematerialize_function_t rt =
706 +     [=](const Tensors& vec) -> Tensors {
707 +         return {at::sum(vec.at(0), b_, c, d)};
708 +     };
709 + return CheckpointTensorImpl::make("sum_dim_IntList", rt, {a})[0];
710 +}
711 +
712 +Tensor checkpoint_threshold(const Tensor& a, c10::Scalar b, c10::Scalar c) {
713 + rematerialize_function_t rt =
714 +     [=](const Tensors& vec) -> Tensors {
715 +         return {at::threshold(vec.at(0), b, c)};
716 +     };
717 + return CheckpointTensorImpl::make("threshold", rt, {a})[0];
718 +}
719 +
720 +Tensor& checkpoint_threshold_(Tensor& a, c10::Scalar b, c10::Scalar c) {
721 + mutate_function_t mt =
722 +     [=](const Tensors& vec) {
723 +         Tensor a_ = vec.at(0);
724 +         at::threshold_(a_, b, c);
725 +     };
726 + CheckpointTensorImpl::mutate("threshold_", mt, {a}, {0});
727 + return a;
728 +}
729 +
730 +Tensor& checkpoint_threshold_out(Tensor& a, const Tensor& b, c10::Scalar c,
731 + c10::Scalar d) {
732 + mutate_function_t mt =
733 +     [=](const Tensors& vec) {
734 +         Tensor a_ = vec.at(0);
735 +         at::threshold_out(a_, b, c, d);
736 +     };
737 + CheckpointTensorImpl::mutate("threshold_out", mt, {a}, {0});
738 + return a;
739 +}
740 +
741 +Tensor checkpoint_threshold_backward(const Tensor& a, const Tensor& b,
742 + c10::Scalar c) {
743 + rematerialize_function_t rt =
744 +     [=](const Tensors& vec) -> Tensors {
745 +         return {at::threshold_backward(vec.at(0), vec.at(1), c)};
746 +     };
747 + return CheckpointTensorImpl::make("threshold_backward", rt, {a, b})[0];
748 +}
749 +
750 +Tensor checkpoint_select(const Tensor& a, long b, long c) {

```

```

749 + rematerialize_function_t rt =
750 +     [=](const Tensors& vec) -> Tensors {
751 +         return {at::select(vec.at(0), b, c)};
752 +     };
753 + return CheckpointTensorImpl::make("select", rt, {a})[0];
754 +}
755 +
756 +Tensor checkpoint_select_backward(const Tensor& a, c10::ArrayRef<long> b,
    long c, long d) {
757 + std::vector<long> b_ = b.vec();
758 + rematerialize_function_t rt =
759 +     [=](const Tensors& vec) -> Tensors {
760 +         return {at::select_backward(vec.at(0), b_, c, d)};
761 +     };
762 + return CheckpointTensorImpl::make("select_backward", rt, {a})[0];
763 +}
764 +
765 +Tensor checkpoint_slice(const Tensor& a, long b, long c, long d, long e) {
766 + rematerialize_function_t rt =
767 +     [=](const Tensors& vec) -> Tensors {
768 +         return {at::slice(vec.at(0), b, c, d, e)};
769 +     };
770 + return CheckpointTensorImpl::make("slice", rt, {a})[0];
771 +}
772 +
773 +Tensor checkpoint_slice_backward(const Tensor& a, c10::ArrayRef<long> b,
    long c, long d, long e, long f) {
774 + std::vector<long> b_ = b.vec();
775 + rematerialize_function_t rt =
776 +     [=](const Tensors& vec) -> Tensors {
777 +         return {at::slice_backward(vec.at(0), b_, c, d, e, f)};
778 +     };
779 + return CheckpointTensorImpl::make("slice_backward", rt, {a})[0];
780 +}
781 +
782 +Tensor& checkpoint_zero_(Tensor& a) {
783 + mutate_function_t mt =
784 +     [=](const Tensors& vec) {
785 +         vec.at(0).zero_();
786 +     };
787 + CheckpointTensorImpl::mutate("zero_", mt, {a}, {0});
788 + return a;
789 +}
790 +
791 +Tensor& checkpoint_squeeze_(at::Tensor& a, at::Dimname b) {
792 + mutate_function_t mt =
793 +     [=](const Tensors& vec) {

```

```

794 +     vec.at(0).squeeze_(b);
795 + };
796 + CheckpointTensorImpl::mutate("squeeze_", mt, {a}, {0});
797 + return a;
798 +}
799 +
800 +Tensor& checkpoint_squeeze_(at::Tensor& a) {
801 +     mutate_function_t mt =
802 +         [=](const Tensors& vec) {
803 +             vec.at(0).squeeze_();
804 +         };
805 +     CheckpointTensorImpl::mutate("squeeze_", mt, {a}, {0});
806 +     return a;
807 +}
808 +
809 +Tensor& checkpoint_squeeze_(at::Tensor& a, long b) {
810 +     mutate_function_t mt =
811 +         [=](const Tensors& vec) {
812 +             vec.at(0).squeeze_(b);
813 +         };
814 +     CheckpointTensorImpl::mutate("squeeze_", mt, {a}, {0});
815 +     return a;
816 +}
817 +
818 +Tensor checkpoint_sigmoid_backward(at::Tensor const& a, at::Tensor const& b)
819 +{
820 +     rematerialize_function_t rt =
821 +         [=](const Tensors& vec) -> Tensors {
822 +             return {at::sigmoid_backward(vec.at(0), vec.at(1))};
823 +         };
824 +     return CheckpointTensorImpl::make("sigmoid_backward", rt, {a, b})[0];
825 +}
826 +
827 +Tensor& checkpoint_sigmoid_backward_out(at::Tensor& a, at::Tensor const& b,
828 +    at::Tensor const& c) {
829 +     mutate_function_t mt =
830 +         [=](const Tensors& vec) {
831 +             Tensor a_ = vec.at(0);
832 +             at::sigmoid_backward_out(a_, vec.at(1), vec.at(2));
833 +         };
834 +     CheckpointTensorImpl::mutate("sigmoid_backward_out", mt, {a, b, c}, {0});
835 +     return a;
836 +}
837 +
838 +Tensor& checkpoint_sign_out(at::Tensor& a, at::Tensor const& b) {
839 +     mutate_function_t mt =
840 +         [=](const Tensors& vec) {

```

```

839 +     Tensor a_ = vec.at(0);
840 +     at::sign_out(a_, vec.at(1));
841 + };
842 + CheckpointTensorImpl::mutate("sign_out", mt, {a, b}, {0});
843 + return a;
844 +}
845 +
846 +Tensor checkpoint_sign(const Tensor& a) {
847 + rematerialize_function_t rt =
848 + [=](const Tensors& vec) -> Tensors {
849 +     return {at::sign(vec.at(0))};
850 + };
851 + return CheckpointTensorImpl::make("sign", rt, {a})[0];
852 +}
853 +
854 +Tensor checkpoint_tanh(const Tensor& a) {
855 + rematerialize_function_t rt =
856 + [=](const Tensors& vec) -> Tensors {
857 +     return {at::tanh(vec.at(0))};
858 + };
859 + return CheckpointTensorImpl::make("tanh", rt, {a})[0];
860 +}
861 +
862 +Tensor checkpoint_tanh_backward(at::Tensor const& a, at::Tensor const& b) {
863 + rematerialize_function_t rt =
864 + [=](const Tensors& vec) -> Tensors {
865 +     return {at::tanh_backward(vec.at(0), vec.at(1))};
866 + };
867 + return CheckpointTensorImpl::make("tanh_backward", rt, {a, b})[0];
868 +}
869 +
870 +Tensor& checkpoint_tanh_backward_out(at::Tensor& a, at::Tensor const& b,
at::Tensor const& c) {
871 + mutate_function_t mt =
872 + [=](const Tensors& vec) {
873 +     Tensor a_ = vec.at(0);
874 +     at::tanh_backward_out(a_, vec.at(1), vec.at(2));
875 + };
876 + CheckpointTensorImpl::mutate("tanh_backward_out", mt, {a, b, c}, {0});
877 + return a;
878 +}
879 +
880 +Tensor checkpoint_neg(at::Tensor const& a) {
881 + rematerialize_function_t rt =
882 + [=](const Tensors& vec) -> Tensors {
883 +     return {at::neg(vec.at(0))};
884 + };

```

```

885 + return CheckpointTensorImpl::make("neg", rt, {a})[0];
886 +}
887 +
888 +Tensor checkpoint_sub(at::Tensor const& a, at::Tensor const& b, c10::Scalar
c) {
889 +   rematerialize_function_t rt =
890 +       [=](const Tensors& vec) -> Tensors {
891 +           return {at::sub(vec.at(0), vec.at(1), c)};
892 +       };
893 +   return CheckpointTensorImpl::make("sub", rt, {a, b})[0];
894 +}
895 +
896 +Tensor& checkpoint_sub_(at::Tensor& a, at::Tensor const& b, c10::Scalar c) {
897 +   mutate_function_t mt =
898 +       [=](const Tensors& vec) {
899 +           Tensor self = vec.at(0);
900 +           self.sub_(vec.at(1), c);
901 +       };
902 +   CheckpointTensorImpl::mutate("sub_", mt, {a, b}, {0});
903 +   return a;
904 +}
905 +
906 +Tensor checkpoint_repeat(const at::Tensor& a, c10::ArrayRef<long> b) {
907 +   std::vector<long> b_ = b.vec();
908 +   rematerialize_function_t rt =
909 +       [=](const Tensors& vec) -> Tensors {
910 +           return {vec.at(0).repeat(b_)};
911 +       };
912 +   return CheckpointTensorImpl::make("repeat", rt, {a})[0];
913 +}
914 +
915 +Tensor checkpoint_mean(const Tensor& self, c10::optional<c10::ScalarType>
dtype) {
916 +   rematerialize_function_t rt =
917 +       [=](const Tensors& vec) -> Tensors {
918 +           return {at::native::mean_cpu_gpu(vec[0], dtype)};
919 +       };
920 +   return CheckpointTensorImpl::make("mean", rt, {self})[0];
921 +}
922 +
923 +Tensor checkpoint_mean(const Tensor& self, IntArrayRef dim, bool keepdim,
c10::optional<c10::ScalarType> dtype) {
924 +   std::vector<long> dim_ = dim.vec();
925 +   rematerialize_function_t rt =
926 +       [=](const Tensors& vec) -> Tensors {
927 +           return {at::native::mean_cpu_gpu(vec[0], dim_, keepdim, dtype)};
928 +       };

```

```

929 + return CheckpointTensorImpl::make("mean.dim", rt, {self})[0];
930 +}
931 +
932 +Tensor checkpoint__cat(c10::ArrayRef<Tensor> a, long b) {
933 + rematerialize_function_t rt =
934 + [=](const Tensors& vec) -> Tensors {
935 +     return {at::cat(vec, b)};
936 + };
937 + std::vector<Tensor> s;
938 + for (const Tensor& t : a) {
939 +     s.push_back(t);
940 + }
941 + return CheckpointTensorImpl::make("_cat", rt, s)[0];
942 +}
943 +
944 +Tensor& checkpoint__cat_out(Tensor& a, c10::ArrayRef<Tensor> b, long c) {
945 + std::vector<Tensor> args;
946 + args.push_back(a);
947 + for (const Tensor& t : b) {
948 +     args.push_back(t);
949 + }
950 + mutate_function_t mt =
951 + [=](const Tensors& vec) {
952 +     Tensor t = vec[0];
953 +     at::cat_out(t, ArrayRef<Tensor>(vec.data() + 1, vec.size() - 1), c);
954 + };
955 + CheckpointTensorImpl::mutate("_cat_out", mt, args, {0});
956 + return a;
957 +}
958 +
959 +Tensor checkpoint_kl_div(at::Tensor const& a, at::Tensor const& b, long c,
    bool d) {
960 + rematerialize_function_t rt =
961 + [=](const Tensors& vec) -> Tensors {
962 +     return {at::kl_div(vec.at(0), vec.at(1), c, d)};
963 + };
964 + return CheckpointTensorImpl::make("kl_div", rt, {a, b})[0];
965 +}
966 +
967 +Tensor checkpoint_kl_div_backward(at::Tensor const& a, at::Tensor const& b,
    at::Tensor const& c, long d, bool e) {
968 + rematerialize_function_t rt =
969 + [=](const Tensors& vec) -> Tensors {
970 +     return {at::kl_div_backward(vec.at(0), vec.at(1), vec.at(2), d, e)};
971 + };
972 + return CheckpointTensorImpl::make("kl_div_backward", rt, {a, b, c})[0];
973 +}

```

```

974 +
975 +Tensor checkpoint_upsample_bilinear2d(at::Tensor const& self,
    c10::ArrayRef<long> output_size, bool align_corners, c10::optional<double>
    scales_h, c10::optional<double> scales_w) {
976 + std::vector<long> output_size_ = output_size.vec();
977 + rematerialize_function_t rt =
978 +     [=](const Tensors& vec) -> Tensors {
979 +     return {at::upsample_bilinear2d(vec.at(0), output_size_, align_corners,
    scales_h, scales_w)};
980 + };
981 + return CheckpointTensorImpl::make("upsample_bilinear2d", rt, {self})[0];
982 +}
983 +
984 +Tensor& checkpoint_upsample_bilinear2d_out(at::Tensor& out, const
    at::Tensor& self, c10::ArrayRef<long> output_size, bool align_corners,
    c10::optional<double> scales_h, c10::optional<double> scales_w) {
985 + std::vector<long> output_size_ = output_size.vec();
986 + mutate_function_t mt =
987 +     [=](const Tensors& vec) {
988 +     Tensor out = vec.at(0);
989 +     at::upsample_bilinear2d_out(out, vec.at(1), output_size_, align_corners,
    scales_h, scales_w);
990 + };
991 + CheckpointTensorImpl::mutate("binary_cross_entropy_out", mt, {out, self},
    {0});
992 + return out;
993 +}
994 +
995 +Tensor& checkpoint_upsample_bilinear2d_backward_out(at::Tensor& grad_input,
    const at::Tensor& grad_output, c10::ArrayRef<long> output_size,
    c10::ArrayRef<long> input_size, bool align_corners, c10::optional<double>
    scales_h, c10::optional<double> scales_w) {
996 + std::vector<long> output_size_ = output_size.vec();
997 + std::vector<long> input_size_ = input_size.vec();
998 + mutate_function_t mt =
999 +     [=](const Tensors& vec) {
1000 +     Tensor grad_input = vec.at(0);
1001 +     at::upsample_bilinear2d_backward_out(grad_input, vec.at(1),
    output_size_, input_size_, align_corners, scales_h, scales_w);
1002 + };
1003 + CheckpointTensorImpl::mutate("upsample_bilinear2d_backward_out", mt,
    {grad_input, grad_output}, {0});
1004 + return grad_input;
1005 +}
1006 +
1007 +Tensor checkpoint_upsample_bilinear2d_backward(at::Tensor const&
    grad_output, c10::ArrayRef<long> output_size, c10::ArrayRef<long> input_size,

```



```

    bool align_corners, c10::optional<double> scales_h, c10::optional<double>
    scales_w) {
1008 +   std::vector<long> output_size_ = output_size.vec();
1009 +   std::vector<long> input_size_ = input_size.vec();
1010 +   rematerialize_function_t rt =
1011 +       [=](const Tensors& vec) -> Tensors {
1012 +       return {at::upsample_bilinear2d_backward(vec.at(0), output_size_,
1013 +       input_size_, align_corners, scales_h, scales_w)};
1014 +   };
1015 +   return CheckpointTensorImpl::make("upsample_bilinear2d_backward", rt,
1016 +   {grad_output})[0];
1017 +}
1018 +
1019 +Tensor& checkpoint_clamp_min_(Tensor& a, Scalar min) {
1020 +   mutate_function_t mt =
1021 +       [=](const Tensors& vec) {
1022 +       Tensor self = vec.at(0);
1023 +       at::clamp_min_(self, min);
1024 +   };
1025 +   CheckpointTensorImpl::mutate("clamp_min_", mt, {a}, {0});
1026 +   return a;
1027 +}
1028 +
1029 +Tensor& checkpoint_clamp_min_out(Tensor& out, const Tensor& self, Scalar
1030 +min) {
1031 +   mutate_function_t mt =
1032 +       [=](const Tensors& vec) {
1033 +       Tensor out = vec.at(0);
1034 +       at::clamp_min_out(out, vec.at(1), min);
1035 +   };
1036 +   CheckpointTensorImpl::mutate("clamp_min__out", mt, {out, self}, {0});
1037 +   return out;
1038 +}
1039 +
1040 +Tensor checkpoint_binary_cross_entropy_with_logits(const Tensor& input,
1041 +const Tensor& target, const Tensor& weight, const Tensor& pos_weight, int64_t
1042 +reduction) {
1043 +   rematerialize_function_t rt =
1044 +       [=](const Tensors& vec) -> Tensors {
1045 +       return {at::binary_cross_entropy_with_logits(vec.at(0), vec.at(1),
1046 +       vec.at(2), vec.at(3), reduction)};
1047 +   };
1048 +   return CheckpointTensorImpl::make("binary_cross_entropy_with_logits", rt,
1049 +   {input, target, weight, pos_weight})[0];
1050 +}
1051 +

```

```

1045 +Tensor checkpoint_binary_cross_entropy_with_logits_backward(const Tensor&
      grad, const Tensor& input, const Tensor& target, const Tensor& weight, const
      Tensor& pos_weight, int64_t reduction) {
1046 +   rematerialize_function_t rt =
1047 +       [=](const Tensors& vec) -> Tensors {
1048 +       return {at::binary_cross_entropy_with_logits_backward(vec.at(0),
      vec.at(1), vec.at(2), vec.at(3), vec.at(4), reduction)};
1049 +   };
1050 +   return
      CheckpointTensorImpl::make("binary_cross_entropy_with_logits_backward", rt,
      {grad, input, target, weight, pos_weight})[0];
1051 +}
1052 +
1053 +std::tuple<Tensor, Tensor> checkpoint__fused_dropout(const Tensor & self,
      double p, c10::optional<Generator> g) {
1054 +   // TODO: Figure out how to properly duplicate the generator;
1055 +   // note that the commented-out code below results in a segfault!
1056 +   // Ref<std::shared_ptr<Generator>> gen;
1057 +   rematerialize_function_t rt =
1058 +       [=](const Tensors& vec) -> Tensors {
1059 +       // Generator* cur = gen.t ? gen.t.get() : g;
1060 +       // auto newG = cur->clone();
1061 +       // auto res = at::_fused_dropout(vec.at(0), p, cur);
1062 +       // gen.t = newG;
1063 +       auto res = at::_fused_dropout(vec.at(0), p, g);
1064 +       return {std::get<0>(res), std::get<1>(res)};
1065 +   };
1066 +   auto res = CheckpointTensorImpl::make("_fused_dropout_", rt, {self});
1067 +   return {res[0], res[1]};
1068 +}
1069 +
1070 +std::tuple<Tensor, Tensor> checkpoint_nll_loss2d_forward(at::Tensor const&
      self, const Tensor& target, const Tensor& weight, long reduction, long
      ignore_index) {
1071 +   rematerialize_function_t rt =
1072 +       [=](const Tensors& vec) -> Tensors {
1073 +       auto ret = at::nll_loss2d_forward(vec.at(0), vec.at(1), vec.at(2),
      reduction, ignore_index);
1074 +       return {std::get<0>(ret), std::get<1>(ret)};
1075 +   };
1076 +   auto ret = CheckpointTensorImpl::make("nll_loss2d_forward", rt, {self,
      target, weight});
1077 +   return {ret[0], ret[1]};
1078 +}
1079 +
1080 +std::tuple<Tensor&, Tensor&> checkpoint_nll_loss2d_forward_out(at::Tensor&
      output, at::Tensor& total_weight, const Tensor& self, const Tensor& target,

```

```

const Tensor& weight, long reduction, long ignore_index) {
1081 +   mutate_function_t mt =
1082 +       [=](const Tensors& vec) {
1083 +           Tensor out_ = vec.at(0);
1084 +           Tensor total_weight_ = vec.at(1);
1085 +           at::nll_loss2d_forward_out(out_, total_weight_, vec.at(2),
vec.at(3), vec.at(4), reduction, ignore_index);
1086 +       };
1087 +   CheckpointTensorImpl::mutate("nll_loss2d_forward_out", mt, {output,
total_weight, self, target, weight}, {0, 1});
1088 +   return {output, total_weight};
1089 +}
1090 +
1091 +Tensor& checkpoint_nll_loss2d_backward_out(Tensor& grad_input, const Tensor&
grad_output, const Tensor& self, const Tensor& target, const Tensor& weight,
long reduction, long ignore_index, const Tensor& total_weight) {
1092 +   mutate_function_t mt =
1093 +       [=](const Tensors& vec) {
1094 +           Tensor grad_input_ = vec.at(0);
1095 +           at::nll_loss2d_backward_out(grad_input_, vec.at(1), vec.at(2),
vec.at(3), vec.at(4), reduction, ignore_index, vec.at(5));
1096 +       };
1097 +   CheckpointTensorImpl::mutate("nll_loss2d_backward_out", mt, {grad_input,
grad_output, self, target, weight, total_weight}, {0});
1098 +   return {grad_input};
1099 +}
1100 +
1101 +Tensor checkpoint_nll_loss2d_backward(const Tensor& grad_output, const
Tensor& self, const Tensor& target, const Tensor& weight, long reduction,
long ignore_index, const Tensor& total_weight) {
1102 +   rematerialize_function_t rt =
1103 +       [=](const Tensors& vec) -> Tensors {
1104 +           return {at::nll_loss2d_backward(vec.at(0), vec.at(1), vec.at(2),
vec.at(3), reduction, ignore_index, vec.at(4))};
1105 +       };
1106 +   return CheckpointTensorImpl::make("nll_loss2d_backward", rt, {grad_output,
self, target, weight, total_weight})[0];
1107 +}
1108 +
1109 +std::tuple<Tensor, Tensor, Tensor> checkpoint__thnn_fused_lstm_cell(const
Tensor& input_gates, const Tensor& hidden_gates, const Tensor& cx, const
Tensor& input_bias, const Tensor& hidden_bias) {
1110 +   rematerialize_function_t rt =
1111 +       [=](const Tensors& vec) -> Tensors {
1112 +           auto res = at::_thnn_fused_lstm_cell(vec.at(0), vec.at(1), vec.at(2),
1113 +               vec.at(3), vec.at(4));
1114 +           return {std::get<0>(res), std::get<1>(res), std::get<2>(res)};

```

```

1115 + };
1116 + auto res = CheckpointTensorImpl::make("_thnn_fused_lstm_cell", rt,
1117 +                                     {input_gates, hidden_gates, cx,
                                     input_bias, hidden_bias});
1118 + return {res[0], res[1], res[2]};
1119 +}
1120 +
1121 +std::tuple<Tensor, Tensor, Tensor, Tensor, Tensor>
checkpoint__thnn_fused_lstm_cell_backward(const Tensor& grad_hy, const
Tensor& grad_cy, const Tensor& cx, const Tensor& cy, const Tensor& workspace,
bool has_bias) {
1122 + rematerialize_function_t rt =
1123 +     [=](const Tensors& vec) -> Tensors {
1124 +         auto res = at::_thnn_fused_lstm_cell_backward(vec.at(0), vec.at(1),
vec.at(2), vec.at(3), vec.at(4), has_bias);
1125 +         return {std::get<0>(res), std::get<1>(res),
1126 +                 std::get<2>(res), std::get<3>(res), std::get<4>(res)};
1127 +     };
1128 + auto res = CheckpointTensorImpl::make("_thnn_fused_lstm_cell_backward", rt,
1129 +                                     {grad_hy, grad_cy, cx, cy,
                                     workspace});
1130 + return {res[0], res[1], res[2], res[3], res[4]};
1131 +}
1132 +
1133 +std::tuple<Tensor, Tensor> checkpoint__thnn_fused_gru_cell(const Tensor&
input_gates, const Tensor& hidden_gates, const Tensor& hx, const Tensor&
input_bias, const Tensor& hidden_bias) {
1134 + rematerialize_function_t rt =
1135 +     [=](const Tensors& vec) -> Tensors {
1136 +         auto res = at::_thnn_fused_gru_cell(vec.at(0), vec.at(1), vec.at(2),
vec.at(3), vec.at(4));
1137 +         return {std::get<0>(res), std::get<1>(res)};
1138 +     };
1139 + auto res = CheckpointTensorImpl::make("_thnn_fused_gru_cell", rt,
1140 +                                     {input_gates, hidden_gates, hx,
                                     input_bias, hidden_bias});
1141 + return {res[0], res[1]};
1142 +}
1143 +
1144 +std::tuple<Tensor, Tensor, Tensor, Tensor, Tensor>
checkpoint__thnn_fused_gru_cell_backward(const Tensor& grad_hy, const Tensor&
workspace, bool has_bias) {
1145 + rematerialize_function_t rt =
1146 +     [=](const Tensors& vec) -> Tensors {
1147 +         auto res = at::_thnn_fused_gru_cell_backward(vec.at(0), vec.at(1),
has_bias);
1148 +         return {std::get<0>(res), std::get<1>(res),

```

```

1149 +         std::get<2>(res), std::get<3>(res), std::get<4>(res));
1150 +     };
1151 +     auto res = CheckpointTensorImpl::make("_thnn_fused_gru_cell_backward", rt,
1152 +         {grad_hy, workspace});
1153 +     return {res[0], res[1], res[2], res[3], res[4]};
1154 +}
1155 +
1156 +Tensor& checkpoint_bitwise_and_out(Tensor& self, const Tensor& other, const
Tensor& out) {
1157 +     mutate_function_t mt =
1158 +         [=](const Tensors& vec) {
1159 +             Tensor self = vec.at(0);
1160 +             at::bitwise_and_out(self, vec.at(1), vec.at(2));
1161 +         };
1162 +     CheckpointTensorImpl::mutate("bitwise_and_out", mt, {self, other, out},
{0});
1163 +     return self;
1164 +}
1165 +
1166 +Tensor& checkpoint_bitwise_and_out(Tensor& self, const Tensor& out, Scalar
other) {
1167 +     mutate_function_t mt =
1168 +         [=](const Tensors& vec) {
1169 +             Tensor self = vec.at(0);
1170 +             at::bitwise_and_out(self, vec.at(1), other);
1171 +         };
1172 +     CheckpointTensorImpl::mutate("bitwise_and_out", mt, {self, out}, {0});
1173 +     return self;
1174 +}
1175 +
1176 +Tensor& checkpoint_fill_(Tensor& self, const Tensor& value) {
1177 +     mutate_function_t mt =
1178 +         [=](const Tensors& vec) {
1179 +             Tensor self = vec.at(0);
1180 +             at::fill_(self, vec.at(1));
1181 +         };
1182 +     CheckpointTensorImpl::mutate("fill_", mt, {self, value}, {0});
1183 +     return self;
1184 +}
1185 +
1186 +Tensor& checkpoint_fill_(Tensor& self, Scalar value) {
1187 +     mutate_function_t mt =
1188 +         [=](const Tensors& vec) {
1189 +             Tensor self = vec.at(0);
1190 +             at::fill_(self, value);
1191 +         };
1192 +     CheckpointTensorImpl::mutate("fill_", mt, {self}, {0});

```

```

1193 + return self;
1194 +}
1195 +
1196 +Tensor& checkpoint_masked_select_out(Tensor& self, const Tensor& mask, const
Tensor& out) {
1197 + mutate_function_t mt =
1198 + [=](const Tensors& vec) {
1199 + Tensor self = vec.at(0);
1200 + at::masked_select_out(self, vec.at(1), vec.at(2));
1201 + };
1202 + CheckpointTensorImpl::mutate("masked_select_out", mt, {self, mask, out},
{0});
1203 + return self;
1204 +}
1205 +
1206 +Tensor checkpoint_masked_select(const Tensor& self, const Tensor& mask) {
1207 + rematerialize_function_t rt =
1208 + [=](const Tensors& vec) -> Tensors {
1209 + return {at::masked_select(vec.at(0), vec.at(1))};
1210 + };
1211 + return CheckpointTensorImpl::make("masked_select", rt, {self, mask})[0];
1212 +}
1213 +
1214 +Tensor checkpoint_index(const Tensor& self, ArrayRef<Tensor> indices) {
1215 + rematerialize_function_t rt =
1216 + [=](const Tensors& vec) -> Tensors {
1217 + auto self = vec.at(0);
1218 + auto indices = std::vector<Tensor>(vec.begin() + 1, vec.end());
1219 + return {at::index(self, indices)};
1220 + };
1221 +
1222 + std::vector<Tensor> s = {self};
1223 + for (const Tensor& t: indices) {
1224 + s.push_back(t);
1225 + }
1226 + return CheckpointTensorImpl::make("index", rt, s)[0];
1227 +}
1228 +
1229 +Tensor& checkpoint_index_put_(Tensor& self, ArrayRef<Tensor> indices, const
Tensor& values, const bool accumulate) {
1230 + mutate_function_t mt =
1231 + [=](const Tensors& vec) {
1232 + Tensor self = vec.at(0);
1233 + auto values = vec.at(1);
1234 + auto indices = std::vector<Tensor>(vec.begin() + 2, vec.end());
1235 + at::index_put_(self, indices, values, accumulate);
1236 + };

```

```

1237 + std::vector<Tensor> s = {self, values};
1238 + for (const Tensor& t: indices) {
1239 +     s.push_back(t);
1240 + }
1241 + CheckpointTensorImpl::mutate("index_put_", mt, s, {0});
1242 + return self;
1243 +}
1244 +
1245 +Tensor checkpoint_bmm(const Tensor& self, const Tensor& mat2) {
1246 +    rematerialize_function_t rt =
1247 +        [=](const Tensors& vec) -> Tensors {
1248 +            return {at::bmm(vec.at(0), vec.at(1))};
1249 +        };
1250 +    return CheckpointTensorImpl::make("bmm", rt, {self, mat2})[0];
1251 +}
1252 +
1253 +Tensor checkpoint__softmax(const Tensor& self, long dim, bool half_to_float)
1254 +{
1255 +    rematerialize_function_t rt =
1256 +        [=](const Tensors& vec) -> Tensors {
1257 +            return {at::_softmax(vec.at(0), dim, half_to_float)};
1258 +        };
1259 +    return CheckpointTensorImpl::make("_softmax", rt, {self})[0];
1260 +}
1261 +
1262 +Tensor checkpoint__softmax_backward_data(const Tensor& grad_output, const
1263 +Tensor& output, long dim, const Tensor& self) {
1264 +    rematerialize_function_t rt =
1265 +        [=](const Tensors& vec) -> Tensors {
1266 +            return {at::_softmax_backward_data(vec.at(0), vec.at(1), dim,
1267 +vec.at(2))};
1268 +        };
1269 +    return CheckpointTensorImpl::make("_softmax_backward_data", rt,
1270 +{grad_output, output, self})[0];
1271 +}
1272 +
1273 +std::tuple<Tensor, Tensor, Tensor>
1274 +checkpoint_layer_norm(const Tensor& input, const Tensor& weight, const
1275 +Tensor& bias, long M, long N, double eps) {
1276 +    rematerialize_function_t rt =
1277 +        [=](const Tensors& vec) -> Tensors {
1278 +            auto ret = at::native_layer_norm(vec.at(0), vec.at(1), vec.at(2), M, N,
1279 +eps);
1280 +            return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret)};
1281 +        };
1282 +    auto ret = CheckpointTensorImpl::make("native_layer_norm", rt, {input,
1283 +weight, bias});

```

```

1277 + return {ret[0], ret[1], ret[2]};
1278 +}
1279 +
1280 +std::tuple<Tensor, Tensor, Tensor>
1281 +checkpoint_layer_norm_backward(const Tensor& grad_out, const Tensor& input,
1282   const Tensor& mean, const Tensor& rstd, const Tensor& weight, long M, long N,
1283   std::array<bool, 3ul> output_mask) {
1284 +   rematerialize_function_t rt =
1285 +     [=](const Tensors& vec) -> Tensors {
1286 +       auto ret = at::native_layer_norm_backward(vec.at(0), vec.at(1),
1287         vec.at(2), vec.at(3), vec.at(4), M, N, output_mask);
1288 +       return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret)};
1289 +     };
1290 +   auto ret = CheckpointTensorImpl::make("native_layer_norm_backward", rt,
1291     {grad_out, input, mean, rstd, weight});
1292 +   return {ret[0], ret[1], ret[2]};
1293 +}
1294 +
1295 +std::tuple<Tensor, Tensor>
1296 +checkpoint_topk(const Tensor& self, long k, long dim, bool largest, bool
1297   sorted) {
1298 +   rematerialize_function_t rt =
1299 +     [=](const Tensors& vec) -> Tensors {
1300 +       auto ret = at::topk(vec.at(0), k, dim, largest, sorted);
1301 +       return {std::get<0>(ret), std::get<1>(ret)};
1302 +     };
1303 +   auto ret = CheckpointTensorImpl::make("topk", rt, {self});
1304 +   return {ret[0], ret[1]};
1305 +}
1306 +
1307 +std::tuple<Tensor&, Tensor&>
1308 +checkpoint_topk_values(Tensor& values, Tensor& indices, const Tensor& self,
1309   long k, long dim, bool largest, bool sorted) {
1310 +   mutate_function_t mt =
1311 +     [=](const Tensors& vec) {
1312 +       Tensor values_ = vec.at(0);
1313 +       Tensor indices_ = vec.at(1);
1314 +       at::topk_out(values_, indices_, vec.at(2), k, dim, largest, sorted);
1315 +     };
1316 +   CheckpointTensorImpl::mutate("topk_values", mt, {values, indices, self},
1317     {0, 1});
1318 +   return {values, indices};
1319 +}
1320 +
1321 +Tensor& checkpoint_masked_fill_(Tensor& self, const Tensor& mask, Scalar
1322   value) {
1323 +   mutate_function_t mt =

```



```

1316 +     [=](const Tensors& vec) {
1317 +     Tensor self_ = vec.at(0);
1318 +     self_.masked_fill_(vec.at(1), value);
1319 + };
1320 + CheckpointTensorImpl::mutate("masked_fill_Scalar", mt, {self, mask}, {0});
1321 + return {self};
1322 +}
1323 +
1324 +Tensor& checkpoint_masked_fill_(Tensor& self, const Tensor& mask, const
    Tensor& value) {
1325 +     mutate_function_t mt =
1326 +     [=](const Tensors& vec) {
1327 +     Tensor self_ = vec.at(0);
1328 +     self_.masked_fill_(vec.at(1), vec.at(2));
1329 + };
1330 + CheckpointTensorImpl::mutate("masked_fill_Tensor", mt, {self, mask,
    value}, {0});
1331 + return {self};
1332 +}
1333 +
1334 +Tensor checkpoint_clamp(const Tensor& self, c10::optional<Scalar> min,
    c10::optional<Scalar> max) {
1335 +     rematerialize_function_t rt =
1336 +     [=](const Tensors& vec) -> Tensors {
1337 +     return {at::clamp(vec.at(0), min, max)};
1338 + };
1339 + return CheckpointTensorImpl::make("clamp", rt, {self})[0];
1340 +}
1341 +
1342 +Tensor& checkpoint_clamp_(Tensor& self, c10::optional<Scalar> min,
    c10::optional<Scalar> max) {
1343 +     mutate_function_t mt =
1344 +     [=](const Tensors& vec) {
1345 +     Tensor self_ = vec.at(0);
1346 +     at::clamp_(self_, min, max);
1347 + };
1348 + CheckpointTensorImpl::mutate("clamp_", mt, {self}, {0});
1349 + return {self};
1350 +}
1351 +
1352 +Tensor& checkpoint_clamp_out(Tensor& out, const Tensor& self,
    c10::optional<Scalar> min, c10::optional<Scalar> max) {
1353 +     mutate_function_t mt =
1354 +     [=](const Tensors& vec) {
1355 +     Tensor out_ = vec.at(0);
1356 +     at::clamp_out(out_, vec.at(1), min, max);
1357 + };

```

```

1358 + CheckpointTensorImpl::mutate("clamp_out", mt, {out, self}, {0});
1359 + return {out};
1360 +}
1361 +
1362 +std::tuple<Tensor&, Tensor&, Tensor&>
checkpoint_thnn_conv2d_forward_out(Tensor& output, Tensor& finput, Tensor&
fgrad_input, const Tensor& self, const Tensor& weight, c10::ArrayRef<long>
kernel_size, const Tensor& bias, c10::ArrayRef<long> stride,
c10::ArrayRef<long> padding) {
1363 + auto kernel_size_ = kernel_size.vec();
1364 + auto stride_ = stride.vec();
1365 + auto padding_ = padding.vec();
1366 + mutate_function_t mt =
1367 + [=](const Tensors& vec) {
1368 + Tensor output_ = vec.at(0);
1369 + Tensor finput_ = vec.at(1);
1370 + Tensor fgrad_input_ = vec.at(2);
1371 + at::thnn_conv2d_forward_out(output_, finput_, fgrad_input_, vec.at(3),
vec.at(4), kernel_size_, vec.at(5), stride_, padding_);
1372 + };
1373 + CheckpointTensorImpl::mutate("thnn_conv2d_forward_out", mt, {output,
finput, fgrad_input, self, weight, bias}, {0, 1, 2});
1374 + return {output, finput, fgrad_input};
1375 +}
1376 +
1377 +std::tuple<Tensor, Tensor, Tensor> checkpoint_thnn_conv2d_forward(const
Tensor& self, const Tensor& weight, c10::ArrayRef<long> kernel_size, const
Tensor& bias, c10::ArrayRef<long> stride, c10::ArrayRef<long> padding) {
1378 + auto kernel_size_ = kernel_size.vec();
1379 + auto stride_ = stride.vec();
1380 + auto padding_ = padding.vec();
1381 + rematerialize_function_t rt =
1382 + [=](const Tensors& vec) -> Tensors {
1383 + auto ret = at::thnn_conv2d_forward(vec.at(0), vec.at(1), kernel_size_,
vec.at(2), stride_, padding_);
1384 + return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret)};
1385 + };
1386 + auto ret = CheckpointTensorImpl::make("thnn_conv2d_forward", rt, {self,
weight, bias});
1387 + return {ret[0], ret[1], ret[2]};
1388 +}
1389 +
1390 +std::tuple<Tensor&, Tensor&, Tensor&>
checkpoint_thnn_conv2d_backward_out(Tensor& grad_input, Tensor& grad_weight,
Tensor& grad_bias, const Tensor& grad_output, const Tensor& self, const
Tensor& weight, c10::ArrayRef<long> kernel_size, c10::ArrayRef<long> stride,

```

```

        c10::ArrayRef<long> padding, const Tensor& finput, const Tensor& fgrad_input)
    {
1391 +   auto kernel_size_ = kernel_size.vec();
1392 +   auto stride_ = stride.vec();
1393 +   auto padding_ = padding.vec();
1394 +   mutate_function_t mt =
1395 +       [=](const Tensors& vec) {
1396 +           Tensor grad_input_ = vec.at(0);
1397 +           Tensor grad_weight_ = vec.at(1);
1398 +           Tensor grad_bias_ = vec.at(2);
1399 +           at::thnn_conv2d_backward_out(grad_input_, grad_weight_, grad_bias_,
vec.at(3), vec.at(4), vec.at(5), kernel_size_, stride_, padding_, vec.at(6),
vec.at(7));
1400 +       };
1401 +   CheckpointTensorImpl::mutate("thnn_conv2d_backward_out", mt, {grad_input,
grad_weight, grad_bias, grad_output, self, weight, finput, fgrad_input}, {0,
1, 2});
1402 +   return {grad_input, grad_weight, grad_bias};
1403 +}
1404 +
1405 +std::tuple<Tensor, Tensor, Tensor> checkpoint_thnn_conv2d_backward(const
Tensor& grad_output, const Tensor& self, const Tensor& weight,
c10::ArrayRef<long> kernel_size, c10::ArrayRef<long> stride,
c10::ArrayRef<long> padding, const Tensor& finput, const Tensor& fgrad_input,
std::array<bool, 3ul> output_mask) {
1406 +   auto kernel_size_ = kernel_size.vec();
1407 +   auto stride_ = stride.vec();
1408 +   auto padding_ = padding.vec();
1409 +   rematerialize_function_t rt =
1410 +       [=](const Tensors& vec) -> Tensors {
1411 +           auto ret = at::thnn_conv2d_backward(vec.at(0), vec.at(1), vec.at(2),
kernel_size_, stride_, padding_, vec.at(3), vec.at(4), output_mask);
1412 +           return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret)};
1413 +       };
1414 +   auto ret = CheckpointTensorImpl::make("thnn_conv2d_backward", rt,
{grad_output, self, weight, finput, fgrad_input});
1415 +   return {ret[0], ret[1], ret[2]};
1416 +}
1417 +
1418 +std::tuple<Tensor, Tensor, Tensor> checkpoint_native_batch_norm(const
Tensor& input, const Tensor& weight, const Tensor& bias, const Tensor&
running_mean, const Tensor& running_var, bool training, double momentum,
double eps) {
1419 +   rematerialize_function_t rt =
1420 +       [=](const Tensors& vec) -> Tensors {
1421 +           auto ret = at::native_batch_norm(vec.at(0), vec.at(1), vec.at(2),
vec.at(3), vec.at(4), training, momentum, eps);

```

```

1422 +     return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret)};
1423 + };
1424 + auto ret = CheckpointTensorImpl::make("native_batch_norm", rt, {input,
    weight, bias, running_mean, running_var});
1425 + return {ret[0], ret[1], ret[2]};
1426 +}
1427 +
1428 +std::tuple<Tensor&, Tensor&, Tensor&>
    checkpoint_native_batch_norm_out(Tensor& out, Tensor& save_mean, Tensor&
    save_invstd, const Tensor& input, const Tensor& weight, const Tensor& bias,
    const Tensor& running_mean, const Tensor& running_var, bool training, double
    momentum, double eps) {
1429 +     mutate_function_t mt =
1430 +         [=](const Tensors& vec) {
1431 +             Tensor out_ = vec.at(0);
1432 +             Tensor save_mean_ = vec.at(1);
1433 +             Tensor save_invstd_ = vec.at(2);
1434 +             at::native_batch_norm_out(out_, save_mean_, save_invstd_, vec.at(3),
                vec.at(4), vec.at(5), vec.at(6), vec.at(7), training, momentum, eps);
1435 +         };
1436 +     CheckpointTensorImpl::mutate("native_batch_norm_out", mt, {out, save_mean,
        save_invstd, input, weight, bias, running_mean, running_var}, {0, 1, 2});
1437 +     return {out, save_mean, save_invstd};
1438 +}
1439 +
1440 +std::tuple<Tensor, Tensor, Tensor>
    checkpoint_native_batch_norm_backward(const Tensor& grad_out, const Tensor&
    input, const Tensor& weight, const Tensor& running_mean, const Tensor&
    running_var, const Tensor& save_mean, const Tensor& save_invstd, bool train,
    double eps, std::array<bool, 3ul> output_mask) {
1441 +     rematerialize_function_t rt =
1442 +         [=](const Tensors& vec) -> Tensors {
1443 +             auto ret = at::native_batch_norm_backward(vec.at(0), vec.at(1),
                vec.at(2), vec.at(3), vec.at(4), vec.at(5), vec.at(6), train, eps,
                output_mask);
1444 +             return {std::get<0>(ret), std::get<1>(ret), std::get<2>(ret)};
1445 +         };
1446 +     auto ret = CheckpointTensorImpl::make("native_batch_norm_backward", rt,
        {grad_out, input, weight, running_mean, running_var, save_mean, save_invstd});
1447 +     return {ret[0], ret[1], ret[2]};
1448 +}
1449 +
1450 +std::tuple<Tensor, Tensor> checkpoint__cudnn_ctc_loss(const Tensor&
    log_probs, const Tensor& targets, ArrayRef<long> input_lengths,
    ArrayRef<long> target_lengths, long blank, bool deterministic, bool
    zero_infinity) {
1451 +     auto input_lengths_ = input_lengths.vec();

```

```

1452 + auto target_lengths_ = target_lengths.vec();
1453 + rematerialize_function_t rt =
1454 +     [=](const Tensors& vec) -> Tensors {
1455 +         auto ret = at::_cudnn_ctc_loss(vec.at(0), vec.at(1), input_lengths_,
1456 +             target_lengths_, blank, deterministic, zero_infinity);
1457 +         return {std::get<0>(ret), std::get<1>(ret)};
1458 +     };
1459 + auto ret = CheckpointTensorImpl::make("_cudnn_ctc_loss", rt, {log_probs,
1460 +     targets});
1461 + return {ret[0], ret[1]};
1462 +}
1463 +
1464 +std::tuple<Tensor, Tensor> checkpoint__ctc_loss(const Tensor& log_probs,
1465 +     const Tensor& targets, ArrayRef<long> input_lengths, ArrayRef<long>
1466 +     target_lengths, long blank, bool zero_infinity) {
1467 + auto input_lengths_ = input_lengths.vec();
1468 + auto target_lengths_ = target_lengths.vec();
1469 + rematerialize_function_t rt =
1470 +     [=](const Tensors& vec) -> Tensors {
1471 +         auto ret = at::_ctc_loss(vec.at(0), vec.at(1), input_lengths_,
1472 +             target_lengths_, blank, zero_infinity);
1473 +         return {std::get<0>(ret), std::get<1>(ret)};
1474 +     };
1475 + auto ret = CheckpointTensorImpl::make("_ctc_loss", rt, {log_probs,
1476 +     targets});
1477 + return {ret[0], ret[1]};
1478 +}
1479 +
1480 +Tensor checkpoint__ctc_loss_backward(const Tensor& grad, const Tensor&
1481 +     log_probs, const Tensor& targets, ArrayRef<long> input_lengths,
1482 +     ArrayRef<long> target_lengths, const Tensor& neg_log_likelihood, const
1483 +     Tensor& log_alpha, long blank, bool zero_infinity) {
1484 + auto input_lengths_ = input_lengths.vec();
1485 + auto target_lengths_ = target_lengths.vec();
1486 + rematerialize_function_t rt =
1487 +     [=](const Tensors& vec) -> Tensors {
1488 +         return {at::_ctc_loss_backward(vec.at(0), vec.at(1), vec.at(2),
1489 +             input_lengths_, target_lengths_, vec.at(3), vec.at(4), blank, zero_infinity)};
1490 +     };
1491 + return CheckpointTensorImpl::make("_ctc_loss_backward", rt, {grad,
1492 +     log_probs, targets, neg_log_likelihood, log_alpha})[0];
1493 +}
1494 +
1495 +Tensor& checkpoint_hardtanh_backward_out(Tensor& grad_input, const Tensor&
1496 +     grad_output, const Tensor& self, Scalar min_val, Scalar max_val) {
1497 + mutate_function_t mt =
1498 +     [=](const Tensors& vec) {

```

```

1487 +   Tensor grad_input_ = vec.at(0);
1488 +   at::hardtanh_backward_out(grad_input_, vec.at(1), vec.at(2), min_val,
1489 +   };
1490 +   CheckpointTensorImpl::mutate("hardtanh_backward_out", mt, {grad_input,
1491 +   grad_output, self}, {0});
1492 +   return {grad_input};
1493 +}
1494 +
1495 +Tensor checkpoint_hardtanh_backward(const Tensor& grad_output, const Tensor&
1496 +self, Scalar min_val, Scalar max_val) {
1497 +   rematerialize_function_t rt =
1498 +   [=](const Tensors& vec) -> Tensors {
1499 +       return {at::hardtanh_backward(vec.at(0), vec.at(1), min_val, max_val)};
1500 +   };
1501 +   return CheckpointTensorImpl::make("hardtanh_backward", rt, {grad_output,
1502 +   self})[0];
1503 +}
1504 +
1505 +Tensor checkpoint_nonzero(const Tensor& self) {
1506 +   rematerialize_function_t rt =
1507 +   [=](const Tensors& vec) -> Tensors {
1508 +       return {at::nonzero(vec.at(0))};
1509 +   };
1510 +   return CheckpointTensorImpl::make("nonzero", rt, {self})[0];
1511 +}
1512 +
1513 +Tensor& checkpoint_nonzero_out(Tensor& out, const Tensor& self) {
1514 +   mutate_function_t mt =
1515 +   [=](const Tensors& vec) {
1516 +       Tensor out_ = vec.at(0);
1517 +       at::nonzero_out(out_, vec.at(1));
1518 +   };
1519 +   CheckpointTensorImpl::mutate("nonzero_out", mt, {out, self}, {0});
1520 +   return {out};
1521 +}
1522 +
1523 +Tensor checkpoint_lt(const Tensor& self, Scalar other) {
1524 +   rematerialize_function_t rt =
1525 +   [=](const Tensors& vec) -> Tensors {
1526 +       return {at::lt(vec.at(0), other)};
1527 +   };
1528 +   return CheckpointTensorImpl::make("lt_Scalar", rt, {self})[0];
1529 +}
1530 +
1531 +Tensor& checkpoint_lt_out(Tensor& out, const Tensor& self, Scalar other) {
1532 +   mutate_function_t mt =

```

```

1530 +     [=](const Tensors& vec) {
1531 +         Tensor out_ = vec.at(0);
1532 +         at::lt_out(out_, vec.at(1), other);
1533 +     };
1534 +     CheckpointTensorImpl::mutate("lt_Scalar_out", mt, {out, self}, {0});
1535 +     return {out};
1536 +}
1537 +
1538 +Tensor checkpoint_lt(const Tensor& self, const Tensor& other) {
1539 +    rematerialize_function_t rt =
1540 +        [=](const Tensors& vec) -> Tensors {
1541 +            return {at::lt(vec.at(0), vec.at(1))};
1542 +        };
1543 +    return CheckpointTensorImpl::make("lt_Tensor", rt, {self, other})[0];
1544 +}
1545 +
1546 +Tensor& checkpoint_lt_out(Tensor& out, const Tensor& self, const Tensor&
    other) {
1547 +    mutate_function_t mt =
1548 +        [=](const Tensors& vec) {
1549 +            Tensor out_ = vec.at(0);
1550 +            at::lt_out(out_, vec.at(1), vec.at(2));
1551 +        };
1552 +    CheckpointTensorImpl::mutate("lt_Tensor_out", mt, {out, self, other}, {0});
1553 +    return {out};
1554 +}
1555 +
1556 +Tensor checkpoint_any(const Tensor& self) {
1557 +    rematerialize_function_t rt =
1558 +        [=](const Tensors& vec) -> Tensors {
1559 +            return {at::any(vec.at(0))};
1560 +        };
1561 +    return CheckpointTensorImpl::make("any", rt, {self})[0];
1562 +}
1563 +
1564 +bool checkpoint__use_cudnn_ctc_loss(const Tensor& log_probs, const Tensor&
    targets, ArrayRef<long> input_lengths, ArrayRef<long> target_lengths, long
    blank) {
1565 +    return at::_use_cudnn_ctc_loss(decheckpoint(log_probs),
        decheckpoint(targets), input_lengths, target_lengths, blank);
1566 +}
1567 +
1568 +bool checkpoint_equal(const Tensor& self, const Tensor& other) {
1569 +    // there can't possibly be a reason to rematerialize
1570 +    // a single bool so we'll just compute it now
1571 +    return at::equal(decheckpoint(self), decheckpoint(other));
1572 +}

```

```

1573 +
1574 +Scalar checkpoint__local_scalar_dense(at::Tensor const& a) {
1575 + return at::_local_scalar_dense(decheckpoint(a));
1576 +}
1577 +
1578 +Tensor checkpoint_split_with_sizes_backward(c10::ArrayRef<at::Tensor> a,
      c10::ArrayRef<long> b, long c, c10::ArrayRef<long> d, c10::TensorOptions
      const& e) {
1579 + std::vector<Tensor> a_ = a.vec();
1580 + std::vector<long> d_ = d.vec();
1581 + rematerialize_function_t rt =
1582 + [=](const Tensors& vec) -> Tensors {
1583 +     return {at::split_with_sizes_backward(vec, b, c, d_, e)};
1584 + };
1585 + return CheckpointTensorImpl::make("split_with_sizes_backward", rt, a_)[0];
1586 +}
1587 +
1588 +std::vector<Tensor> checkpoint_split_with_sizes(at::Tensor const& a,
      c10::ArrayRef<long> b, long c) {
1589 + std::vector<long> b_ = b.vec();
1590 + rematerialize_function_t rt =
1591 + [=](const Tensors& vec) -> Tensors {
1592 +     return at::split_with_sizes(vec.at(0), b_, c);
1593 + };
1594 + return CheckpointTensorImpl::make("split_with_sizes", rt, {a});
1595 +}
1596 +
1597 +Tensor checkpoint_split_backward(c10::ArrayRef<at::Tensor> a, long b, long
      c, c10::ArrayRef<long> d, const c10::TensorOptions& e) {
1598 + std::vector<Tensor> a_ = a.vec();
1599 + std::vector<long> d_ = d.vec();
1600 + rematerialize_function_t rt =
1601 + [=](const Tensors& vec) -> Tensors {
1602 +     return {at::split_backward(vec, b, c, d_, e)};
1603 + };
1604 + return CheckpointTensorImpl::make("split_backward", rt, a_)[0];
1605 +}
1606 +
1607 +std::vector<Tensor> checkpoint_split(const at::Tensor& a, long b, long c) {
1608 + rematerialize_function_t rt =
1609 + [=](const Tensors& vec) -> Tensors {
1610 +     return at::split(vec.at(0), b, c);
1611 + };
1612 + return CheckpointTensorImpl::make("split", rt, {a});
1613 +}
1614 +

```



```

1615 +Tensor checkpoint_expand(at::Tensor const& a, c10::ArrayRef<long> b, bool c)
1616 + {
1617 +   std::vector<long> b_ = b.vec();
1618 +   rematerialize_function_t rt =
1619 +     [=](const Tensors& vec) -> Tensors {
1620 +       return {vec.at(0).expand(b_, c)};
1621 +     };
1622 +   return CheckpointTensorImpl::make("expand", rt, {a})[0];
1623 +}
1624 +
1625 +Tensor checkpoint_diag(at::Tensor const& self, long diagonal) {
1626 +   rematerialize_function_t rt =
1627 +     [=](const Tensors& vec) -> Tensors {
1628 +       return {at::diag(vec.at(0), diagonal)};
1629 +     };
1630 +   return CheckpointTensorImpl::make("diag", rt, {self})[0];
1631 +}
1632 +
1633 +Tensor& checkpoint_diag_out(at::Tensor& out, const Tensor& self, long
1634 +   diagonal) {
1635 +   mutate_function_t mt =
1636 +     [=](const Tensors& vec) {
1637 +       Tensor out_ = vec.at(0);
1638 +       at::diag_out(out_, vec.at(1), diagonal);
1639 +     };
1640 +   CheckpointTensorImpl::mutate("diag_out", mt, {out, self}, {0});
1641 +   return {out};
1642 +}
1643 +
1644 +Tensor checkpoint_mv(at::Tensor const& self, at::Tensor const& vec) {
1645 +   rematerialize_function_t rt =
1646 +     [=](const Tensors& vec) -> Tensors {
1647 +       return {at::mv(vec.at(0), vec.at(1))};
1648 +     };
1649 +   return CheckpointTensorImpl::make("mv", rt, {self, vec})[0];
1650 +}
1651 +
1652 +Tensor& checkpoint_mv_out(at::Tensor& out, const Tensor& self, const Tensor&
1653 +   vec) {
1654 +   mutate_function_t mt =
1655 +     [=](const Tensors& vec) {
1656 +       Tensor out_ = vec.at(0);
1657 +       at::mv_out(out_, vec.at(1), vec.at(2));
1658 +     };
1659 +   CheckpointTensorImpl::mutate("mv_out", mt, {out, self, vec}, {0});
1660 +   return {out};
1661 +}

```

```
1659 +
1660 +}}
```

DELTA

```
1 diff -ru dtr_pytorch/aten/src/ATen/native/Checkpoint.cpp
   delta/aten/src/ATen/native/Checkpoint.cpp
2 --- dtr_pytorch/aten/src/ATen/native/Checkpoint.cpp      2024-04-28
   16:09:10.546175266 +0800
3 +++ delta/aten/src/ATen/native/Checkpoint.cpp            2024-04-09
   14:45:52.865654719 +0800
4 @@ -137,6 +137,30 @@
5     return CheckpointTensorImpl::make("to", rt, {a})[0];
6 }
7
8 +Tensor checkpoint_to(at::Tensor const& a, at::Tensor const& b, bool c, bool
   d, c10::optional<c10::MemoryFormat> e) {
9 +   rematerialize_function_t rt =
10 +   [=](const Tensors& vec) -> Tensors {
11 +       return {vec.at(0).to(b, c, d, e)};
12 +   };
13 +   return CheckpointTensorImpl::make("to", rt, {a})[0];
14 +}
15 +
16 +Tensor checkpoint_to(at::Tensor const& a, c10::ScalarType b, bool c, bool d,
   c10::optional<c10::MemoryFormat> e) {
17 +   rematerialize_function_t rt =
18 +   [=](const Tensors& vec) -> Tensors {
19 +       return {vec.at(0).to(b, c, d, e)};
20 +   };
21 +   return CheckpointTensorImpl::make("to", rt, {a})[0];
22 +}
23 +
24 +Tensor checkpoint_to(at::Tensor const& a, c10::Device b, c10::ScalarType c,
   bool d, bool e, c10::optional<c10::MemoryFormat> f) {
25 +   rematerialize_function_t rt =
26 +   [=](const Tensors& vec) -> Tensors {
27 +       return {vec.at(0).to(b, c, d, e, f)};
28 +   };
29 +   return CheckpointTensorImpl::make("to", rt, {a})[0];
30 +}
31 +
32   Tensor checkpoint_div(const Tensor& a, const Tensor& b) {
33       rematerialize_function_t rt =
34       [=](const Tensors& vec) -> Tensors {
```

```

35 @@ -162,6 +186,24 @@
36     return CheckpointTensorImpl::make("clone", rt, {a})[0];
37 }
38
39 +Tensor checkpoint_pow(at::Tensor const& a, c10::Scalar b) {
40 +   rematerialize_function_t rt =
41 +   [=](const Tensors& vec) -> Tensors {
42 +       return {at::pow(vec.at(0), b)};
43 +   };
44 +   return CheckpointTensorImpl::make("pow", rt, {a})[0];
45 +}
46 +
47 +Tensor& checkpoint_pow_out(at::Tensor& a, const Tensor& b, c10::Scalar c) {
48 +   mutate_function_t mt =
49 +   [=](const Tensors& vec) {
50 +       Tensor a_ = vec.at(0);
51 +       at::pow_out(a_, vec.at(1), c);
52 +   };
53 +   CheckpointTensorImpl::mutate("pow_out", mt, {a, b}, {0});
54 +   return a;
55 +}
56 +
57   Tensor checkpoint_where(at::Tensor const& a, at::Tensor const& b, at::Tensor
const& c) {
58       rematerialize_function_t rt =
59       [=](const Tensors& vec) -> Tensors {
60 @@ -703,6 +745,40 @@
61       return CheckpointTensorImpl::make("sum_dim_IntList", rt, {a})[0];
62   }
63
64   +Tensor& checkpoint_transpose_(at::Tensor& a, long b, long c) {
65   +   mutate_function_t mt =
66   +   [=](const Tensors& vec) {
67   +       Tensor a_ = vec.at(0);
68   +       a_.transpose_(b, c);
69   +   };
70   +   CheckpointTensorImpl::mutate("transpose_", mt, {a}, {0});
71   +   return a;
72   +}
73   +
74   +Tensor checkpoint_transpose(at::Tensor const& a, long b, long c) {
75   +   rematerialize_function_t rt =
76   +   [=](const Tensors& vec) -> Tensors {
77   +       return {at::transpose(vec.at(0), b, c)};
78   +   };
79   +   return CheckpointTensorImpl::make("transpose", rt, {a})[0];
80   +}

```

```

81 +
82 +Tensor checkpoint_gelu(at::Tensor const& a) {
83 +    rematerialize_function_t rt =
84 +        [=](const Tensors& vec) -> Tensors {
85 +            return {at::gelu(vec.at(0))};
86 +        };
87 +    return CheckpointTensorImpl::make("gelu", rt, {a})[0];
88 +}
89 +
90 +Tensor checkpoint_matmul(at::Tensor const& a, at::Tensor const& b) {
91 +    rematerialize_function_t rt =
92 +        [=](const Tensors& vec) -> Tensors {
93 +            return {at::matmul(vec.at(0), vec.at(1))};
94 +        };
95 +    return CheckpointTensorImpl::make("matmul", rt, {a, b})[0];
96 +}
97 +
98     Tensor checkpoint_threshold(const Tensor& a, c10::Scalar b, c10::Scalar c) {
99         rematerialize_function_t rt =
100             [=](const Tensors& vec) -> Tensors {
101 @@ -1244,6 +1320,34 @@
102         return CheckpointTensorImpl::make("bmm", rt, {self, mat2})[0];
103     }
104
105 +Tensor& checkpoint_bmm_out(Tensor& a, const Tensor& b, const Tensor& c) {
106 +    mutate_function_t mt =
107 +        [=](const Tensors& vec) {
108 +            Tensor a_ = vec.at(0);
109 +            at::bmm_out(a_, vec.at(1), vec.at(2));
110 +        };
111 +    CheckpointTensorImpl::mutate("bmm_out", mt, {a, b, c}, {0});
112 +    return a;
113 +}
114 +
115 +// Tensor checkpoint_bmm_(const Tensor& self, const Tensor& mat2, bool
deterministic) {
116 +//     rematerialize_function_t rt =
117 +//         [=](const Tensors& vec) -> Tensors {
118 +//             return {at::_bmm(vec.at(0), vec.at(1), deterministic)};
119 +//         };
120 +//     return CheckpointTensorImpl::make("bmm", rt, {self, mat2})[0];
121 +// }
122 +
123 +// Tensor& checkpoint_bmm_out_(Tensor& a, const Tensor& b, bool c) {
124 +//     mutate_function_t mt =
125 +//         [=](const Tensors& vec) {
126 +//             Tensor a_ = vec.at(0);

```

```

127 +//      at::_bmm_out(a_, vec.at(1), vec.at(2), c);
128 +//  };
129 +//  CheckpointTensorImpl::mutate("_bmm_out", mt, {a, b}, {0});
130 +//  return a;
131 +// }
132 +
133 Tensor checkpoint__softmax(const Tensor& self, long dim, bool half_to_float)
{
134     rematerialize_function_t rt =
135         [=](const Tensors& vec) -> Tensors {
136 @@ -1651,4 +1755,78 @@
137         return {out};
138     }
139
140 +Tensor checkpoint_erf(const Tensor& self) {
141 +    rematerialize_function_t rt =
142 +        [=](const Tensors& vec) -> Tensors {
143 +            return {at::erf(vec.at(0))};
144 +        };
145 +    return CheckpointTensorImpl::make("erf", rt, {self})[0];
146 +}
147 +
148 +Tensor& checkpoint_erf_(Tensor& a) {
149 +    mutate_function_t mt =
150 +        [=](const Tensors& vec){
151 +            Tensor a_ = vec.at(0);
152 +            at::erf_(a_);
153 +        };
154 +    CheckpointTensorImpl::mutate("erf_", mt, {a}, {0});
155 +    return a;
156 +}
157 +
158 +Tensor checkpoint_erfc(const Tensor& self) {
159 +    rematerialize_function_t rt =
160 +        [=](const Tensors& vec) -> Tensors {
161 +            return {at::erfc(vec.at(0))};
162 +        };
163 +    return CheckpointTensorImpl::make("erfc", rt, {self})[0];
164 +}
165 +
166 +Tensor& checkpoint_erfc_(Tensor& a) {
167 +    mutate_function_t mt =
168 +        [=](const Tensors& vec){
169 +            Tensor a_ = vec.at(0);
170 +            at::erfc_(a_);
171 +        };
172 +    CheckpointTensorImpl::mutate("erfc_", mt, {a}, {0});

```

```

173 + return a;
174 +}
175 +
176 +Tensor& checkpoint_erf_out(Tensor& a, const Tensor& b) {
177 + mutate_function_t mt =
178 + [=](const Tensors& vec){
179 +     Tensor a_ = vec.at(0);
180 +     at::erf_out(a_, vec.at(1));
181 + };
182 + CheckpointTensorImpl::mutate("erf_out", mt, {a, b}, {0});
183 + return a;
184 +}
185 +
186 +Tensor& checkpoint_erfc_out(Tensor& a, const Tensor& b) {
187 + mutate_function_t mt =
188 + [=](const Tensors& vec){
189 +     Tensor a_ = vec.at(0);
190 +     at::erfc_out(a_, vec.at(1));
191 + };
192 + CheckpointTensorImpl::mutate("erfc_out", mt, {a, b}, {0});
193 + return a;
194 +}
195 +
196 +Tensor checkpoint_exp(const Tensor& self) {
197 + rematerialize_function_t rt =
198 + [=](const Tensors& vec) -> Tensors {
199 +     return {at::exp(vec.at(0))};
200 + };
201 + return CheckpointTensorImpl::make("exp", rt, {self})[0];
202 +}
203 +
204 +Tensor& checkpoint_exp_(Tensor& a) {
205 + mutate_function_t mt =
206 + [=](const Tensors& vec){
207 +     Tensor a_ = vec.at(0);
208 +     at::exp_(a_);
209 + };
210 + CheckpointTensorImpl::mutate("exp_", mt, {a}, {0});
211 + return a;
212 +}
213 +
214 +}

```

!!!新增 `aten/src/ATen/CheckpointTensorImpl.h`

DTR

```
1 diff --git a/aten/src/ATen/CheckpointTensorImpl.h
  b/aten/src/ATen/CheckpointTensorImpl.h
2 new file mode 100644
3 index 000000000000..b1edbf2c80
4 --- /dev/null
5 +++ b/aten/src/ATen/CheckpointTensorImpl.h
6 @@ -0,0 +1,469 @@
7 +#pragma once
8 +
9 +#include <atomic>
10 +#include <memory>
11 +#include <numeric>
12 +#include <random>
13 +
14 +#include <c10/core/Backend.h>
15 +#include <c10/core/MemoryFormat.h>
16 +#include <c10/core/Storage.h>
17 +#include <c10/core/TensorOptions.h>
18 +#include <c10/core/DispatchKeySet.h>
19 +#include <c10/core/impl/LocalDispatchKeySet.h>
20 +#include <c10/core/CopyBytes.h>
21 +
22 +#include <c10/util/Exception.h>
23 +#include <c10/util/Optional.h>
24 +#include <c10/util/Flags.h>
25 +#include <c10/util/Logging.h>
26 +#include <c10/util/python_stub.h>
27 +#include <c10/core/TensorImpl.h>
28 +#include <ATen/Tensor.h>
29 +#include <ATen/ATen.h>
30 +
31 +#define likely(x)      __builtin_expect(!!(x), 1)
32 +#define unlikely(x)    __builtin_expect(!!(x), 0)
33 +#define TORCH_CHECK(a, ...) // profile mode
34 +
35 +// System Description:
36 +// Every Tensor is managed by a CheckpointTensor,
37 +// that describe how it is computed, (the function and the inputs)
38 +// And might optionally hold the tensor value.
39 +// The tensor value might be dropped, and when requested later, recomputed
  and cached again.
40 +
41 +// Corner Cases:
```

```
42  ///  
43  In this case the underlying data must not require_grad,  
44  as we want backpropagation on the outer, uncheckpoined level.  
45  To be more specific, suppose a tensor is recomputed multiple times.  
46  We want to only compute the gradient exactly once.  
47  To do this, the wrapper must be require_grad, and the wrapped value  
    must not.  
48  A CheckpointedTensor might be constant.  
49  In this case it is unevictable.  
50  An operator might return multiple output.  
51  In this case the computation info (rematerializer) is shared between  
    all of them,  
52  And when the function get computed again all value get cached.  
53  An operator might not return value, but only mutate input value.  
54  To combat this, we COW the operator, and wrap CheckpointTensor with a  
    Ref.  
55  By doing this the inner CheckpointTensor is kept purely functional.  
56  An operator might try to mutate uncheckpointed tensor.  
57  We do not support this and will error.  
58  An operator might create aliases.  
59  We track alias in AliasPool.  
60  Each AliasPool hold a set of tensor that is alias to eachother.  
61  An operator might try to create Alias to an unevictable tensor.  
62  In such a case the output tensor is unevictable.  
63  An operator might try to mutate Tensor with Alias.  
64  We do not support this case an will error if a Tensor has any alive  
    Alias.  
65  However it could be done without a major redesign of the system -  
66  Each AliasPool will hold weak pointers to the External Reference.  
67  When alias mutation occur,  
68  we make a rematerialize_function that take in the base tensor (other  
    tensor alias from)  
69  and output all the new value of the aliases, then update the Ref.  
70  Of course, the cleaner way is to not support this.  
71  Shame on those who use this feature.  
72  +  
73  ///  
    Memory Safety:  
74  The objects here will have lots of backedges.  
75  In order to collect memory when computation is completed,  
76  We require that all strong pointer is of the form of value -> input.  
77  This ensure that everything will be released if there is no external ref  
    whatsoever.  
78  +  
79  ///  
    Optimization:  
80  We treat tensor that has no external reference differently -  
81  They will never be externally used again so we assume their next use time  
    is infinite
```



```

82  ///  
83  +  
84  ///  
85  ///  
86  +  
87  +namespace at {  
88  +  
89  ///  
90  +template<typename T>  
91  +struct EquivalentClassNode : intrusive_ptr_target {  
92  +   explicit EquivalentClassNode(const T& t) : t_unsafe(t) { }  
93  +   mutable intrusive_ptr<EquivalentClassNode> parent;  
94  +   bool is_root() {  
95  +       return !parent;  
96  +   }  
97  +   void release_resources() override {  
98  +       parent.reset();  
99  +   }  
100  +   T t_unsafe;  
101  +};  
102  +  
103  +template<typename T>  
104  +T& get_t(const intrusive_ptr<EquivalentClassNode<T>>& n) {  
105  +   return find_root(n)->t_unsafe;  
106  +}  
107  +  
108  +template<typename T>  
109  +static void update_t(const intrusive_ptr<EquivalentClassNode<T>>& n, const  
110  +   T& t) {  
111  +   find_root(n)->t_unsafe = t;  
112  +}  
113  +template<typename T>  
114  +intrusive_ptr<EquivalentClassNode<T>> find_root(const  
115  +   intrusive_ptr<EquivalentClassNode<T>>& n) {  
116  +   if (n->is_root()) {  
117  +       return n;  
118  +   } else {  
119  +       n->parent = find_root(n->parent);  
120  +       return n->parent;  
121  +   }  
122  +}  
123  +template<typename T>

```

```

124 +intrusive_ptr<EquivalentClassNode<T>> merge(const std::function<T(const T&,
125 +                                     const T&>& merge_t,
126 +                                     const
127 +                                     intrusive_ptr<EquivalentClassNode<T>>& lhs,
128 +                                     const
129 +                                     intrusive_ptr<EquivalentClassNode<T>>& rhs) {
130 +     auto l = find_root(lhs);
131 +     auto r = find_root(rhs);
132 +     if (l == r) {
133 +         return l;
134 +     }
135 +     l->parent = r;
136 +     r->t_unsafe = merge_t(l->t_unsafe, r->t_unsafe);
137 +     return r;
138 + }
139 +}
140 +
141 +size_t memory(const Tensor& t);
142 +
143 +template<typename T>
144 +struct RefCell final : intrusive_ptr_target {
145 +     mutable T value;
146 +     void release_resources() final {
147 +         static_release_resources(value);
148 +     }
149 +     RefCell(const T& t) : value(t) { }
150 +};
151 +
152 +template<typename T>
153 +using Ref = intrusive_ptr<RefCell<T>>;
154 +
155 +template<typename T>
156 +void static_release_resources(intrusive_ptr<T>& ptr) {
157 +     ptr.reset();
158 + }
159 +
160 +class CheckpointTensorCell;
161 +using strong = intrusive_ptr<CheckpointTensorCell>;
162 +using strongs = std::vector<strong>;
163 +using weak = weak_intrusive_ptr<CheckpointTensorCell>;
164 +using weaks = std::vector<weak>;
165 +using Tensors = std::vector<Tensor>;
166 +using rematerialize_function_t = std::function<Tensors(const Tensors&>>;
167 +using mutate_function_t = std::function<void(const Tensors&>>;
168 +
169 +using time_t = std::chrono::time_point<std::chrono::system_clock>;
170 +using duration_t = std::chrono::system_clock::duration;
171 +struct CheckpointInfo {

```

```

168 + duration_t compute_cost;
169 + // Floating Point instability?
170 + double cost(size_t memory, size_t staleness) const {
171 +     TORCH_CHECK(memory > 0);
172 +     TORCH_CHECK(staleness > 0);
173 +     return compute_cost.count() / static_cast<double>(memory * staleness);
174 + }
175 + CheckpointInfo(duration_t compute_cost) :
176 +     compute_cost(compute_cost) {
177 + }
178 +};
179 +
180 +// ecn represent a evicted tensor group.
181 +// it is a set of tensor that are evicted, and if two evicted tensor are
182 +input -> output to each other,
183 +// they must be in an ecn.
184 +// note: we try to support removal from ecn by subtracting compute_cost and
185 +memory.
186 +// this will create suprious connection but that should be fine empircally.
187 +// below is an example of a suprious connection:
188 +// a -> b, a -> c
189 +// a, b, c got evicted so belong to a single ecn.
190 +// a got rematerialized.
191 +// b, c still belong to a single ecn although there is no connection.
192 +using ecn_ptr = intrusive_ptr<EquivalentClassNode<CheckpointInfo>>;
193 +
194 +struct Unsafe { };
195 +
196 +// The rematerializer could be called to reinvoke an operator.
197 +// Tensor point to remat which point to Tensor.
198 +// To build the cycle remat support a default constructor,
199 +// And allow you to fill in the member later.
200 +struct Rematerializer : intrusive_ptr_target {
201 +    rematerialize_function_t func;
202 +    strongs inputs;
203 +    weaks outputs;
204 +    duration_t compute_cost;
205 +    // when some output in here get evicted, they should belong to this ecn.
206 +    // a rematerializer have to track this,
207 +    // because when multiple output of a rematerializer get evicted,
208 +    // we only want to count the compute cost once.
209 +    ecn_ptr ecn;
210 +    Rematerializer(const Unsafe&,
211 +        const rematerialize_function_t& func,
212 +        const strongs& inputs,
213 +        duration_t compute_cost) :
214 +        func(func),

```

```

213 +     inputs(inputs),
214 +     compute_cost(compute_cost) {
215 + }
216 + void release_resources() final {
217 +     func = rematerialize_function_t();
218 +     inputs.clear();
219 +     outputs.clear();
220 + }
221 + void remat();
222 + ecn_ptr get_ecn();
223 + CheckpointInfo get_cpi();
224 +};
225 +
226 +// Track all Tensor that share the same Storage.
227 +// This is the atomic level of eviction - when evicting, everything here
will get evicted.
228 +// When an AliasPool is evicted, the Storage of the underlying tensor must
be freed.
229 +// Additionally, the AliasPool contain weak pointer to all children of
tensors,
230 +// in order to compute the score of evicting a Storage.
231 +struct AliasPool : intrusive_ptr_target {
232 +     weakens tensors;
233 +     weakens neighbors;
234 +     std::set<ecn_ptr> neighbor_ecn();
235 +     // get() might hold some raw Tensor, rendering them unevictable.
236 +     // it is likely that get() will run out of memory, and when it does so, it
will try to evict.
237 +     // so, it is crucial that we dont try to evict those tensors - doing so
will not evict anything.
238 +     // lock_count count how many time a tensor is referenced by get.
239 +     size_t lock_count = 0;
240 +     size_t external_count = 0;
241 +     void lock() {
242 +         ++lock_count;
243 +     }
244 +     void unlock() {
245 +         --lock_count;
246 +     }
247 +     intrusive_ptr<Rematerializer> head_remat;
248 +     bool evictable() const {
249 +         return lock_count == 0 && head_remat;
250 +     }
251 +     // if it is not evictable it must not be evicted.
252 +     bool is_evicted = false;
253 +     size_t memory;
254 +     time_t last_used_time;

```

```

255 + // An aliaspool cant register itself to the checkpointpool - you have to
    do it yourself.
256 + AliasPool(const Unsafe&, intrusive_ptr<Rematerializer> head_remat, size_t
    memory) :
257 +     head_remat(head_remat),
258 +     memory(memory),
259 +     last_used_time(std::chrono::system_clock::now()) {
260 + }
261 + // if it is evicted, then hold the evicted tensor group.
262 + ecn_ptr ecn;
263 + double cost(time_t current_time);
264 + void evict();
265 + void register_external() {
266 +     ++external_count;
267 + }
268 + void release_external() {
269 +     --external_count;
270 +     if (external_count == 0) {
271 +         if (lock_count > 0) {return;}
272 +         TORCH_CHECK(lock_count == 0);
273 +         if (memory > 0 && (!ecn) && head_remat) {
274 +             evict();
275 +         }
276 +     }
277 + }
278 + // if it was evicted, refresh it. otherwise do nothing.
279 + // have to check so, because when we rematerialize a single tensor in an
    aliaspool,
280 + // we will set it to non-evicted, and when we rematerialize it's tensor
    they will also reset this.
281 + void set_not_evicted(const intrusive_ptr<AliasPool>& self);
282 + void release_resources() final {
283 +     tensors.clear();
284 +     neighbors.clear();
285 +     head_remat.reset();
286 + }
287 +};
288 +
289 +struct CheckpointTensorCell : intrusive_ptr_target {
290 +     std::unique_ptr<Tensor> t;
291 +     bool defined = false;
292 +     bool is_undefined_tensor;
293 +     DispatchKeySet key_set_;
294 +     DispatchKeySet key_set() const {
295 +         TORCH_CHECK(defined);
296 +         return key_set_;
297 +     }

```

```

298 + caffe2::TypeMeta dtype_;
299 + caffe2::TypeMeta dtype() const {
300 +     TORCH_CHECK(defined);
301 +     return dtype_;
302 + }
303 + c10::optional<Device> optional_device_;
304 + c10::optional<Device> optional_device() const {
305 +     TORCH_CHECK(defined);
306 +     return optional_device_;
307 + }
308 + // A Tensor is evictable iff it's AliasPool is evictable.
309 + // A evictable tensor must have Rematerializer.
310 + intrusive_ptr<AliasPool> pool;
311 + intrusive_ptr<Rematerializer> remat;
312 + void evict() {
313 +     TORCH_CHECK(remat);
314 +     t.reset();
315 + }
316 + void fill(const Tensor& t);
317 + explicit CheckpointTensorCell(const Tensor& t, const
    intrusive_ptr<AliasPool>& pool) : pool(pool) {
318 +     fill(t);
319 + }
320 + explicit CheckpointTensorCell(const Tensor& t,
321 +                                const intrusive_ptr<AliasPool>& pool,
322 +                                const intrusive_ptr<Rematerializer>& remat) :
323 +     pool(pool), remat(remat) {
324 +     fill(t);
325 + }
326 + size_t memory() {
327 +     TORCH_CHECK(defined);
328 +     return pool->memory;
329 + }
330 + Tensor get() {
331 +     if (! t) {
332 +         TORCH_CHECK(remat);
333 +         remat->remat();
334 +     }
335 +     TORCH_CHECK(t);
336 +     TORCH_CHECK(! t->key_set().has(DispatchKey::CheckpointTensorId));
337 +     pool->last_used_time = std::chrono::system_clock::now();
338 +     return *t;
339 + }
340 + void pin() {
341 +     get();
342 +     pool->head_remat.reset();
343 +     remat.reset();

```

```

344 + }
345 + void release_resources() final {
346 +     t.reset();
347 +     pool.reset();
348 +     remat.reset();
349 + }
350 +};
351 +
352 +// An external reference.
353 +// Each strong will have at most one external reference.
354 +// By keeping such an invariant, whenever an external reference die,
355 +// We know that the underlying strong is only used internally.
356 +// Thus, when it die we can apply optimization like banishing/infinite
    staleness.
357 +// We keep this invariant by only allowing CheckpointTensorImpl to make new
    External,
358 +// When new CheckpointTensorImpl is constructed.
359 +struct External : intrusive_ptr_target {
360 +     External(const strong& value) : value(value) {
361 +         value->pool->register_external();
362 +     }
363 +     External(const Tensor& value) :
364 +         External(strong::make(value,
365 +                                 intrusive_ptr<AliasPool>::make(Unsafe(),
366 +
    intrusive_ptr<Rematerializer>()),
    memory(value)))) { }
367 +
368 +     External(const Tensor& value,
369 +             const intrusive_ptr<AliasPool>& pool,
370 +             const intrusive_ptr<Rematerializer>& remat) :
371 +         External(strong::make(value, pool, remat)) { }
372 +     strong value;
373 +     void release_resources() override;
374 +};
375 +
376 +inline DispatchKeySet convert_key_set(const DispatchKeySet& t) {
377 +     CHECK(!t.has(DispatchKey::Checkpoint));
378 +     auto ret = t.add(DispatchKey::Checkpoint);
379 +     return ret;
380 +}
381 +
382 +struct CheckpointTensorImpl : TensorImpl {
383 +     int id = gen_counter();
384 +     static int counter;
385 +     static int gen_counter() {
386 +         return counter++;
387 +     }

```

```

388 + std::string counter_name() const {
389 +     return std::string("x") + std::to_string(id);
390 + }
391 +
392 + Ref<intrusive_ptr<External>> ref;
393 +
394 + void release_resources() final;
395 +
396 + explicit CheckpointTensorImpl(const Ref<intrusive_ptr<External>>& ref) :
397 +     TensorImpl(convert_key_set(ref->value->value->key_set()),
398 +                 ref->value->value->dtype(),
399 +                 ref->value->value->optional_device()),
400 +     ref(ref) {
401 +     if (key_set().has(DispatchKey::Autograd)) {
402 +         set_requires_grad(true);
403 +     }
404 + }
405 +
406 + explicit CheckpointTensorImpl(const intrusive_ptr<External>& e) :
407 +     CheckpointTensorImpl(Ref<intrusive_ptr<External>>::make(e)) { }
408 +
409 + explicit CheckpointTensorImpl(const Tensor& t);
410 +
411 + static Tensors make(const std::string& name,
412 +                     const rematerialize_function_t& remat,
413 +                     const Tensors& inputs);
414 +
415 + // mutate_idx indicate which of the inputs will get mutated.
416 + static void mutate(const std::string& name,
417 +                   const mutate_function_t& mutate,
418 +                   const Tensors& inputs,
419 +                   const std::vector<size_t>& mutate_idx);
420 + intrusive_ptr<TensorImpl> shallow_copy_and_detach(const VariableVersion&
421 + version_counter,
422 +
423 +                                     bool
424 + allow_tensor_metadata_change) const override;
425 + void shallow_copy_from(const c10::intrusive_ptr<TensorImpl>& impl)
426 + override;
427 + int64_t dim() const override {
428 +     return ref->value->value->get().dim();
429 + }
430 + int64_t numel() const override {
431 +     return ref->value->value->get().numel();
432 + }
433 + IntArrayRef sizes() const override {
434 +     return ref->value->value->get().sizes();
435 + }

```



```

432 + int64_t size(int64_t d) const override {
433 +     return ref->value->value->get().size(d);
434 + }
435 + IntArrayRef strides() const override {
436 +     return ref->value->value->get().strides();
437 + }
438 + int64_t stride(int64_t d) const override {
439 +     return ref->value->value->get().stride(d);
440 + }
441 + bool has_storage() const override {
442 +     return false;
443 + }
444 +};
445 +
446 +// CheckpointPool keep a list of AliasPool, and search over them to choose
447 +// the best one to evict.
447 +struct CheckpointPool {
448 +    std::vector<weak_intrusive_ptr<AliasPool>> aps;
449 +    std::vector<weak_intrusive_ptr<External>> exts;
450 +    std::random_device rd;
451 +    std::mt19937 gen = std::mt19937(rd());
452 +    // whether to take a square-root sample of the pool during an eviction loop
453 +    bool sample_tensors = true;
454 +    // ignore tensors < 1% of the average tensor size
455 +    bool ignore_small_tensors = true;
456 +    bool has_memory_budget = false;
457 +    long memory_budget;
458 +    void evict();
459 +    void auto_evict();
460 +    void clear_checkpointpool();
461 +    void add(const intrusive_ptr<AliasPool>&);
462 +    CheckpointPool();
463 +};
464 +
465 +inline CheckpointTensorImpl* get_cpti(const Tensor& t) {
466 +    auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
467 +    TORCH_CHECK(cpti != nullptr);
468 +    return cpti;
469 +}
470 +
471 +inline Ref<intrusive_ptr<External>> cell_from_tensor(const Tensor& t) {
472 +    return get_cpti(t)->ref;
473 +}
474 +
475 +}

```

DELTA

```
1 diff -ru dtr_pytorch/aten/src/ATen/CheckpointTensorImpl.h
delta/aten/src/ATen/CheckpointTensorImpl.h
2 --- dtr_pytorch/aten/src/ATen/CheckpointTensorImpl.h      2024-04-28
16:09:10.546175266 +0800
3 +++ delta/aten/src/ATen/CheckpointTensorImpl.h            2024-04-09
14:45:52.849651606 +0800
4 @@ -4,6 +4,7 @@
5     #include <memory>
6     #include <numeric>
7     #include <random>
8     +#include <queue>
9
10    #include <c10/core/Backend.h>
11    #include <c10/core/MemoryFormat.h>
12 @@ -24,7 +25,7 @@
13
14    #define likely(x)      __builtin_expect(!!(x), 1)
15    #define unlikely(x)    __builtin_expect(!!(x), 0)
16    -#define TORCH_CHECK(a, ...) // profile mode
17    +// #define TORCH_CHECK(a, ...) // profile mode
18
19    // System Description:
20    // Every Tensor is managed by a CheckpointTensor,
21 @@ -160,12 +161,30 @@
22    using duration_t = std::chrono::system_clock::duration;
23    struct CheckpointInfo {
24        duration_t compute_cost;
25        - // Floating Point instability?
26        + double bandwidth = 0.35*32*1024*1024*1024/1000000;
27        + // @ZACH: Floating Point instability?
28        double cost(size_t memory, size_t staleness) const {
29            TORCH_CHECK(memory > 0);
30            TORCH_CHECK(staleness > 0);
31            + // Filter Function
32            + return 1 / static_cast<double>(memory * staleness);
33            + // return compute_cost.count() / static_cast<double>(memory * staleness);
34        }
35        + double compute_cost_func(size_t memory, size_t staleness) const {
36            + TORCH_CHECK(memory > 0);
37            + TORCH_CHECK(staleness > 0);
38            + return compute_cost.count() / static_cast<double>(memory * staleness);
39        }
40        + double swap_cost_(size_t memory) const {
41            + TORCH_CHECK(memory > 0);
```

```

42 +     return static_cast<double>(memory) / bandwidth;
43 + }
44 + double fake_decision(size_t memory, size_t staleness) const {
45 +     TORCH_CHECK(memory > 0);
46 +     TORCH_CHECK(staleness > 0);
47 +     double swap_cost = static_cast<double>(memory) / bandwidth;
48 +     return (2 * swap_cost) / compute_cost.count();
49 + }
50     CheckpointInfo(duration_t compute_cost) :
51         compute_cost(compute_cost) {
52     }
53 @@ -194,6 +213,8 @@
54     strongs inputs;
55     weaks outputs;
56     duration_t compute_cost;
57 + // add for reload
58 + duration_t swap_cost;
59     // when some output in here get evicted, they should belong to this ecn.
60     // a rematerializer have to track this,
61     // because when multiple output of a rematerializer get evicted,
62 @@ -213,6 +234,7 @@
63     outputs.clear();
64 }
65 void remat();
66 + void reload();
67     ecn_ptr get_ecn();
68     CheckpointInfo get_cpi();
69 };
70 @@ -244,8 +266,16 @@
71 }
72 // if it is not evictable it must not be evicted.
73 bool is_evicted = false;
74 + bool is_offloaded = false;
75 + bool onGPU = true;
76 + bool swapped = false;
77     size_t memory;
78     time_t last_used_time;
79 + double swap_cost;
80 + std::shared_ptr<Storage> cpuStorage;
81 + // for prefetch
82 + // std::queue<strong> offload_queue;
83 + // void prefetch();
84     // An aliaspool cant register itself to the checkpointpool - you have to
    do it yourself.
85     AliasPool(const Unsafe&, intrusive_ptr<Rematerializer> head_remat, size_t
memory) :
86         head_remat(head_remat),

```

```

87  @@ -255,7 +285,10 @@
88      // if it is evicted, then hold the evicted tensor group.
89      ecn_ptr ecn;
90      double cost(time_t current_time);
91  + double compute_cost_func(time_t current_time);
92  + double decision_func(time_t current_time);
93      void evict();
94  + void offload();
95      void register_external() {
96          ++external_count;
97      }
98  @@ -273,6 +306,7 @@
99      // have to check so, because when we rematerialize a single tensor in an
100      aliaspool,
101      // we will set it to non-evicted, and when we rematerialize it's tensor
102      they will also reset this.
103      void set_not_evicted(const intrusive_ptr<AliasPool>& self);
104  + void set_not_offloaded(const intrusive_ptr<AliasPool>& self);
105      void release_resources() final {
106          tensors.clear();
107          neighbors.clear();
108  @@ -284,6 +318,13 @@
109      std::unique_ptr<Tensor> t;
110      bool defined = false;
111      bool is_undefined_tensor;
112  + bool evicted = false;
113  + bool offloaded = false;
114  + bool onGPU = true;
115  + double compute_cost;
116  + double swap_cost;
117  + std::shared_ptr<Storage> cpuStorage;
118  + Tensor cpuTensor;
119      DispatchKeySet key_set_;
120      DispatchKeySet key_set() const {
121          TORCH_CHECK(defined);
122  @@ -305,8 +346,15 @@
123      intrusive_ptr<Rematerializer> remat;
124      void evict() {
125          TORCH_CHECK(remat);
126  + onGPU = false;
127  + evicted = true;
128  + t.reset();
129  + }
130  + void offload() {
131  + TORCH_CHECK(remat);
132      t.reset();
133  }

```

```

132 + void reload();
133     void fill(const Tensor& t);
134     explicit CheckpointTensorCell(const Tensor& t, const
intrusive_ptr<AliasPool>& pool) : pool(pool) {
135         fill(t);
136 @@ -323,11 +371,16 @@
137     }
138     Tensor get() {
139         if (! t) {
140 -         TORCH_CHECK(remat);
141 -         remat->remat();
142 +         if (evicted) {
143 +             TORCH_CHECK(remat);
144 +             remat->remat();
145 +         } else if (offloaded) {
146 +             reload();
147 +         } else {
148 +             TORCH_CHECK(false); }
149         }
150         TORCH_CHECK(t);
151 -         TORCH_CHECK(! t->key_set().has(DispatchKey::CheckpointTensorId));
152 +         TORCH_CHECK(! t->key_set().has(DispatchKey::Checkpoint));
153         pool->last_used_time = std::chrono::system_clock::now();
154         return *t;
155     }
156 @@ -368,9 +421,16 @@
157 };
158
159 inline DispatchKeySet convert_key_set(const DispatchKeySet& t) {
160 - CHECK(!t.has(DispatchKey::Checkpoint));
161 - auto ret = t.add(DispatchKey::Checkpoint);
162 - return ret;
163 + // CHECK(!t.has(DispatchKey::Checkpoint));
164 + // auto ret = t.add(DispatchKey::Checkpoint);
165 + if (!t.has(DispatchKey::Checkpoint)) {
166 +     auto ret = t.add(DispatchKey::Checkpoint);
167 +     return ret;
168 + }
169 + else {
170 +     auto ret = t;
171 +     return ret;
172 + }
173 }
174
175 struct CheckpointTensorImpl : TensorImpl {
176 @@ -444,7 +504,7 @@
177     std::random_device rd;

```

```

178     std::mt19937 gen = std::mt19937(rd());
179     // whether to take a square-root sample of the pool during an eviction loop
180 -    bool sample_tensors = true;
181 +    bool sample_tensors = false;
182     // ignore tensors < 1% of the average tensor size
183     bool ignore_small_tensors = true;
184     bool has_memory_budget = false;
185 @@ -453,6 +513,11 @@
186     void auto_evict();
187     void clear_checkpointpool();
188     void add(const intrusive_ptr<AliasPool>&);
189 +    // for prefetch
190 +    std::queue<strong> offload_queue;
191 +    void prefetch();
192 +    int64_t prefetch_count = 5;
193 +    // void clear_offload_queue(std::queue<strong> q);
194     CheckpointPool();
195 };

```

!!!新增 `aten/src/ATen/CheckpointTensorImpl.cpp`

DTR

```

1  diff --git a/aten/src/ATen/CheckpointTensorImpl.cpp
    b/aten/src/ATen/CheckpointTensorImpl.cpp
2  new file mode 100644
3  index 000000000000..7c29fdc2f0
4  --- /dev/null
5  +++ b/aten/src/ATen/CheckpointTensorImpl.cpp
6  @@ -0,0 +1,710 @@
7  +#include <ATen/CheckpointTensorImpl.h>
8  +#include <ATen/Logger.h>
9  +#include <c10/cuda/CUDACachingAllocator.h>
10 +
11 +#include <chrono>
12 +#include <string>
13 +#include <random>
14 +#include <cmath>
15 +
16 +namespace at {
17 +
18 +using Clock = std::chrono::high_resolution_clock;
19 +using Time = Clock::time_point;
20 +using Duration = Clock::duration;

```

```

21 +using FinalTime = std::chrono::nanoseconds;
22 +
23 +struct PerfStats;
24 +
25 +struct Timer {
26 +   std::string name;
27 +   Time start;
28 +   Timer(std::string name, Time start) : name(name), start(start) {}
29 +   Timer() {}
30 +   ~Timer();
31 +};
32 +
33 +constexpr bool stats = true;
34 +
35 +struct PerfStats {
36 +   using TimerStats = std::tuple<std::string, Time, Time, Duration>;
37 +   Time start;
38 +   std::unordered_map<std::string, int> calls;
39 +   std::vector<PerfStats::TimerStats> timers;
40 +
41 +   PerfStats() : start(Clock::now()), calls(0), timers() {}
42 +
43 +   /*Timer track(std::string name) {
44 +       if (stats) {
45 +           auto it = this->calls.find(name);
46 +           if (it != this->calls.end()) {
47 +               it->second += 1;
48 +           } else {
49 +               this->calls.insert({name, 0});
50 +           }
51 +
52 +           return Timer(name, Clock::now());
53 +       }
54 +       return Timer();
55 +   }*/
56 +   void track(const char*) { }
57 +
58 +   ~PerfStats() {
59 +       if (!stats) { return; }
60 +       auto start = std::get<1>(this->timers[0]);
61 +       auto now = Clock::now();
62 +       std::cout << "All done. Here are some perf stats fresh off the preses."
63 +       << std::endl;
64 +
65 +       std::unordered_map<std::string, Duration> durations;
66 +
67 +       Duration total = now - this->start;

```

```

67 + // For now simple strategy, count up all the time taken
68 + // by each "tagged call site".
69 + for (auto timer : timers) {
70 +     auto name = std::get<0>(timer);
71 +     Duration duration = std::get<3>(timer);
72 +     auto it = durations.find(name);
73 +     if (it != durations.end()) {
74 +         it->second += duration;
75 +     } else {
76 +         durations.insert({name, duration});
77 +     }
78 + }
79 +
80 + std::vector<std::pair<std::string, Duration>> data;
81 +
82 + // Convert the durations
83 + for (auto d : durations) {
84 +     // auto duration = std::chrono::duration_cast<FinalTime>(d.second);
85 +     data.push_back(d);
86 + }
87 +
88 + std::sort(data.begin(), data.end(),
89 + [](const std::pair<std::string, Duration> & a, const
std::pair<std::string, Duration> & b) -> bool {
90 +     return a.second > b.second;
91 + });
92 +
93 + for (auto d : data) {
94 +     auto duration = std::chrono::duration_cast<FinalTime>(d.second);
95 +     auto total_duration = std::chrono::duration_cast<FinalTime>(total);
96 +     double percentage =
((double)duration.count())/((double)total_duration.count()) * 100;
97 +     auto call_count = this->calls.find(d.first);
98 +     TORCH_CHECK(call_count != this->calls.end());
99 +     std::cout << "CallSite: " << d.first << " CallCount: " << call_count-
>second << " Cost: " << duration.count() << "ns" << " (% " << percentage << " "
<< std::endl;
100 + }
101 + }
102 +};
103 +
104 +static PerfStats STATS = PerfStats();
105 +
106 +size_t memory_sum = 0;
107 +size_t memory_max = 0;
108 +size_t memory_count = 0;
109 +

```



```

110 +void reset_memory_stat() {
111 +   memory_sum = 0;
112 +   memory_max = 0;
113 +   memory_count = 0;
114 +}
115 +
116 +inline size_t memory(const Tensor& t) {
117 +   if (! t.has_storage()) {
118 +       return 0;
119 +   }
120 +   auto& storage = t.storage();
121 +   size_t res = storage.nbytes();
122 +   memory_sum += res;
123 +   memory_max = std::max(memory_max, res);
124 +   memory_count += 1;
125 +   return res;
126 +}
127 +
128 +Timer::~Timer() {
129 +   Time now = Clock::now();
130 +   Duration elapsed = now - start;
131 +   PerfStats::TimerStats stats = { name , start, now, elapsed };
132 +   STATS.timers.push_back(stats);
133 +}
134 +
135 +bool use_log_ = false;
136 +bool use_profile_ = false;
137 +long base_compute_time_ = 0;
138 +long remat_compute_time_ = 0;
139 +long search_time_ = 0;
140 +long cost_time_ = 0;
141 +
142 +CheckpointPool pool;
143 +void CheckpointPool::add(const intrusive_ptr<AliasPool>& p) {
144 +   if (p->memory > 0 && (memory_count == 0 || !ignore_small_tensors || p->memory >= 0.01 * double(memory_sum/memory_count))) {
145 +       aps.push_back(weak_intrusive_ptr<AliasPool>(p));
146 +   }
147 +}
148 +
149 +long current_memory() {
150 +   STATS.track("current_memory");
151 +   auto device_stat = c10::cuda::CUDACachingAllocator::getDeviceStats(0);
152 +   return device_stat.allocated_bytes[0].current;
153 +}
154 +
155 +void CheckpointPool::auto_evict() {

```

```

156 + STATS.track("CheckpointPool::auto_evict");
157 + if (has_memory_budget) {
158 +     while (current_memory() > memory_budget) {
159 +         evict();
160 +     }
161 + }
162 +}
163 +
164 +void CheckpointPool::evict() {
165 +     time_t pre = std::chrono::system_clock::now();
166 +     STATS.track("CheckpointPool::evict");
167 +     TORCH_CHECK(aps.size() > 0);
168 +     // shrunk: either something has been evicted or the pools have gotten
169 +     smaller
170 +     bool shrunk = false;
171 +     int evict_idx = -1;
172 +     double evict_cost = INFINITY;
173 +     time_t current_time = std::chrono::system_clock::now();
174 +     auto remove_from_aps = [&](size_t i) {
175 +         aps[i] = aps[aps.size() - 1];
176 +         aps.pop_back();
177 +     };
178 +     std::uniform_int_distribution<> distrib(1, 1 * std::max(1, static_cast<int>
179 +         (std::sqrt(aps.size()))));
180 +     // sampling a random independent subset of all evictable tensors to find
181 +     the cheapest tensor to evict.
182 +     for (size_t i = 0; i < aps.size(); i++) {
183 +         auto cannot_evict = [&]() {
184 +             shrunk = true;
185 +             remove_from_aps(i);
186 +         };
187 +         auto ap_strong = aps[i].lock();
188 +         if (!ap_strong.defined()) {
189 +             cannot_evict();
190 +         }
191 +         else if (ap_strong->ecn) {
192 +             cannot_evict();
193 +         }
194 +         else {
195 +             if (ap_strong->evictable()) {
196 +                 double cost = ap_strong->cost(current_time);
197 +                 if (cost < evict_cost) {
198 +                     evict_cost = cost;
199 +                     evict_idx = i;

```

```

200 +     if (sample_tensors) {
201 +         i += distrib(gen);
202 +     } else {
203 +         i += 1;
204 +     }
205 + }
206 + }
207 + if (evict_idx == -1) {
208 +     TORCH_CHECK(shrunk);
209 + } else {
210 +     auto evict_from_idx = [&](size_t idx) {
211 +         auto ap_strong = aps[idx].lock();
212 +         TORCH_CHECK(ap_strong.defined());
213 +         ap_strong->evict();
214 +         remove_from_aps(evict_idx);
215 +     };
216 +     evict_from_idx(evict_idx);
217 + }
218 + time_t post = std::chrono::system_clock::now();
219 + search_time_ += (post - pre).count();
220 +}
221 +
222 +CheckpointPool::CheckpointPool() { }
223 +
224 +namespace native {
225 +
226 +Tensor checkpoint(const Tensor& t) {
227 +    STATS.track("checkpoint");
228 +    auto cpti = intrusive_ptr<CheckpointTensorImpl>::make(t);
229 +    if (use_log_) {
230 +        DTRLogConstant(cpti->counter_name());
231 +        DTRLogMemory(cpti->counter_name(), cpti->ref->value->value->memory());
232 +    }
233 +    return Tensor(cpti);
234 +}
235 +
236 +Tensor uncheckpoint(const Tensor& t) {
237 +    STATS.track("uncheckpoint");
238 +    auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
239 +    TORCH_CHECK(cpti != nullptr);
240 +    return cpti->ref->value->value->get();
241 +}
242 +
243 +void pin(Tensor& t) {
244 +    STATS.track("pin");
245 +    auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
246 +    TORCH_CHECK(cpti != nullptr);

```

```

247 + cpti->ref->value->value->pin();
248 +}
249 +
250 +Tensor decheckpoint(const Tensor& t) {
251 +  STATS.track("decheckpoint");
252 +  auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
253 +  return cpti ? cpti->ref->value->value->get() : t;
254 +}
255 +
256 +bool is_checkpoint(const Tensor& t) {
257 +  STATS.track("is_checkpoint");
258 +  auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
259 +  return cpti != nullptr;
260 +}
261 +
262 +Tensor try_checkpoint(const Tensor& t) {
263 +  STATS.track("try_checkpoint");
264 +  return is_checkpoint(t) ? t : checkpoint(t);
265 +}
266 +
267 +void new_log(std::string str) {
268 +  DTRLogger::logger().out =
269 +    std::ofstream(DTRLogger::logger().get_filename(str));
270 +}
271 +
272 +void annotate_log(std::string str) {
273 +  if (!use_log_) { return; }
274 +  if (log_json) {
275 +    json j;
276 +    j[INSTRUCTION] = "ANNOTATE";
277 +    j[ANNOTATION] = str;
278 +    DTRLogger::logger().log(j.dump());
279 +  } else {
280 +    DTRLogger::logger().log("# " + str);
281 +  }
282 +}
283 +
284 +void toggle_log(bool b) {
285 +  use_log_ = b;
286 +}
287 +
288 +void clear_checkpointpool() {
289 +  while (likely(!pool.exts.empty())) {
290 +    if (auto e = pool.exts.back().lock()) {
291 +      e->value->pin();
292 +    }
293 +    pool.exts.pop_back();

```

```
293 + }
294 +}
295 +
296 +void unset_memory_budget() {
297 + pool.has_memory_budget = false;
298 +}
299 +
300 +void set_memory_budget(long budget) {
301 + pool.memory_budget = budget;
302 + pool.has_memory_budget = true;
303 +}
304 +
305 +void toggle_sampling(bool sample) {
306 + pool.sample_tensors = sample;
307 +}
308 +
309 +void toggle_ignore_small_tensors(bool ignore) {
310 + pool.ignore_small_tensors = ignore;
311 +}
312 +
313 +void reset_profile() {
314 + base_compute_time_ = 0;
315 + remat_compute_time_ = 0;
316 + search_time_ = 0;
317 + cost_time_ = 0;
318 +}
319 +
320 +void toggle_profile(bool profile) {
321 + use_profile_ = profile;
322 +}
323 +
324 +long compute_time() {
325 + return base_compute_time() + remat_compute_time();
326 +}
327 +
328 +long cost_time() {
329 + return cost_time_;
330 +}
331 +
332 +long search_time() {
333 + return search_time_;
334 +}
335 +
336 +long remat_compute_time() {
337 + return remat_compute_time_;
338 +}
339 +
```

```

340 +long base_compute_time() {
341 +   return base_compute_time_;
342 +}
343 +
344 +long loop_time() {
345 +   return search_time() - cost_time();
346 +}
347 +
348 +}
349 +
350 +[[inline]]
351 +Tensor uncheckpoint(const strong& input) {
352 +   return input->get();
353 +}
354 +
355 +Tensors uncheckpoint(const strongs& inputs) {
356 +   STATS.track("uncheckpoint");
357 +   Tensors ret;
358 +   ret.reserve(inputs.size());
359 +   for (const strong& input : inputs) {
360 +       // inlined manually
361 +       ret.push_back(input->get());
362 +   }
363 +   return ret;
364 +};
365 +
366 +Tensors try_checkpoint(const Tensors& inputs) {
367 +   STATS.track("try_checkpoint");
368 +   Tensors ret;
369 +   ret.reserve(inputs.size());
370 +   for (const Tensor& input : inputs) {
371 +       ret.push_back(at::native::try_checkpoint(input));
372 +   }
373 +   return ret;
374 +}
375 +
376 +CheckpointInfo merge_cpi(CheckpointInfo l, CheckpointInfo r) {
377 +   STATS.track("merge_cpi");
378 +   return CheckpointInfo(l.compute_cost + r.compute_cost);
379 +}
380 +
381 +void AliasPool::evict() {
382 +   STATS.track("AliasPool::evict");
383 +   TORCH_CHECK(!ecn);
384 +   ecn = head_remat->get_ecn();
385 +   auto ecns = neighbor_ecn();
386 +   for (const auto& necn : ecns) {

```

```

387 +     merge<CheckpointInfo>(merge_cpi, ecn, necn);
388 + }
389 + TORCH_CHECK(memory > 0);
390 + TORCH_CHECK(lock_count == 0);
391 + TORCH_CHECK(!is_evicted);
392 + is_evicted = true;
393 + for (const weak& w : tensors) {
394 +     if (auto cell = w.lock()) {
395 +         cell->evict();
396 +     }
397 + }
398 +}
399 +
400 +double AliasPool::cost(time_t current_time) {
401 +     time_t pre = std::chrono::system_clock::now();
402 +     auto cpi = head_remat->get_cpi();
403 +     auto ecns = neighbor_ecn();
404 +     for (const auto& necn : ecns) {
405 +         cpi = merge_cpi(cpi, get_t(necn));
406 +     }
407 +     auto ret = cpi.cost(memory, (current_time - last_used_time).count());
408 +     time_t post = std::chrono::system_clock::now();
409 +     cost_time_ += (post - pre).count();
410 +     return ret;
411 +}
412 +
413 +void External::release_resources() {
414 +     value->pool->release_external();
415 +     value.reset();
416 +}
417 +
418 +void Rematerializer::remat() {
419 +     // TODO: refactor using RAII for exception safety.
420 +     for (const strong& s : inputs) {
421 +         s->pool->lock();
422 +     }
423 +     Tensors ts = uncheckpoint(inputs);
424 +     time_t pre = std::chrono::system_clock::now();
425 +     auto ret = func(ts);
426 +     time_t post = std::chrono::system_clock::now();
427 +     pool.auto_evict();
428 +     remat_compute_time_ += (post - pre).count();
429 +     TORCH_CHECK(ret.size() == outputs.size());
430 +     for (size_t i = 0; i < outputs.size(); ++i) {
431 +         if (auto output_cell = outputs[i].lock()) {
432 +             output_cell->fill(ret[i]);
433 +         }

```

```

434 + }
435 + ecn.reset();
436 + for (const strong& s : inputs) {
437 +     s->pool->unlock();
438 + }
439 +}
440 +
441 +ecn_ptr Rematerializer::get_ecn() {
442 +     if (!ecn) {
443 +         ecn = ecn_ptr::make(CheckpointInfo(compute_cost));
444 +     }
445 +     return ecn;
446 +}
447 +
448 +CheckpointInfo Rematerializer::get_cpi() {
449 +     return CheckpointInfo(ecn ? duration_t(0) : compute_cost);
450 +}
451 +
452 +std::set<ecn_ptr> AliasPool::neighbor_ecn() {
453 +     STATS.track("AliasPool::neighbor_ecn");
454 +     std::set<ecn_ptr> ptr_set;
455 +     int size = neighbors.size();
456 +     for (size_t i = 0; i < size; i++) {
457 +         if (auto cptc = neighbors[i].lock()) {
458 +             if (cptc->pool->ecn) {
459 +                 ptr_set.insert(cptc->pool->ecn);
460 +             }
461 +             ++i;
462 +         } else {
463 +             neighbors[i] = neighbors[size - 1];
464 +             size = size - 1;
465 +         }
466 +     }
467 +     if (size < neighbors.size()) {
468 +         neighbors.erase(neighbors.begin() + size);
469 +     }
470 +     return ptr_set;
471 +}
472 +
473 +void AliasPool::set_not_evicted(const intrusive_ptr<AliasPool>& self) {
474 +     if (unlikely(is_evicted)) {
475 +         STATS.track("AliasPool::set_not_evicted(inside)");
476 +         is_evicted = false;
477 +         if (ecn) {
478 +             TORCH_CHECK(head_remat);
479 +             auto cpi = get_t(ecn);

```



```

480 +     update_t(ecn, CheckpointInfo(cpi.compute_cost - head_remat-
    >compute_cost));
481 +     ecn.reset();
482 + }
483 + pool.add(self);
484 + }
485 +}
486 +
487 +void CheckpointTensorCell::fill(const Tensor& t) {
488 +     STATS.track("CheckpointTensorCell::fill");
489 +     if (!(this->t)) {
490 +         this->t = std::make_unique<Tensor>(t.detach());
491 +         pool->set_not_evicted(pool);
492 +         if (!defined) {
493 +             defined = true;
494 +             is_undefined_tensor = !t.defined();
495 +             key_set_ = t.key_set();
496 +             if (t.requires_grad()) {
497 +                 key_set_ = key_set_.add(DispatchKey::Autograd);
498 +             }
499 +             dtype_ = t.dtype();
500 +             optional_device_ = t.optional_device();
501 +         }
502 +     }
503 +}
504 +
505 +intrusive_ptr<TensorImpl> CheckpointTensorImpl::shallow_copy_and_detach(const
    VariableVersion& version_counter,
506 +
    bool
    allow_tensor_metadata_change) const {
507 +     auto ret = intrusive_ptr<CheckpointTensorImpl>::make(ref);
508 +     if (use_log_) {
509 +         DTRLogCopy(ret->counter_name(), counter_name());
510 +     }
511 +     return ret;
512 +}
513 +
514 +void CheckpointTensorImpl::shallow_copy_from(const
    c10::intrusive_ptr<TensorImpl>& impl) {
515 +     STATS.track("CheckpointTensorCell::shallow_copy_from");
516 +     TORCH_CHECK(impl->key_set().has(DispatchKey::CheckpointTensorId));
517 +     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(impl.get());
518 +     TORCH_CHECK(cpti != nullptr);
519 +     ref->value = cpti->ref->value;
520 +     if (use_log_) {
521 +         DTRLogCopyFrom(counter_name(), cpti->counter_name());
522 +     }

```

```

523 +}
524 +
525 +int CheckpointTensorImpl::counter = 0;
526 +
527 +bool is_alias(const Tensor& l, const Tensor& r) {
528 +     return l.defined() && r.defined() && l.is_alias_of(r);
529 +}
530 +
531 +// return an index for alias.
532 +// we dont care which one because they all lead to the same alias pool.
533 +// return -1 for no alias.
534 +// may god forgive my sin.
535 +int get_alias(const Tensors& ts, const Tensor& t) {
536 +     if (t.defined()) {
537 +         for (size_t i = 0; i < ts.size(); ++i) {
538 +             if (ts[i].defined() && t.is_alias_of(ts[i])) {
539 +                 return i;
540 +             }
541 +         }
542 +     }
543 +     return -1;
544 +}
545 +
546 +struct MakeRawResult {
547 +     std::vector<intrusive_ptr<External>> outputs;
548 +     std::vector<int> aliases;
549 +     duration_t time;
550 +     intrusive_ptr<Rematerializer> rematerializer;
551 +};
552 +
553 +void add_neighbor(const strong& l, const strong& r) {
554 +     l->pool->neighbors.push_back(weak(r));
555 +     r->pool->neighbors.push_back(weak(l));
556 +}
557 +
558 +// remat take a single vector of tensors,
559 +// while there are two vector, one storing nonconstants and one storing
560 +// constants.
561 +// the constants are small and they will not be considered for eviction.
562 +// however, we have to stitch the two vectors together to pass it in remat.
563 +// the size_t in constants decide the location to stitch them in, while
564 +// input_values fill in the rest.
565 +MakeRawResult make_raw(const rematerialize_function_t& remat_f,
566 +                        const strongs& inputs) {
567 +     STATS.track("make_raw");
568 +     for (const strong& s : inputs) {
569 +         s->pool->lock();

```

```

568 + }
569 + Tensors raw_inputs = uncheckedpoint(inputs);
570 + time_t pre = std::chrono::system_clock::now();
571 + auto raw_outputs = remat_f(raw_inputs);
572 + time_t post = std::chrono::system_clock::now();
573 + pool.auto_evict();
574 + base_compute_time_ += (post - pre).count();
575 + std::vector<intrusive_ptr<External>> outputs;
576 + std::vector<int> aliases;
577 + weaks weak_outputs;
578 + auto remat = intrusive_ptr<Rematerializer>::make(Unsafe(), remat_f,
    inputs, post - pre);
579 +
580 + for (const Tensor& t : raw_outputs) {
581 +     intrusive_ptr<AliasPool> alias_pool;
582 +     int alias = get_alias(raw_inputs, t);
583 +     if (alias == -1) {
584 +         auto m = memory(t);
585 +         alias_pool = intrusive_ptr<AliasPool>::make(Unsafe(), remat, m);
586 +         pool.add(alias_pool);
587 +     }
588 +     else {
589 +         alias_pool = inputs[alias]->pool;
590 +         if (alias_pool->head_remat) {
591 +             alias_pool->head_remat->compute_cost += (post - pre);
592 +         }
593 +     }
594 +     auto e = intrusive_ptr<External>::make(t, alias_pool, remat);
595 +     pool.exts.push_back(weak_intrusive_ptr<External>(e));
596 +     alias_pool->tensors.push_back(weak(e->value));
597 +     outputs.push_back(e);
598 +     aliases.push_back(alias);
599 +     weak_outputs.push_back(weak(outputs.back()->value));
600 + }
601 + remat->outputs = weak_outputs;
602 + for (size_t i = 0; i < inputs.size(); ++i) {
603 +     for (size_t j = 0; j < outputs.size(); ++j) {
604 +         if (!is_alias(raw_inputs[i], raw_outputs[j])) {
605 +             add_neighbor(inputs[i], outputs[j]->value);
606 +         }
607 +     }
608 + }
609 + for (const strong& s : inputs) {
610 +     s->pool->unlock();
611 + }
612 + return {outputs, aliases, post - pre, remat};
613 +}

```

```

614 +
615 +std::string from_time(duration_t t) {
616 + return std::to_string(std::chrono::nanoseconds(t).count());
617 +}
618 +
619 +Tensors CheckpointTensorImpl::make(const std::string& name,
620 +                                   const rematerialize_function_t& remat,
621 +                                   const Tensors& inputs) {
622 + STATS.track("CheckPointTensorImpl::make");
623 + Tensors checkpointed_inputs = try_checkpoint(inputs);
624 + auto input_size = checkpointed_inputs.size();
625 +
626 + strongs input_values;
627 + input_values.reserve(input_size);
628 +
629 + std::vector<std::string> args;
630 + args.reserve(input_size);
631 +
632 + for (const Tensor& t: checkpointed_inputs) {
633 +     auto* cpti = dynamic_cast<CheckpointTensorImpl*>
634 +         (t.unsafeGetTensorImpl());
635 +     TORCH_CHECK(cpti);
636 +     input_values.push_back(cpti->ref->value->value);
637 +     if (use_log_) {
638 +         args.push_back(cpti->counter_name());
639 +     }
640 + }
641 +
642 + auto ret = make_raw(remat, input_values);
643 +
644 + Tensors tensors;
645 + tensors.reserve(ret.outputs.size());
646 +
647 + for (const auto& t: ret.outputs) {
648 +     auto cp = Tensor(intrusive_ptr<CheckpointTensorImpl>::make(t));
649 +     tensors.push_back(cp);
650 + }
651 +
652 + if (use_log_) {
653 +     std::vector<std::string> res;
654 +     res.reserve(ret.outputs.size());
655 +
656 +     for (const auto& tensor : tensors) {
657 +         res.push_back(get_cpti(tensor)->counter_name());
658 +     }
659 +
660 +     DTRLogCall(res, name, args, from_time(ret.time));

```

```

660 +     for (size_t i = 0; i < tensors.size(); ++i) {
661 +         Tensor t = tensors[i];
662 +         auto cpti = get_cpti(t);
663 +         DTRLogMemory(cpti->counter_name(), cpti->ref->value->value->memory());
664 +         DTRLogAlias(cpti->counter_name(), ret.aliases[i]);
665 +     }
666 + }
667 +
668 + return tensors;
669 +}
670 +
671 +// TODO: check that mutated value does not have alias.
672 +void CheckpointTensorImpl::mutate(const std::string& name,
673 +    const mutate_function_t& mutate,
674 +    const Tensors& inputs,
675 +    const std::vector<size_t>& mutate_idx) {
676 +    auto remat = [=](const Tensors& t) -> Tensors {
677 +        Tensors new_input_values = t;
678 +        for (size_t idx: mutate_idx) {
679 +            new_input_values[idx] = t[idx].clone();
680 +        }
681 +        mutate(new_input_values);
682 +        return new_input_values;
683 +    };
684 +    Tensors checkpointed_inputs = try_checkpoint(inputs);
685 +    strongs input_values;
686 +    std::vector<std::string> args;
687 +    for (const Tensor& t: checkpointed_inputs) {
688 +        auto* cpti = dynamic_cast<CheckpointTensorImpl*>
689 +            (t.unsafeGetTensorImpl());
690 +        TORCH_CHECK(cpti);
691 +        input_values.push_back(cpti->ref->value->value);
692 +        if (use_log_) {
693 +            args.push_back(cpti->counter_name());
694 +        }
695 +    }
696 +    auto ret = make_raw(remat, input_values);
697 +    const auto& modified = ret.outputs;
698 +    for (size_t idx: mutate_idx) {
699 +        cell_from_tensor(inputs[idx])>value = modified[idx];
700 +    }
701 +    if (use_log_) {
702 +        DTRLogMutate(name, args, mutate_idx, from_time(ret.time));
703 +    }
704 +}
705 +
706 +void CheckpointTensorImpl::release_resources() {

```

```

706 +   if (use_log_) {
707 +       DTRLogRelease(counter_name());
708 +   }
709 +   ref.reset();
710 +}
711 +
712 +CheckpointTensorImpl::CheckpointTensorImpl(const Tensor& t) :
CheckpointTensorImpl(intrusive_ptr<External>::make(t)) {
713 +   pool.exts.push_back(weak_intrusive_ptr<External>(ref->value));
714 +}
715 +
716 +}

```

DELTA

```

1  diff -ru dtr_pytorch/aten/src/ATen/CheckpointTensorImpl.cpp
delta/aten/src/ATen/CheckpointTensorImpl.cpp
2  --- dtr_pytorch/aten/src/ATen/CheckpointTensorImpl.cpp      2024-04-28
16:09:10.546175266 +0800
3  +++ delta/aten/src/ATen/CheckpointTensorImpl.cpp            2024-04-09
14:45:52.849651606 +0800
4  @@ -1,6 +1,11 @@
5     #include <ATen/CheckpointTensorImpl.h>
6     #include <ATen/Logger.h>
7  +#include <ATen/cuda/CUDAContext.h>
8  +#include <ATen/cuda/CUDAEvent.h>
9     #include <c10/cuda/CUDACachingAllocator.h>
10  +#include <c10/cuda/CUDASTream.h>
11  +#include <c10/core/Allocator.h>
12
13     #include <chrono>
14     #include <string>
15  @@ -126,13 +131,17 @@
16     STATS.timers.push_back(stats);
17     }
18
19  -bool use_log_ = false;
20  -bool use_profile_ = false;
21  +bool use_log_ = true;
22  +bool use_profile_ = true;
23     long base_compute_time_ = 0;
24     long remat_compute_time_ = 0;
25     long search_time_ = 0;
26     long cost_time_ = 0;
27

```

```

28 +long total_count = 0;
29 +long evict_count = 0;
30 +long offload_count = 0;
31
32 CheckpointPool pool;
33 void CheckpointPool::add(const intrusive_ptr<AliasPool>& p) {
34     if (p->memory > 0 && (memory_count == 0 || !ignore_small_tensors || p-
35 >memory >= 0.01 * double(memory_sum/memory_count))) {
36 @@ -149,7 +158,11 @@
37     void CheckpointPool::auto_evict() {
38         STATS.track("CheckpointPool::auto_evict");
39         if (has_memory_budget) {
40 +         // if (!offload_queue.empty() && current_memory() < memory_budget) {
41 +         //     prefetch();
42 +         // }
43         while (current_memory() > memory_budget) {
44 +         // std::cout << "current memory is: " << current_memory() << std::endl;
45         evict();
46     }
47 @@ -162,7 +175,7 @@
48     // shrunk: either something has been evicted or the pools have gotten
49     smaller
50     bool shrunk = false;
51     int evict_idx = -1;
52 - double evict_cost = INFINITY;
53 + double filter_score = INFINITY;
54     time_t current_time = std::chrono::system_clock::now();
55     auto remove_from_aps = [&](size_t i) {
56         aps[i] = aps[aps.size() - 1];
57 @@ -170,6 +183,7 @@
58     };
59     std::uniform_int_distribution<> distrib(1, 1 * std::max(1, static_cast<int>
60 (std::sqrt(aps.size()))));
61     // sampling a random independent subset of all evictable tensors to find
62     the cheapest tensor to evict.
63 + // std::cout << "aps size: " << aps.size() << std::endl;
64     for (size_t i = 0; i < aps.size(); i++) {
65         auto cannot_evict = [&]() {
66             shrunk = true;
67 @@ -181,12 +195,15 @@
68         }
69         else if (ap_strong->ecn) {
70             cannot_evict();
71 -         }
72 +         }
73 +     }
74 +     else if (ap_strong->is_offloaded) {

```

```

71 +     cannot_evict();
72 + }
73     else {
74         if (ap_strong->evictable()) {
75             double cost = ap_strong->cost(current_time);
76 -             if (cost < evict_cost) {
77 -                 evict_cost = cost;
78 +             if (cost < filter_score) {
79 +                 filter_score = cost;
80                 evict_idx = i;
81             }
82         }
83 @@ -198,21 +215,78 @@
84         }
85     }
86 }
87 + // std::cout << "evict idx " << evict_idx << std::endl;
88     if (evict_idx == -1) {
89         TORCH_CHECK(shrunk);
90     } else {
91 +     auto release_aps = aps[evict_idx].lock();
92 +     double real_decision;
93 +     std::cout << "release_aps->swapped " << release_aps->swapped <<
std::endl;
94 +     if (release_aps->swapped) {
95 +         double _computed_cost = release_aps->compute_cost_func(current_time);
96 +         double _swap_cost = release_aps->swap_cost;
97 +         real_decision = _swap_cost / _computed_cost;
98 +     } else {
99 +         real_decision = release_aps->decision_func(current_time);
100 +     }
101 +     std::cout << "decision is " << real_decision << std::endl;
102     auto evict_from_idx = [&](size_t idx) {
103         auto ap_strong = aps[idx].lock();
104         TORCH_CHECK(ap_strong.defined());
105         ap_strong->evict();
106         remove_from_aps(evict_idx);
107     };
108 -     evict_from_idx(evict_idx);
109 +     auto offload_from_idx = [&](size_t idx){
110 +         auto ap_strong = aps[idx].lock();
111 +         TORCH_CHECK(ap_strong.defined());
112 +         ap_strong->offload();
113 +         remove_from_aps(evict_idx);
114 +     };
115 +     total_count += 1;
116 +     if (real_decision > 1) {

```



```

117 +     evict_from_idx(evict_idx);
118 +     evict_count += 1;
119 +     // std::cout << "evict aaaaa" << std::endl;
120 +     // offload_from_idx(evict_idx);
121 + } else {
122 +     // evict_from_idx(evict_idx);
123 +     offload_from_idx(evict_idx);
124 +     offload_count += 1;
125 +     // auto ap_strong = aps[evict_idx].lock();
126 +     // TORCH_CHECK(ap_strong.defined());
127 +     // remove_from_aps(evict_idx);
128 + }
129 }
130 + std::cout << "total count is: " << total_count << std::endl;
131 + std::cout << "evict count is: " << evict_count << std::endl;
132 + std::cout << "offload count is: " << offload_count << std::endl;
133 +
134 +     time_t post = std::chrono::system_clock::now();
135 +     search_time_ += (post - pre).count();
136 + }
137
138 +void CheckpointPool::prefetch() {
139 +    if (offload_queue.empty()) {
140 +        return;
141 +    }
142 +    strong front_operand = offload_queue.front();
143 +    if (front_operand) {
144 +        for (int i=0; i < prefetch_count; i++) {
145 +            auto total_memory = current_memory() + memory(front_operand->get());
146 +            if (total_memory < memory_budget) {
147 +                front_operand->reload();
148 +                offload_queue.pop();
149 +            }
150 +            if (offload_queue.empty()) {
151 +                break;
152 +            } else {
153 +                front_operand = offload_queue.front();
154 +            }
155 +        }
156 +    }
157 +    return;
158 +}
159 +
160 +CheckpointPool::CheckpointPool() { }
161
162 +namespace native {
163 @@ -292,6 +366,7 @@

```

```

164     }
165
166     void set_memory_budget(long budget) {
167 +     std::cout << "memory budget: " << budget << std::endl;
168         pool.memory_budget = budget;
169         pool.has_memory_budget = true;
170     }
171 @@ -341,17 +416,51 @@
172
173     }
174
175 +void CheckpointTensorCell::reload() {
176 +     STATS.track("AliasPool::reload");
177 +     onGPU = true;
178 +     offloaded = false;
179 +     at::cuda::CUDASTream stream = at::cuda::getCurrentCUDASTream();
180 +     cudaMemcpyKind kind = cudaMemcpyHostToDevice;
181 +     // size_t res = input->cpuStorage->nbytes();
182 +     c10::Allocator* allocator = at::cuda::getCUDADeviceAllocator();
183 +     // auto ptr = std::make_shared<Storage>(Storage::use_byte_size_t(), res,
184 +     // allocator, false);
185 +     // cudaMemcpyAsync(ptr->data_ptr().get(), input->cpuStorage->
186 +     // data_ptr().get(), res, kind, stream);
187 +     c10::Device device = optional_device_.value();
188 +     // std::cout << "device: " << device << std::endl;
189 +     Tensor gpuTensor = cpuTensor.to(device);
190 +     cpuTensor.reset();
191 +     t = std::make_unique<Tensor>(gpuTensor.detach());
192 +     pool->onGPU = true;
193 +     pool->is_offloaded = false;
194 +     at::pool.aps.push_back(weak_intrusive_ptr<AliasPool>(pool));
195 +     // how to set tensor device and on undefined tensor error
196 +     // input->cpuStorage.reset();
197 +     // std::cout << "Reload finished " << std::endl;
198 +     // return gpuTensor;
199 + }
200 +
201 + [[inline]]
202 + Tensor uncheckpoint(const strong& input) {
203 +     return input->get();
204 + }
205
206 -Tensors uncheckpoint(const strongs& inputs) {
207 +Tensors uncheckpoint(strongs& inputs) {
208     STATS.track("uncheckpoint");
209     Tensors ret;
210     ret.reserve(inputs.size());

```

```

209 - for (const strong& input : inputs) {
210 + for (strong& input : inputs) {
211     // inlined manually
212 + // if (!input->onGPU && input->offloaded) {
213 + //     input->reload();
214 + //     ret.push_back(input->get());
215 + // }
216 + // else if (!input->onGPU && input->evicted) {
217 + //     ret.push_back(input->get());
218 + // } else {
219 + //     ret.push_back(input->get());
220 + // }
221 + // reload(input);
222     ret.push_back(input->get());
223 }
224 return ret;
225 @@ -384,6 +493,7 @@
226     TORCH_CHECK(lock_count == 0);
227     TORCH_CHECK(!is_evicted);
228     is_evicted = true;
229 + onGPU = false;
230     for (const weak& w : tensors) {
231         if (auto cell = w.lock()) {
232             cell->evict();
233 @@ -391,19 +501,82 @@
234     }
235 }
236
237 +void AliasPool::offload() {
238 +    STATS.track("AliasPool::offload");
239 +    // TORCH_CHECK(!ecn);
240 +    // ecn = head_remat->get_ecn();
241 +    // auto ecns = neighbor_ecn();
242 +    // for (const auto& necn : ecns) {
243 +    //     merge<CheckpointInfo>(merge_cpi, ecn, necn);
244 +    // }
245 +    TORCH_CHECK(memory > 0);
246 +    TORCH_CHECK(lock_count == 0);
247 +    TORCH_CHECK(!is_offloaded);
248 +    at::cuda::CUDASTream stream = at::cuda::getCurrentCUDASTream();
249 +    cudaMemcpyKind kind = cudaMemcpyDeviceToHost;
250 +    c10::Allocator* allocator = at::getCPUALlocator();
251 +    onGPU = false;
252 +    is_offloaded = true;
253 +    swapped = true;
254 +    for (const weak& w : tensors) {
255 +        if (auto cell = w.lock()) {

```

```

256 +     auto& storage = cell->t->storage();
257 +     size_t res = storage.nbytes();
258 +     // auto dtype_ = cell->t->dtype();
259 +     time_t pre = std::chrono::system_clock::now();
260 +     // cell->cpuStorage = std::make_shared<Storage>
    (Storage::use_byte_size_t(), res, allocator, false);
261 +     // cudaMemcpyAsync(cell->cpuStorage->data_ptr().get(),
    storage.data_ptr().get(), res, kind, stream);
262 +     cell->cpuTensor = cell->t->to(DeviceType::CPU);
263 +     time_t post = std::chrono::system_clock::now();
264 +     cell->t.reset();
265 +     cell->swap_cost = (post - pre).count();
266 +     cell->onGPU = false;
267 +     cell->offloaded = true;
268 +     // std::cout << "offloaded tensor device " << cell-
    >optional_device_.value() << std::endl;
269 +     // pool.offload_queue.push(cell);
270 + }
271 + }
272 + // std::cout << "offload finished " << std::endl;
273 +}
274 +
275 double AliasPool::cost(time_t current_time) {
276     time_t pre = std::chrono::system_clock::now();
277     auto cpi = head_remat->get_cpi();
278     auto ecns = neighbor_ecn();
279 + // for (const auto& necn : ecns) {
280 + //     cpi = merge_cpi(cpi, get_t(necn));
281 + // }
282 + auto ret = cpi.cost(memory, (current_time - last_used_time).count());
283 + time_t post = std::chrono::system_clock::now();
284 + cost_time_ += (post - pre).count();
285 + return ret;
286 +}
287 +
288 +double AliasPool::compute_cost_func(time_t current_time) {
289 + time_t pre = std::chrono::system_clock::now();
290 + auto cpi = head_remat->get_cpi();
291 + auto ecns = neighbor_ecn();
292 + for (const auto& necn : ecns) {
293 +     cpi = merge_cpi(cpi, get_t(necn));
294 + }
295 - auto ret = cpi.cost(memory, (current_time - last_used_time).count());
296 + auto ret = cpi.compute_cost_func(memory, (current_time -
    last_used_time).count());
297 + time_t post = std::chrono::system_clock::now();
298 + cost_time_ += (post - pre).count();

```

```

299     return ret;
300 }
301
302 +double AliasPool::decision_func(time_t current_time) {
303 +   time_t pre = std::chrono::system_clock::now();
304 +   auto cpi = head_remat->get_cpi();
305 +   // auto ecns = neighbor_ecn();
306 +   // for (const auto& necn : ecns) {
307 +   //     cpi = merge_cpi(cpi, get_t(necn));
308 +   // }
309 +   auto ret = cpi.fake_decision(memory, (current_time -
last_used_time).count());
310 +   time_t post = std::chrono::system_clock::now();
311 +   return ret;
312 +}
313 +
314 void External::release_resources() {
315     value->pool->release_external();
316     value.reset();
317 @@ -478,6 +651,20 @@
318     }
319 }
320
321 +void AliasPool::set_not_offloaded(const intrusive_ptr<AliasPool>& self) {
322 +   if (unlikely(is_offloaded)) {
323 +       STATS.track("AliasPool::set_not_offloaded(inside)");
324 +       is_offloaded = false;
325 +       if (ecn) {
326 +           TORCH_CHECK(head_remat);
327 +           auto cpi = get_t(ecn);
328 +           update_t(ecn, CheckpointInfo(cpi.compute_cost - head_remat-
>compute_cost));
329 +           ecn.reset();
330 +       }
331 +       pool.add(self);
332 +   }
333 +}
334 +
335 void CheckpointTensorCell::fill(const Tensor& t) {
336     STATS.track("CheckpointTensorCell::fill");
337     if (!(this->t)) {
338 @@ -507,7 +694,7 @@
339
340 void CheckpointTensorImpl::shallow_copy_from(const
c10::intrusive_ptr<TensorImpl>& impl) {
341     STATS.track("CheckpointTensorCell::shallow_copy_from");
342 -   TORCH_CHECK(impl->key_set().has(DispatchKey::CheckpointTensorId));

```

```

343 + TORCH_CHECK(impl->key_set().has(DispatchKey::Checkpoint));
344     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(impl.get());
345     TORCH_CHECK(cpti != nullptr);
346     ref->value = cpti->ref->value;
347 @@ -555,7 +742,7 @@
348     // however, we have to stitch the two vectors together to pass it in remat.
349     // the size_t in constants decide the location to stitch them in, while
350     input_values fill in the rest.
351     MakeRawResult make_raw(const rematerialize_function_t& remat_f,
352 -                          const strongs& inputs) {
353 +                          strongs& inputs) {
354     STATS.track("make_raw");
355     for (const strong& s : inputs) {
356         s->pool->lock();
357 @@ -563,6 +750,7 @@
358     Tensors raw_inputs = uncheckpoint(inputs);
359     time_t pre = std::chrono::system_clock::now();
360     auto raw_outputs = remat_f(raw_inputs);
361 +
362     time_t post = std::chrono::system_clock::now();
363     pool.auto_evict();
364     base_compute_time_ += (post - pre).count();
365 @@ -633,7 +821,6 @@
366     }
367
368     auto ret = make_raw(remat, input_values);
369 -
370     Tensors tensors;
371     tensors.reserve(ret.outputs.size());

```

!DELTA更新

aten/src/ATen/native/TensorConversions.cpp 新增 to_backward 函数

```

1 diff -ru dtr_pytorch/aten/src/ATen/native/TensorConversions.cpp
2   delta/aten/src/ATen/native/TensorConversions.cpp
3 --- dtr_pytorch/aten/src/ATen/native/TensorConversions.cpp      2024-04-28
4   16:07:35.193886934 +0800
5 +++ delta/aten/src/ATen/native/TensorConversions.cpp      2024-04-09
6   14:45:52.873656278 +0800
7 @@ -93,6 +93,10 @@
8
9     return to_impl(self, options.memory_format(optional_memory_format),
10    non_blocking, copy);

```

```

6     }
7
8     +Tensor to_backward(const Tensor& grad, const Tensor& input) {
9     +     return grad;
10    +}
11    +
12    Tensor to_dense_backward(const Tensor& grad, const Tensor& input_) {
13        AT_ASSERT(input_.layout() != c10::kStrided);
14        if (input_.layout() == c10::kSparse) {

```

[暂不知作用]DELTA修改

aten/src/ATen/native/LinearAlgebra.cpp

```

1  diff -ru dtr_pytorch/aten/src/ATen/native/LinearAlgebra.cpp
    delta/aten/src/ATen/native/LinearAlgebra.cpp
2  --- dtr_pytorch/aten/src/ATen/native/LinearAlgebra.cpp      2024-04-28
    16:07:35.189885885 +0800
3  +++ delta/aten/src/ATen/native/LinearAlgebra.cpp            2024-04-09
    14:45:52.869655498 +0800
4  @@ -598,7 +598,6 @@
5      } else if (dim_tensor1 >= 3 && (dim_tensor2 == 1 || dim_tensor2 == 2)) {
6          // optimization: use mm instead of bmm by folding tensor1's batch into
7          // its leading matrix dimension.
8      -
9          Tensor t2 = dim_tensor2 == 1 ? tensor2.unsqueeze(-1) : tensor2;
10         auto size1 = tensor1.sizes();
11         auto size2 = t2.sizes();
12     @@ -665,9 +664,10 @@
13         tensor2_bmm_view.insert(tensor2_bmm_view.end(), {m2, p});
14
15         // flatten expanded batches
16     -   Tensor tensor1_expanded =
17         tensor1.expand(tensor1_expand_size).contiguous().view(tensor1_bmm_view);
17     -   Tensor tensor2_expanded =
18         tensor2.expand(tensor2_expand_size).contiguous().view(tensor2_bmm_view);
18     -
19     +   // Tensor tensor1_expanded =
20         tensor1.expand(tensor1_expand_size).contiguous().view(tensor1_bmm_view);
21     +   Tensor tensor1_expanded =
22         tensor1.expand(tensor1_expand_size).reshape(tensor1_bmm_view);
21     +   // Tensor tensor2_expanded =
22         tensor2.expand(tensor2_expand_size).contiguous().view(tensor2_bmm_view);

```

```

22 +   Tensor tensor2_expanded =
    tensor2.expand(tensor2_expand_size).reshape(tensor2_bmm_view);
23     // reshape batches back into result
24     std::vector<int64_t> output_shape(expand_batch_portion);
25     if (dim_tensor1 > 1) {
26 @@ -679,7 +679,6 @@
27
28     Tensor output = has_out ? at::_unsafe_view(at::bmm_out(out,
    tensor1_expanded, tensor2_expanded), output_shape)
29         :
    at::_unsafe_view(tensor1_expanded.bmm(tensor2_expanded), output_shape);
30 -
31     return has_out ? out.set_(output) : output;
32 }
33
34 @@ -1151,7 +1150,6 @@
35     1.461661507209034e+00, // deg 12
36     3.010066362817634e+00 // deg 18
37 };
38 -
39     return mexp_impl<float>(a_3d, thetas_float,
    compute_highest_degree_approx)
40     .view(a.sizes());
41 }
42 @@ -1164,7 +1162,6 @@
43     2.996158913811580e-01, // deg 12
44     1.090863719290036e+00 // deg 18
45 };
46 -
47     return mexp_impl<double>(a_3d, thetas_double,
    compute_highest_degree_approx)
48     .view(a.sizes());
49 }

```

笔记

1. contiguous() 和 view() :

- contiguous() : 确保张量在内存中是连续的。很多操作（比如 view()）要求张量是连续的，这样才能在内存中进行正确的重塑（reshape）。
- view() : 改变张量的形状，但要求原始张量是连续的。view() 只是对原始张量的一个视图，因此它不会改变底层数据，只是改变了数据的视图。

2. reshape() :

- `reshape()` : 也可以改变张量的形状, 但它会更灵活, 如果需要, 它可以返回一个新的张量, 即使原始张量不是连续的。 `reshape()` 也会尝试返回一个视图, 但如果不能, 它会返回一个新的张量。